Project No: A2285-251

Hardware-Software Co-Design For Large Language Model

Submitted by: Yap Chen Kit

Supervisor: Professor Chang Chip Hong

Co-supervisor: Koca Nazim Altar

School of Electrical and Electronic Engineering

A final year project report presented to Nanyang Technological University
in partial fulfilment of the requirements for the
Degree of Bachelor of Engineering

2026

Contents

Abstract **v**

Acknowledgement **vi**

Acronyms **vii**

Symbols **ix**

List of Figures **ix**

List of Tables **x**

1 Introduction **1**
1.1 Motivations 1
1.2 Objectives and Scope 2
1.3 Organisation of Report 2

2 Literature Review **3**
2.1 Transformer Model Architecture 3
2.2 TinyLlama Model Architecture 4
2.2.1 Token Embeddings 5
2.2.2 Rotary Positional Embedding 6
2.2.3 RMSNorm 7
2.2.4 Grouped-Query Self-Attention 7
2.2.5 Softmax 9
2.2.6 SwiGLU Feed-Forward Network 10
2.3 Efficient Attention Mechanisms 11
2.3.1 Sparse Transformer 11
2.3.2 BlockBERT 12
2.3.3 Performer 13
2.3.4 Linear Transformer 13
2.4 FPGA Acceleration of Neural Networks 14
2.5 Quantisation for Efficient Inference 14
2.6 HLS-Based Hardware Design 14

2.6.1 Pipelining and the Initiation Interval 14
2.6.2 Scaling Transformations: Unrolling and Dataflow 15
2.6.3 Memory Transformations 16

3 System Architecture and Design 17
3.1 Top-Level System Overview 17
3.2 Hardware/Software Partitioning 18
3.3 Memory Architecture 18
3.4 Quantisation Scheme 20

4 Implementation 22
4.1 RMSNorm 23
4.1.1 Implementation 23
4.1.2 C-Simulation Results 24
4.1.3 HLS Synthesis Results 25
4.1.4 Full TinyLlama Scale 27
4.2 Quantisation 31
4.2.1 Implementation 31
4.2.2 C-Simulation Results 32
4.2.3 HLS Synthesis Results 33
4.2.4 Full TinyLlama Scale 35
4.3 Dequantisation 38
4.3.1 Implementation 38
4.3.2 C-Simulation Results 39
4.3.3 HLS Synthesis Results 40
4.3.4 Full TinyLlama Scale 41
4.4 Matrix–Vector Multiplication 44
4.4.1 Implementation 44
4.4.2 C-Simulation Results 46
4.4.3 HLS Synthesis Results 46
4.4.4 Full TinyLlama Scale: Tiled Implementation 48
4.5 Softmax 52
4.5.1 Implementation 52
4.5.2 C-Simulation Results 54
4.5.3 HLS Synthesis Results 55
4.5.4 Iteration 3: Sum Loop Unroll 58
4.5.5 Iteration 4: I-BERT Integer Softmax 60
4.5.6 Iteration 6: Full TinyLlama Scale 63

4.6 Grouped-Query Attention 67
4.6.1 Baseline: Floating-Point Attention 67
4.6.2 Refined: Per-Head INT8 Attention 72
4.6.3 Iteration 3: INT8 KV Cache 77
4.6.4 Iteration 4: I-BERT Integer Softmax Attention 81
4.6.5 Iteration 6: Full TinyLlama Scale 84
4.7 SwiGLU Feed-Forward Network 89
4.7.1 Implementation 89
4.7.2 C-Simulation Results 90
4.7.3 HLS Synthesis Results 91
4.7.4 Full TinyLlama Scale 93
4.8 Top-Level Forward Function 97
4.8.1 Baseline Prototype (`forward.cpp`) 97
4.8.2 Full-Scale Kernel (`FFN_update3.cpp`) 98
4.8.3 Persistent Omissions 98

5 Results and Discussion 99
5.1 Experimental Setup 99
5.2 HLS Synthesis Results 100
5.2.1 Full Forward Kernel 100
5.2.2 Latency Breakdown 101
5.3 Functional Verification 102
5.3.1 Primitive-Level C-Simulation 102
5.3.2 End-to-End Stress Test 103
5.4 Discussion 105
5.4.1 Bottleneck Analysis 105
5.4.2 Accuracy Characterisation 105
5.4.3 Design Limitations 105

6 Conclusion and Future Work 106
6.1 Conclusion 106
6.2 Recommendations for Future Work 106

Reflection on Learning Outcome Attainment 108

Acknowledging/Declaring the Use of GAI 109

References 112

A HLS Kernel Source Code **113**

A.1 `forward.cpp` — Baseline Prototype 113

A.2 `FFN_update3.cpp` — Full-Scale Kernel 116

Abstract

Small-scale large language models (LLMs) such as TinyLlama-1.1B have emerged as a compelling class of compact transformer models that deliver meaningful natural language understanding capabilities within a fraction of the memory footprint of frontier models such as GPT-4. Yet even at the billion-parameter scale, these models impose substantial computational demands that limit their deployment in embedded and edge environments.

Currently, these language models are mostly deployed on Graphics Processing Units (GPUs). However, GPU inference is energy-intensive and memory-bandwidth-bound, particularly during the autoregressive decode phase, since the time and memory complexity of self-attention are quadratic in sequence length.

Field-programmable gate arrays (FPGAs) offer a compelling alternative to GPU-based inference by providing reconfigurable hardware with low power consumption and predictable latency. However, mapping a billion-parameter transformer to resource-constrained FPGA fabric presents significant challenges in quantisation, memory tiling, and hardware-software partitioning.

This project begins with a comprehensive profiling of an RTX 4060 Laptop GPU running PyTorch inference of the LLaMA 2-based TinyLlama-1.1B model to identify bottlenecks and measure the proportion of wall-clock time consumed by the self-attention and feed-forward layers during the forward pass. Based on these results, an FPGA-based accelerator is proposed to target the identified bottlenecks.

This project implements and optimises the inference pipeline of TinyLlama-1.1B on the Xilinx ZCU102 (`xczu9eg-ffvb1156-2-e`) FPGA platform using Vitis High-Level Synthesis (HLS). The core transformer components, including RMSNorm, matrix–vector multiplication, multi-head attention, and the SwiGLU feed-forward network, were implemented as quantised INT8 HLS kernels.

HLS synthesis of the complete forward kernel on the ZCU102 at a 150 MHz target clock produces a per-step latency of 1.07–1.37 billion cycles (7.15–9.10 seconds), with resource utilisation of 73% LUT, 18% FF, 35% BRAM, and 28% DSP, all within the ZCU102 device budget. C-simulation verified all sixteen kernel primitives against floating-point software references with zero errors, confirming functional correctness of the INT8 quantised datapath.

Acknowledgement

I would like to express my sincere gratitude to Professor Chang Chip Hong for being my supervisor and giving me multiple chances to explore different research areas since my second year until my final year of undergraduate studies, and allowing me to pursue this topic as my FYP.

I would also like to thank my co-supervisor, Koca Nazim Altar, for supporting me during my final year project, in addition to his full-time job and his PhD tasks. His guidance have helped me overcome technical and mental roadblocks with his kindness and expertise. Altar have provided me valuable knowledge that helped bring this project to fruition.

Yap Chen Kit
May 2026

Acronyms

AXI	Advanced eXtensible Interface
BRAM	Block Random Access Memory
DDR	Double Data Rate (SDRAM)
DMA	Direct Memory Access
DRAM	Dynamic Random Access Memory
DSP	Digital Signal Processor
FF	Flip-Flop
FFN	Feed-Forward Network
FIFO	First-In, First-Out
FP16	16-bit Floating Point
FP32	32-bit Floating Point
FPGA	Field-Programmable Gate Array
GELU	Gaussian Error Linear Unit
GPU	Graphics Processing Unit
GQA	Grouped-Query Attention
HLS	High-Level Synthesis
II	Initiation Interval
INT32	32-bit Integer
INT8	8-bit Integer
IP	Intellectual Property (core)
KV	Key-Value
LLM	Large Language Model
LUT	Look-Up Table
MHA	Multi-Head Attention
MLP	Multi-Layer Perceptron
PL	Programmable Logic
PS	Processing System
PTQ	Post-Training Quantisation
QKV	Query, Key, Value
ReLU	Rectified Linear Unit
RMSNorm	Root Mean Square Layer Normalisation

RoPE	Rotary Positional Embedding
RTL	Register-Transfer Level
SoC	System-on-Chip
SwiGLU	Swish-Gated Linear Unit
XRT	Xilinx Runtime
ZCU102	Zynq UltraScale+ MPSoC Evaluation Board (ZCU102)

Symbols

Symbol	*Description*
d_{model}	Hidden dimension of the model (2048 for TinyLlama-1.1B)
d_{ff}	Feed-forward network intermediate dimension
d_k	Dimension of each attention head
N_L	Number of transformer layers
H	Number of query attention heads
G	Number of KV head groups (GQA)
V	Vocabulary size
T	Sequence length (number of tokens)
W_Q, W_K, W_V	Query, Key, Value projection weight matrices
W_O	Output projection weight matrix
Q, K, V	Query, Key, Value activation matrices
softmax	Softmax normalisation function
s	Quantisation scale factor
s_g	Per-group quantisation scale factor for group g
GS	Quantisation group size
$\mathscr{G}_g$	Index set of quantisation group g
$\hat{x}$	Quantised representation of variable x
I	Initiation interval of a pipelined loop (clock cycles)
L	Pipeline latency (clock cycles)
N	Number of loop iterations
C	Total cycle count: $C = L + I(N-1)$
f_{clk}	Clock frequency (MHz)
N_{LUT}	Number of look-up tables consumed
N_{DSP}	Number of DSP blocks consumed

List of Figures

2.1 Connectivity matrices for full attention (a), Sparse Transformer with strided pattern (b), and Sparse Transformer with fixed pattern (c). In (b) and (c), full connectivity between all elements is preserved when the two attention heads are computed sequentially, while sparsity in each individual head leads to significantly reduced computation [11]. . 12

2.2 Pipeline characteristics showing latency L, initiation interval I, and total cycle count $C = L + I(N-1)$ for N inputs [18]. 15

2.3 Horizontal unrolling, vertical unrolling, and dataflow as means to increase parallelism. Rectangles represent buffer space such as registers or on-chip RAM. (a) Before any transformation. (b) Horizontal unrolling: four independent inputs processed in parallel. (c) Vertical unrolling: one input combined with multiple buffered values. (d) Dataflow: input or partial results streamed through a pipeline rather than broadcast [18]. 15

List of Tables

2.1 TinyLlama-1.1B architectural hyperparameters [2]. 4

3.1 Memory hierarchy of the TinyLlama inference accelerator on the ZCU102. Weight and KV cache sizes are after INT8 quantisation. On-chip buffers are reused across all 22 layers. 20

4.1 Prototype and full TinyLlama-1.1B configuration parameters used across kernel development. 22
4.2 C-simulation output for `rmsnorm`. All eight elements match the software reference. 25
4.3 Timing summary for `rmsnorm_top` at the 150 MHz (6.667 ns) target clock. 25
4.4 Top-level latency summary for `rmsnorm_top` at 150 MHz. 25
4.5 Resource utilisation of the `rmsnorm` function synthesised at $S = 8$ on the ZCU102. Core-only figures exclude the AXI master adapter. . . . 26
4.6 Latency breakdown of `rmsnorm` ($S = 8$) by internal pipeline stage at 150 MHz. 26
4.7 Timing summary for `rmsnorm_update3_top` at the 150 MHz target clock. 29
4.8 Top-level latency summary for `rmsnorm_update3_top` at 150 MHz. . . 29
4.9 Resource utilisation of `rmsnorm` at full scale ($S = 2048$) on the ZCU102. Core-only figures exclude the AXI master adapter. 29
4.10 Per-stage latency breakdown of `rmsnorm` at full scale ($S = 2048$) at 150 MHz. 30
4.11 C-simulation output for `quantize`. All INT8 values and both scale factors match the software reference. 33
4.12 Timing summary for `quantize_top` at 150 MHz. 33
4.13 Top-level latency summary for `quantize_top` at 150 MHz. 34
4.14 Resource utilisation of `quantize` synthesised at $S = 8$ on the ZCU102. The core-only column excludes the AXI master adapter. 34
4.15 Latency breakdown of `quantize` ($S = 8$, $GS = 4$) by internal pipeline stage at 150 MHz. 34

4.16 Timing summary for quantize_update3_top at the 150 MHz target clock. 36
4.17 Top-level latency summary for quantize_update3_top at 150 MHz. . 37
4.18 Resource utilisation of quantize at full scale ($S = 2048$, GS = 64, 32 groups) on the ZCU102. Core-only figures exclude the AXI master adapter. 37
4.19 Per-stage latency breakdown of quantize at full scale ($S = 2048$) at 150 MHz. 37
4.20 C-simulation output for dequantize. All eight dequantised float values match the software reference with zero absolute error. 39
4.21 Timing summary for dequantize_top at 150 MHz. 40
4.22 Top-level latency summary for dequantize_top at 150 MHz. 40
4.23 Resource utilisation of dequantize synthesised at $S = 8$ on the ZCU102. The core-only column excludes the AXI master adapter. 40
4.24 Latency breakdown of dequantize ($S = 8$, $GS = 4$) at 150 MHz. . . . 40
4.25 Timing summary for dequantize_update3_top at the 150 MHz target clock. 42
4.26 Top-level latency summary for dequantize_update3_top at 150 MHz. 42
4.27 Resource utilisation of dequantize at full scale ($S = 2048$, GS = 64) on the ZCU102. Core-only figures exclude the AXI master adapter (gmem0_m_axi). 43
4.28 Per-stage latency breakdown of dequantize at full scale ($S = 2048$) at 150 MHz. 43
4.29 C-simulation output for matmul. All eight output elements match the software reference within tolerance. 46
4.30 Timing summary for matmul_top at the 150 MHz (6.667 ns) target clock. 47
4.31 Top-level latency summary for matmul_top at 150 MHz. 47
4.32 Resource utilisation of matmul synthesised at $N = D = 8$ on the ZCU102. Core-only figures exclude the AXI master adapter. 47
4.33 Latency breakdown of matmul ($N = D = 8$, $GS = 4$) at 150 MHz. . . . 47
4.34 Timing summary for matmul_update3_top at the 150 MHz target clock. 50
4.35 Top-level latency summary for matmul_update3_top at 150 MHz. . . 50
4.36 Resource utilisation of the tiled matmul at full scale ($N = 2048$, $D = 5632$) on the ZCU102. Core-only figures exclude the AXI master adapter. 51
4.37 Latency breakdown of the tiled matmul ($N = 2048$, $D = 5632$, TILE_D = 4) at 150 MHz. 51

4.38 C-simulation output for softmax. All eight elements match the software reference. 55
4.39 Timing summary for softmax_top at the 150 MHz (6.667 ns) target clock. 55
4.40 Top-level latency summary for softmax_top at 150 MHz. The minimum corresponds to size = 1 and the maximum to size = MAXSIZE = 16. 56
4.41 Resource utilisation of softmax synthesised at MAXSIZE = 16 on the ZCU102. The core-only column excludes the AXI master adapter and S-AXILite control interface. 56
4.42 Per-loop latency breakdown of softmax (MAXSIZE = 16) at 150 MHz. 56
4.43 Timing summary for softmax_kvint8_top at the 150 MHz target clock. 58
4.44 Per-loop latency breakdown of the iteration 3 softmax (MAXSIZE = 16) at 150 MHz. 59
4.45 Resource utilisation comparison between the iteration 1 baseline and the iteration 3 softmax designs, both at MAXSIZE = 16 on the ZCU102. 59
4.46 C-simulation output for softmax_ibert. All eight output probabilities match the software reference. 61
4.47 Timing summary for softmax_ibert_top at the 150 MHz target clock. 62
4.48 Top-level latency summary for softmax_ibert_top at 150 MHz. . . . 62
4.49 Resource utilisation of softmax_ibert synthesised at MAX_SIZE = 16 on the ZCU102. 62
4.50 Latency breakdown of softmax_ibert (MAX_SIZE = 16) at 150 MHz. 62
4.51 Timing summary for softmax_ibert_update3_top at the 150 MHz target clock. 65
4.52 Top-level latency summary for softmax_ibert_update3_top at 150 MHz. 65
4.53 Resource utilisation of softmax_ibert at full scale (MAX_SIZE = 2048) on the ZCU102. Core-only figures exclude the AXI master adapter (gmem0_m_axi). 65
4.54 Per-stage latency breakdown of softmax_ibert at full scale (MAX_SIZE = 2048) at 150 MHz. 66
4.55 C-simulation output for the baseline float attention kernel (attn_baseline_top) at pos = 3, loff = 0. All eight elements of the weighted value accumulation xb match the software reference with zero absolute error. 69
4.56 Timing summary for attn_baseline_top at the 150 MHz (6.667 ns) target clock. 70

4.57 Top-level latency summary for attn_baseline_top at 150 MHz. The minimum corresponds to pos = 0 (one attended position) and the maximum to pos = seq_len − 1 = 15 (sixteen attended positions). 70
4.58 Resource utilisation of the baseline float attention block synthesised at prototype scale on the ZCU102. Core-only figures exclude the AXI master adapter. 70
4.59 Per-pipeline-module latency breakdown of the baseline float attention block at 150 MHz, for one head at maximum context length (pos = 15). The outer multihead_attention loop over n_heads = 2 runs sequentially, so the per-head cost is incurred twice. 71
4.60 Comparison of xb outputs from the baseline float attention and the refined per-head INT8 attention for identical inputs at pos = 3. The absolute difference and relative error in each element reflect the quantisation error introduced by the INT8 arithmetic path. Both kernels passed their respective csim runs with zero errors against their own software references. 75
4.61 Resource utilisation comparison between the baseline float attention and the refined per-head INT8 attention at prototype scale on the ZCU102. 76
4.62 Per-pipeline-module latency breakdown of the refined per-head INT8 attention block at 150 MHz, for one head at maximum context (pos = 256, MAXSIZE = 257). The outer multihead_attention loop over n_heads = 2 runs sequentially, giving a worst-case total of approximately 9,332 cycles. 76
4.63 C-simulation xb outputs for all three attention iterations at identical inputs (pos = 3, loff = 0). Iteration 3 outputs are closer to the float baseline than iteration 2, reflecting the higher fidelity of pre-quantised group-scale storage compared to per-position on-the-fly quantisation. . 79
4.64 Resource utilisation across all three attention iterations on the ZCU102 at 150 MHz. 80
4.65 Per-pipeline-module latency breakdown of the iteration 3 INT8-KV attention block at 150 MHz, for one head at pos = 15. The sum stage reflects the unrolled softmax from section 4.5.4. 80
4.66 C-simulation xb outputs for all four attention iterations at pos = 3, loff = 0. Iteration 4 introduces additional approximation error relative to iteration 3 from the coarse LUT exponential, but all four csim runs passed with zero errors against their respective software references. 82

4.67 Resource comparison between iteration 3 and iteration 4 attention implementations on the ZCU102. 83
4.68 Per-module latency breakdown for the iteration 4 attention kernel at 150 MHz. The minimum corresponds to pos = 0 (one attended position) and the maximum to pos = 15 (MAX_SIZE = 16). 83
4.69 C-simulation xb outputs at pos = 3 for iteration 4 (prototype scale, n_heads = 2) and iteration 5 (full scale, n_heads = 32). The output values differ because the full-scale kernel processes 32 heads at head_size = 64 rather than 2 heads at head_size = 4, producing a different output range from the same KV cache input pattern. 87
4.70 Timing summary for attn_update3_top at the 150 MHz target clock. 87
4.71 Top-level latency summary for attn_update3_top at 150 MHz. The minimum corresponds to pos = 0 (one attended position per head) and the maximum to pos = 2047 (full context length). 87
4.72 Resource utilisation of attn_update3_top at full TinyLlama scale on the ZCU102. Core-only figures exclude the AXI master adapters. . . . 88
4.73 Per-module latency breakdown for attn_update3_top at 150 MHz. . 88
4.74 C-simulation output for ffn_baseline. All eight elements agree with the software reference within the INT8 quantisation tolerance. 90
4.75 Timing summary for ffn_baseline_top at the 150 MHz target clock. 91
4.76 Top-level latency summary for ffn_baseline_top at 150 MHz. 91
4.77 Resource utilisation of ffn_baseline synthesised at prototype dimensions (dim = 8, hidden_dim = 32) on the ZCU102. Core-only figures exclude the two AXI master adapters (gmem0_m_axi and gmem1_m_axi). 91
4.78 Per-stage latency breakdown of ffn_baseline_top at 150 MHz. . . . 92
4.79 C-simulation output for ffn_update3 (first 8 of 2048 elements). All absolute errors are well within the 0.05 tolerance. The small output magnitudes reflect three sequential quantised matrix multiplications with weight scale factors of 0.03–0.05. 94
4.80 Timing summary for ffn_update3_top at the 150 MHz target clock. . 95
4.81 Top-level latency summary for ffn_update3_top at 150 MHz. 95
4.82 Resource utilisation of ffn_update3_top on the ZCU102 at full TinyLlama scale. Core-only figures exclude the four AXI master adapters (gmem0–gmem3, 4 BRAM, 512 FF, 580 LUT each) and the S-AXILite control block (673 FF, 1,194 LUT). 95
4.83 Per-stage latency breakdown of ffn_update3_top at 150 MHz. 96

5.1 Timing summary for the `forward()` kernel at the 150 MHz target clock. 100
5.2 Resource utilisation of the `forward()` kernel after HLS synthesis on the ZCU102 (`xczu9eg-ffvb1156-2-e`). Core-only figures exclude the four AXI master adapters. 100
5.3 Resource utilisation of the key synthesised submodule instances within the `forward()` kernel. 101
5.4 Top-level latency summary for the `forward()` kernel at the 150 MHz target clock. The minimum corresponds to `pos = 0` (one attended position per head) and the maximum to `pos = 2047` (full context length). . 101
5.5 Per-operation latency for one transformer layer at 150 MHz. 102
5.6 C-simulation results for all kernel primitives at both prototype and full TinyLlama scale. 103
5.7 End-to-end relative error between the `forward()` kernel and the float software reference across five token positions. 104

Chapter 1 Introduction

1.1 Motivations

Large language models (LLMs) have demonstrated excellent capabilities across a wide range of natural language understanding and generation tasks, but their deployment in resource-constrained environments remains a significant challenge [1]. State-of-the-art large language models (LLMs) such as GPT-4 and LLaMA require tens or hundreds of gigabytes of memory and specialised GPU clusters to achieve practical inference throughput, making them inaccessible for embedded, edge, and real-time applications.

The llama 2-based TinyLlama-1.1B model [2] represents a class of small-scale LLMs that seek to balance capability with deployability. With only 1.1 billion parameters, TinyLlama retains competitive performance on common benchmarks while fitting within a much smaller memory footprint. But even at this scale, running inference on a conventional CPU is far too slow for real-time use, and GPUs consume too much power for battery-operated or always-on deployments.

Field-programmable gate arrays (FPGAs) offer a compelling alternative. Unlike ASICs, FPGAs are reconfigurable. The hardware can be modified precisely to the computational patterns of a specific model, enabling high-throughput, low-latency inference at a fraction of the power cost of a GPU [3].

There are two inefficiencies of GPU-based inference. First, autoregressive decode is memory-bandwidth-bound. Each decode step must reload the full model weights from DRAM to perform matrix–vector multiplications against a single token, yet profiling of TinyLlama-1.1B on an RTX 4060 Laptop GPU reveals low memory bandwidth utilisation as the remainder is lost to kernel launch overhead and memory access latency between hundreds of fine-grained CUDA kernel invocations per step. Second, the GPU runtime cannot fuse operations across transformer sub-layers. RMSNorm, matrix–vector multiplication, and activation functions are peformed as separate kernels, each incurring CPU-to-GPU scheduling latency and cache eviction between them.

An FPGA accelerator eliminates both inefficiencies simultaneously. The weight data flows directly through a statically scheduled pipeline without repeated DRAM round-trips, and the entire transformer sub-layer (normalisation, projection, and activation) can be implemented as a single fused datapath with no inter-kernel overhead.

1.2 Objectives and Scope

The general objective of this project is to implement an AI accelerator by implementing and optimizing the inference pipeline of TinyLlama-1.1B on the Xilinx ZCU102 evaluation board using Vitis HLS. Due to the complexity of the ZCU102 board, we begin our initial kernel design by targeting the KRIA KR260 FPGA board. Since the KRIA KR260 FPGA have insufficient resource to implement the TinyLlama model, we decided to start with a scaled-down version of the model's dimensions.

To understand the bottleneck of AI model inference which will be targeted for acceleration, literature review is performed and profiliers are used to obtain performance metrics from Nvidia's RTX4060 Laptop GPU. Then, a simple matrix multiplication kernel is first implemented with High-Level Synthesis (HLS). By the end of the project, a fused kernel consisting of the self-attention and feed-forward layer of the TinyLlama archietecture is implemented and synthesized on an FPGA.

1.3 Organisation of Report

The remainder of this report is organised as follows. Chapter 2 reviews the relevant literature on FPGA-based transformer acceleration, INT8 quantisation techniques, and related HLS optimisation strategies. Chapter 3 presents the overall system architecture and the design decisions made at the hardware level. Chapter 4 describes the detailed implementation of each HLS kernel and the software driver layer. Chapter 5 presents the experimental results including resource utilisation, performance benchmarks, and comparison against baseline systems. Chapter 6 draws conclusions and identifies directions for future work.

Chapter 2 Literature Review

2.1 Transformer Model Architecture

The transformer, introduced by Vaswani et al. [4], departed from the recurrent architectures that had dominated sequence modelling by replacing recurrence entirely with a mechanism called self-attention. In the original formulation, the model consisted of two stacks. An encoder that reads the full input sequence in parallel, and a decoder that generates the output sequence one token at a time while attending to the encoder's representations.

For language modelling and text generation, the decoder stack alone is sufficient. In a decoder-only transformer, the model is trained to predict the next token given all previous tokens. At inference time, the model operates autoregressively, where it processes a prompt, generates one token, appends that token to its context, and repeats until the desired output length is reached. TinyLlama follows this decoder-only paradigm.

The flow of data through a single decoder-only forward pass proceeds as follows. An input sequence of tokens is first mapped to continuous vector representations by an embedding lookup, yielding a matrix $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$ where T is the sequence length and d_{model} is the hidden dimension. Positional information is then injected, either through a fixed sinusoidal encoding as in the original transformer or through a learned rotary scheme as used in TinyLlama. This embedding matrix is then passed through N_L identical transformer layers stacked in sequence. Within each layer, the representation first passes through a normalisation step, then through a multi-head self-attention (MHA) sub-layer, with a residual connection adding the layer's input back to its output. The result is normalised again and passed through a feed-forward network (FFN) sub-layer, again with a residual connection. After all N_L layers, the final hidden states are projected to the vocabulary dimension by a linear layer called the language model (LM) head, and a softmax function converts the resulting logits to a probability distribution over the vocabulary. The token with the highest probability (or a sampled token) becomes the next output.

The self-attention mechanism is what gives the transformer its expressive power, but it also creates the dominant challenge for hardware accelerators. During the prefill phase, when the model processes the full input prompt in one pass, the computation is

dense and GPU-friendly, as all sequence positions are processed in parallel, achieving high arithmetic throughput. During the autoregressive decode phase, the model processes a single new token per step while attending over all previously generated tokens stored in a key–value (KV) cache. This single-token operation is memory-bandwidth-bound rather than compute-bound. The GPU must load the full model weight matrices from DRAM on every step despite performing only a small fraction of the arithmetic those weights could theoretically support.

2.2 TinyLlama Model Architecture

TinyLlama-1.1B [2] is a compact, open-source, decoder-only language model developed by Zhang et al. at the Singapore University of Technology and Design. Despite its relatively small parameter count of 1.1 billion, TinyLlama achieves competitive performance on commonsense reasoning and problem-solving benchmarks.

TinyLlama adopts the LLaMA-2 architecture [5] with several modifications over the original transformer. Table 2.1 summarises the key architectural hyperparameters. The model uses a hidden dimension $d_{\text{model}} = 2048$, an intermediate feed-forward dimension of 5632, 32 transformer layers, and a vocabulary of 32,000 tokens. Four architectural choices distinguish TinyLlama from the vanilla transformer described in section 2.1: rotary positional embeddings (RoPE), pre-normalisation with RMSNorm, the SwiGLU activation function in the feed-forward sub-layer, and grouped-query attention (GQA). Each of these is briefly introduced here and examined in detail in the subsections that follow.

Hyperparameter	Value
Parameters	1.1B
Hidden dimension d_{model}	2,048
FFN intermediate dim	5,632
Number of layers N_L	22
Query heads	32
KV head groups (GQA)	4
Context length	2,048 tokens
Vocabulary size	32,000

Table 2.1: TinyLlama-1.1B architectural hyperparameters [2].

Rotary Positional Embedding (RoPE). Unlike the absolute sinusoidal positional encodings added to the input embeddings in the original transformer, RoPE [6] encodes

positional information by rotating the query and key vectors in the attention computation.

Pre-normalisation with RMSNorm. The original transformer applied layer normalisation after each sub-layer (post-norm). TinyLlama, following LLaMA-2, applies normalisation before each sub-layer (pre-norm). The normalisation function used is RMSNorm [7], a simplified variant of LayerNorm that omits the mean-centring step, reducing the number of passes over the input vector and lowering hardware implementation complexity.

SwiGLU Feed-Forward Network. The feed-forward sub-layer in TinyLlama replaces the standard two-layer ReLU MLP with the SwiGLU variant [8], which introduces a gating mechanism. The input is projected into two parallel streams, one of which is passed through the Swish activation function and then element-wise multiplied with the other.

Grouped-Query Attention (GQA). Standard multi-head attention computes separate key and value projections for each of the 32 query heads, resulting in a KV cache that grows as $O(L \cdot H \cdot T \cdot d_k)$ with sequence length T. TinyLlama reduces this overhead by sharing key and value representations across groups of query heads, using only 4 KV head groups for 32 query heads [9]. This reduces the KV cache size by a factor of 8, which is significant for memory- constrained FPGA deployment.

2.2.1 Token Embeddings

The first operation in every forward pass converts discrete token indices into continuous vector representations. The model maintains an embedding matrix $\mathbf{E} \in \mathbb{R}^{V \times d_{\text{model}}}$, where V is the vocabulary size and d_{model} is the hidden dimension. For TinyLlama, $V = 32{,}000$ and $d_{\text{model}} = 2{,}048$. Given an input sequence of T token indices $[t_1, t_2, \ldots, t_T]$, the embedding lookup produces:

$$\mathbf{X} = \begin{bmatrix} \mathbf{e}_{t_1} \\ \mathbf{e}_{t_2} \\ \vdots \\ \mathbf{e}_{t_T} \end{bmatrix} \in \mathbb{R}^{T \times d_{\text{model}}} \tag{2.1}$$

where $\mathbf{e}_{t_i}$ denotes the t_i-th row of $\mathbf{E}$. Each row is a d_{model}-dimensional dense vector learned during pretraining that positions semantically similar tokens close together in the embedding space.

Space complexity. The embedding matrix requires $V \times d_{\text{model}}$ parameters. In FP32 this amounts to $32{,}000 \times 2{,}048 \times 4 = 262\,\text{MB}$. This is one of the larger weight tensors in the model and is a candidate for quantisation since its access pattern during inference,

consisting of one row lookup per generated token, is entirely memory-bound.

Time complexity. The lookup itself is $\mathcal{O}(T)$ since it is a table index operation with no arithmetic; however, the subsequent layers treat $\mathbf{X}$ as a dense matrix and all downstream costs scale with $T \times d_{\text{model}}$.

2.2.2 Rotary Positional Embedding

Because the self-attention mechanism is permutation-invariant, treating the input as a set of vectors with no inherent order, the model must be given some means of distinguishing position m from position n. TinyLlama uses Rotary Positional Embedding (RoPE) [6], which encodes position by rotating the query and key vectors inside each attention head.

For a query or key vector $\mathbf{q} \in \mathbb{R}^{d_k}$ at sequence position m, RoPE applies a block-diagonal rotation matrix $\mathbf{R}_m \in \mathbb{R}^{d_k \times d_k}$:

$$\mathbf{R}_m = \begin{bmatrix} \cos m\theta_1 & -\sin m\theta_1 & & & \\ \sin m\theta_1 & \cos m\theta_1 & & & \\ & & \cos m\theta_2 & -\sin m\theta_2 & \\ & & \sin m\theta_2 & \cos m\theta_2 & \\ & & & & \ddots \end{bmatrix} \tag{2.2}$$

where $\theta_i = 10000^{-2(i-1)/d_k}$ for $i = 1, \ldots, d_k/2$ are base frequencies that decrease geometrically, allowing different pairs of dimensions to encode position at different scales. The rotated query is then $\tilde{\mathbf{q}}_m = \mathbf{R}_m \mathbf{q}_m$. A key property is that the inner product between a rotated query at position m and a rotated key at position n depends only on the relative offset $m - n$:

$$\tilde{\mathbf{q}}_m^\top \tilde{\mathbf{k}}_n = \mathbf{q}_m^\top \mathbf{R}_m^\top \mathbf{R}_n \mathbf{k}_n = \mathbf{q}_m^\top \mathbf{R}_{n-m} \mathbf{k}_n \tag{2.3}$$

In practice, the block-diagonal structure of $\mathbf{R}_m$ means the rotation is implemented efficiently as element-wise multiplication with precomputed cos and sin tables, avoiding any explicit matrix-matrix product.

Space complexity. The only parameters are the precomputed cos and sin tables of shape $T_{\max} \times d_k/2$, which for TinyLlama ($T_{\max} = 2{,}048$, $d_k = 64$) requires $2 \times 2{,}048 \times 32 \times 4 = 0.5\,\text{MB}$ in FP32, which is negligible relative to the weight matrices. No learned parameters are introduced by RoPE.

Time complexity. Applying RoPE to all query and key vectors across all heads costs $\mathcal{O}(T \times H \times d_k)$ per layer, where H is the number of heads. This is linear in sequence length and dominated in practice by the attention computation that follows.

2.2.3 RMSNorm

Normalisation is applied before each sub-layer in TinyLlama (pre-norm). TinyLlama uses Root Mean Square Layer Normalisation (RMSNorm) [7], which normalises by the RMS of the input vector.

For an input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$, RMSNorm computes:

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} x_i^2 + \varepsilon} \tag{2.4}$$

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \odot \mathbf{g} \tag{2.5}$$

where $\mathbf{g} \in \mathbb{R}^{d_{\text{model}}}$ is a learned element-wise scale vector and ε is a small constant for numerical stability. Applied to the full sequence matrix $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$, RMSNorm is applied independently to each row:

$$\text{RMSNorm}(\mathbf{X}) = \begin{bmatrix} \frac{\mathbf{x}_1}{\text{RMS}(\mathbf{x}_1)} \\ \frac{\mathbf{x}_2}{\text{RMS}(\mathbf{x}_2)} \\ \vdots \\ \frac{\mathbf{x}_T}{\text{RMS}(\mathbf{x}_T)} \end{bmatrix} \odot \mathbf{1}_T \mathbf{g}^\top \tag{2.6}$$

Space complexity. Each RMSNorm layer stores one scale vector $\mathbf{g}$ of length $d_{\text{model}} = 2{,}048$. With $2L = 44$ normalisation layers across 22 transformer layers (one before attention, one before FFN), the total RMSNorm parameter count is $44 \times 2{,}048 \times 4 = 0.36\,\text{MB}$ in FP32, which is negligible. The intermediate squared accumulation during the RMS computation must be performed at sufficient precision to avoid overflow, which in FP32 poses no issue but becomes a consideration under INT8 quantisation.

Time complexity. Computing RMSNorm for a single token vector requires one pass over d_{model} elements for the squared sum, a square root, and one element-wise multiply, giving $\mathcal{O}(d_{\text{model}})$ operations per token and $\mathcal{O}(T \times d_{\text{model}})$ for a full sequence. The data dependency, specifically that the normalisation factor cannot be computed until all d_{model} elements have been read, means that RMSNorm introduces a reduction bottleneck that limits pipeline depth in hardware.

2.2.4 Grouped-Query Self-Attention

Self-attention is the mechanism that allows each token representation to gather information from all other tokens in the sequence. In the standard multi-head attention

(MHA) formulation [4], the input $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$ is linearly projected into H sets of query, key, and value vectors:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V \tag{2.7}$$

where $\mathbf{W}_Q \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$, $\mathbf{W}_K \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$, and $\mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ in the standard MHA case. Each of the $H = 32$ heads operates on a $d_k = d_{\text{model}}/H = 64$-dimensional slice. The attention output for head h is:

$$\text{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h) = \text{softmax}\left(\frac{\mathbf{Q}_h \mathbf{K}_h^\top}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}_h \tag{2.8}$$

where $\mathbf{M} \in \mathbb{R}^{T \times T}$ is the causal mask that sets future positions to $-\infty$ before the softmax, enforcing the autoregressive constraint that position i may only attend to positions $\leq i$. The outputs of all H heads are concatenated and projected back to d_{model} via an output weight matrix $\mathbf{W}_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$:

$$\mathbf{Y}_{\text{attn}} = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)\,\mathbf{W}_O \tag{2.9}$$

TinyLlama replaces standard MHA with Grouped-Query Attention (GQA) [9]. Rather than maintaining H independent key and value projections, GQA uses $G = 4$ KV head groups shared across the $H = 32$ query heads, meaning every 8 query heads share the same key and value representations. The KV projection matrices are therefore reduced:

$$\mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times (G \cdot d_k)} = \mathbb{R}^{2048 \times 256} \tag{2.10}$$

During autoregressive generation, the key and value tensors for all past positions must be stored in a KV cache so they are not recomputed on every decode step. In standard MHA the cache grows as $2 \times L \times T \times H \times d_k$ per sequence. GQA reduces this by the ratio $H/G = 8$, giving a KV cache of size $2 \times N_L \times T \times G \times d_k$ elements. For TinyLlama at the maximum context length ($N_L = 22$, $T = 2{,}048$, $G = 4$, $d_k = 64$) in FP32, this amounts to:

$$2 \times 22 \times 2{,}048 \times 4 \times 64 \times 4\,\text{bytes} = 90.2\,\text{MB} \tag{2.11}$$

compared to 721.8 MB that full MHA would require, which represents an $8\times$ reduction that is critical for on-chip buffering in FPGA deployment.

Space complexity. The four attention weight matrices $(\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O)$ per layer total $(d_{\text{model}}^2 + 2 \cdot d_{\text{model}} \cdot G d_k + d_{\text{model}}^2)$ parameters. For TinyLlama per layer: $(2048^2 +$

$2 \times 2048 \times 256 + 2048^2) \times 4 = 42.5\,\text{MB}$ in FP32 per layer, or 935 MB across all 22 layers.

Time complexity. The Q, K, V projections each require $\mathcal{O}(T \times d_{\text{model}}^2)$ operations during prefill. The scaled dot-product $\mathbf{Q}_h\mathbf{K}_h^\top$ costs $\mathcal{O}(T^2 \times d_k)$ per head and $\mathcal{O}(H \times T^2 \times d_k) = \mathcal{O}(T^2 \times d_{\text{model}})$ in total, which is the quadratic-in-T cost that makes long-context inference expensive. During decode (single new token $t = 1$), the projection costs reduce to $\mathcal{O}(d_{\text{model}}^2)$, and the attention reduces to $\mathcal{O}(T_{\text{cache}} \times d_{\text{model}})$ where T_{cache} is the current KV cache length.

2.2.5 Softmax

Within the scaled dot-product attention computation, the raw attention scores produced by the query–key dot products must be converted into a probability distribution over the sequence positions before they can be used to weight the value vectors. This conversion is performed by the softmax function. For a vector of scores $\mathbf{a} \in \mathbb{R}^T$, softmax is defined element-wise as:

$$\text{softmax}(\mathbf{a})_t = \frac{\exp(a_t)}{\sum_{t'=1}^{T} \exp(a_{t'})} \tag{2.12}$$

The output is strictly positive and sums to one, making it a valid probability distribution. In practice, Equation 2.12 is not computed directly because exponentials of large positive values overflow in finite-precision arithmetic. The numerically stable variant subtracts the maximum score from all elements before exponentiation:

$$\text{softmax}(\mathbf{a})_t = \frac{\exp(a_t - \max_j a_j)}{\sum_{t'=1}^{T} \exp(a_{t'} - \max_j a_j)} \tag{2.13}$$

Since the subtracted constant cancels between numerator and denominator, Equation 2.13 is mathematically identical to Equation 2.12 but constrains all exponent arguments to $(-\infty, 0]$, preventing overflow while preserving underflow behaviour that is benign in practice.

Causal masking. In autoregressive decode, the model must not attend to future positions. This is enforced by adding a mask $\mathbf{M}$ to the scores before softmax, where $M_{ij} = -\infty$ for $j > i$ and 0 otherwise. Positions set to $-\infty$ produce $\exp(-\infty) = 0$ after exponentiation, effectively zeroing out future positions in the probability distribution without requiring any change to the softmax function itself.

Space complexity. Softmax introduces no learned parameters. The only storage required is the score buffer $\mathbf{a} \in \mathbb{R}^{T_{\text{cache}}}$ per head, which during decode has length equal to the current KV cache size. This buffer must be held in on-chip memory throughout

the computation since two passes are required: one to find the maximum and one to compute the normalised exponentials.

Time complexity. Computing softmax for a single attention head during decode requires $\mathcal{O}(T_{\text{cache}})$ operations: one pass to find the maximum, one pass to compute and sum the exponentials, and one pass to normalise. All three passes are sequential due to the reduction dependencies between them, meaning the total latency is $3T_{\text{cache}}$ operations rather than T_{cache} operations with three-way parallelism. This sequential dependency between passes is the primary reason softmax introduces a pipeline stall in hardware implementations.

Hardware implementation challenge. The exponential function $\exp(\cdot)$ is a transcendental operation that requires either a multi-cycle floating-point unit or a lookup table approximation in hardware. A floating-point exponential typically requires 10 to 20 clock cycles to produce a result, which directly multiplies the per-token softmax latency. For this reason, hardware-efficient inference systems commonly replace the true exponential with a piecewise-linear or lookup-table approximation. The I-BERT quantisation framework [10] proposes such an approximation that bounds the relative error while operating entirely in integer arithmetic.

2.2.6 SwiGLU Feed-Forward Network

Each transformer layer contains a position-wise feed-forward network (FFN) applied independently to each token's representation after the attention sub-layer. TinyLlama uses the SwiGLU variant [8], which replaces the single hidden projection with two parallel streams. One passes through the Swish activation function and the other acts as a multiplicative gate, with both combined before the output projection. Concretely, for a token vector $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$:

$$\text{Swish}(\mathbf{z}) = \mathbf{z} \odot \sigma(\mathbf{z}) \tag{2.14}$$

$$\text{FFN}_{\text{SwiGLU}}(\mathbf{x}) = \Big(\text{Swish}(\mathbf{x}\mathbf{W}_1) \odot \mathbf{x}\mathbf{W}_3\Big)\mathbf{W}_2 \tag{2.15}$$

where $\sigma(\cdot)$ is the sigmoid function, $\mathbf{W}_1, \mathbf{W}_3 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ are the gate and up projection matrices respectively, and $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ is the down projection. For TinyLlama, $d_{\text{ff}} = 5{,}632$. Applied to the full sequence in matrix form:

$$\mathbf{Y}_{\text{ffn}} = \Big(\underbrace{\big(\mathbf{X}\mathbf{W}_1\big) \odot \sigma\big(\mathbf{X}\mathbf{W}_1\big)}_{\text{Swish gate}:T \times d_{\text{ff}}} \odot \underbrace{\mathbf{X}\mathbf{W}_3}_{\text{up proj}:T \times d_{\text{ff}}}\Big) \underbrace{\mathbf{W}_2}_{d_{\text{ff}} \times d_{\text{model}}} \tag{2.16}$$

The element-wise gating in Equation 2.16 means that the effective hidden repre-

sentation at each position is modulated by a learned sigmoid gate. Note that SwiGLU introduces three weight matrices per FFN layer rather than two, which increases the parameter count but allows d_{ff} to be set smaller than in a standard two-layer MLP to maintain comparable total parameter count.

Space complexity. The three FFN weight matrices per layer total $2 \times d_{\text{model}} \times d_{\text{ff}} + d_{\text{ff}} \times d_{\text{model}} = 3 \times d_{\text{model}} \times d_{\text{ff}}$ parameters. For TinyLlama: $3 \times 2{,}048 \times 5{,}632 \times 4 =$ 138.4 MB per layer in FP32, or 3,045 MB across all 22 layers. The FFN layers collectively constitute the largest weight component of the model, making them the highest-priority target for INT8 quantisation.

Time complexity. Each of the three matrix multiplications costs $\mathcal{O}(T \times d_{\text{model}} \times d_{\text{ff}})$ operations. During prefill the total FFN cost is $\mathcal{O}(T \times d_{\text{model}} \times d_{\text{ff}})$ per layer, and during decode ($T = 1$) it reduces to $\mathcal{O}(d_{\text{model}} \times d_{\text{ff}})$, reducing to a matrix-vector multiply rather than a matrix-matrix multiply. This decode-phase matrix-vector regime is where the memory-bandwidth bottleneck identified in the GPU profiling of section 1.1 arises. The full $d_{\text{model}} \times d_{\text{ff}}$ weight matrix must be loaded from DRAM on every decode step regardless of how few arithmetic operations it supports since the batch size is one.

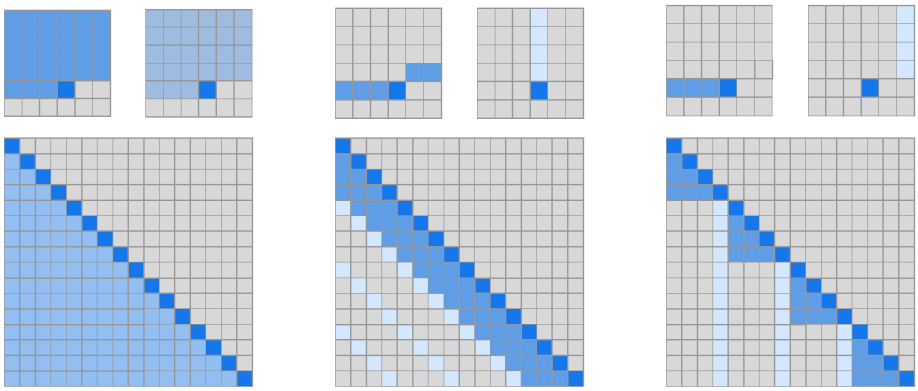
2.3 Efficient Attention Mechanisms

The self-attention mechanism described in section 2.1 requires computing pairwise interactions between all T tokens in a sequence, incurring $\mathcal{O}(T^2)$ time and memory complexity per layer. For the moderate context lengths used in TinyLlama this cost is manageable, but it represents the dominant algorithmic bottleneck and has motivated a substantial body of work on efficient attention variants. These approaches fall into two broad families: structural methods that preserve exact softmax attention over a restricted set of token pairs, and algebraic methods that replace the softmax kernel entirely with a decomposable approximation. The four models surveyed here are representative of both families and inform the design space for hardware-efficient inference.

2.3.1 Sparse Transformer

Child et al. [11] reduce the number of attention connections per layer from $\mathcal{O}(T^2)$ to $\mathcal{O}(T^{3/2})$ by restricting each token to attend only to a small, structured subset of other tokens rather than the full sequence. Two complementary sparsity patterns are interleaved across successive attention layers: a local pattern in which each token attends to its w nearest neighbours, and a strided pattern in which each token attends to one token every w positions, as illustrated in fig. 2.1. By choosing $w = \mathcal{O}(\sqrt{T})$, every token can reach every other within two hops, and Yun et al. [12] subsequently

prove formally that such sparse patterns preserve the universal approximation property of dense attention. Because the non-zero entries remain contiguous blocks, the computation reduces to dense sub-matrix multiplications that map well onto pipelined hardware datapaths. Crucially for decoder models, the sparsity patterns can be applied directly under a causal mask without modification, making the Sparse Transformer natively compatible with autoregressive generation.



(a) Transformer (b) Sparse Transformer (strided) (c) Sparse Transformer (fixed)

Figure 2.1: Connectivity matrices for full attention (a), Sparse Transformer with strided pattern (b), and Sparse Transformer with fixed pattern (c). In (b) and (c), full connectivity between all elements is preserved when the two attention heads are computed sequentially, while sparsity in each individual head leads to significantly reduced computation [11].

2.3.2 BlockBERT

Qiu et al. [13] take a complementary structural approach by partitioning the $T \times T$ attention matrix into an $n \times n$ grid of equal-sized blocks and restricting each attention head to computing attention within exactly one block, determined by a cyclic permutation of the block indices. This reduces the $\mathcal{O}(T^2)$ memory cost by a factor of n, since each block has size $\frac{T}{n} \times \frac{T}{n}$ and only n such blocks are evaluated in total. Different heads are assigned different permutations, so that the model collectively covers both local (identity permutation) and cross-block long-range interactions across the head ensemble. Because the retained entries form dense sub-matrices rather than scattered sparse entries, the method requires no custom sparse kernels and maps directly onto efficient dense matrix operations available in standard deep learning frameworks. Empirically, BlockBERT with $n = 2$ achieves an 18.7% reduction in activation memory and a 12.0% reduction in training time compared to RoBERTa, while matching or exceeding its accuracy on several question-answering benchmarks [13].

2.3.3 Performer

Choromanski et al. [14] reformulate attention algebraically by expressing the softmax kernel as a product of lower-dimensional random feature maps, $K(Q_i, K_j) \approx \phi(Q_i)^\top \phi(K_j)$, where ϕ is constructed from a random matrix $\mathbf{W} \in \mathbb{R}^{M \times d}$ whose rows are drawn orthogonally to reduce approximation variance. This decomposition, known as FAVOR+ (Fast Attention Via Orthogonal Random Features), permits the matrix product order to be rearranged as shown in Equation 2.17: rather than first forming the $T \times T$ attention matrix and multiplying by $\mathbf{V}$, one computes $\mathbf{K}'^\top \mathbf{V}$ first, an $M \times d$ matrix, and then multiplies by $\mathbf{Q}'$, reducing complexity to $\mathscr{O}(TMd)$ which is linear in T when $M \ll T$.

$$\mathbf{Y} = \hat{\mathbf{D}}^{-1}\Big(\mathbf{Q}'\big(\mathbf{K}'^\top \mathbf{V}\big)\Big), \quad \hat{\mathbf{D}} = \operatorname{diag}\Big(\mathbf{Q}'\big(\mathbf{K}'^\top \mathbf{1}_T\big)\Big) \tag{2.17}$$

where $\mathbf{Q}' = \phi(\mathbf{Q})$ and $\mathbf{K}' = \phi(\mathbf{K})$ are the random-feature projections of the query and key matrices respectively, and $\hat{\mathbf{D}}$ is a diagonal normalisation matrix that plays the role of the softmax row-sum. The approximation quality is controlled by M and can be made arbitrarily accurate at the cost of a larger constant. A significant practical limitation, however, is that efficient causal masking cannot be applied within the FAVOR+ framework during training: doing so would require materialising the $d \times d$ matrix $\mathbf{K}'^\top \mathbf{V}$ at every timestep, recovering quadratic cost and making autoregressive training with teacher forcing prohibitively slow [14].

2.3.4 Linear Transformer

Katharopoulos et al. [15] achieve linear complexity through the same kernel decomposition principle as the Performer, but replace the softmax similarity with a deterministic feature map $\operatorname{sim}(q,k) := \phi(q)^\top \phi(k)$, where ϕ is a simple element-wise function such as $\operatorname{ELU}(x) + 1$. In the bidirectional case, the cumulative statistics $\mathbf{S}_N = \sum_j \phi(K_j)\mathbf{V}_j^\top$ and $\mathbf{z}_N = \sum_j \phi(K_j)$ need only be computed once and reused across all query positions, yielding $\mathscr{O}(Nd^2)$ complexity per layer. More significantly for autoregressive decoding, the causal variant reduces naturally to a constant-memory recurrent formulation in which the state is updated incrementally as $\mathbf{S}_i = \mathbf{S}_{i-1} + \phi(K_i)\mathbf{V}_i^\top$ at each new token, without any sequential dependency on T. This recurrent structure is particularly well-suited to FPGA deployment, where a fixed $d \times d$ accumulation datapath can be instantiated once and reused across all decode steps at constant resource cost. The principal accuracy trade-off relative to standard softmax attention is that the chosen ϕ does not guarantee strictly positive attention weights everywhere, which can introduce training instability in some configurations [15].

2.4 FPGA Acceleration of Neural Networks

FPGA-based neural network acceleration has been an active research area for many years. It works by exploiting data-level parallelism and on-chip memory buffering. FPGAs allow the designer to co-optimise the compute pipeline and the memory hierarchy simultaneously, a degree of flexibility that is unavailable in fixed GPU silicon.

For transformer models specifically, the dominant challenge is memory bandwidth rather than raw arithmetic throughput. This memory-bound characteristic means that the primary optimisation target is minimising redundant DRAM accesses through on-chip weight buffering and tiling, rather than maximising multiply-accumulate throughput.

2.5 Quantisation for Efficient Inference

Neural network quantisation reduces the numerical precision of weights and activations from the standard 32-bit floating point (FP32) to lower-precision integer representations, trading a small amount of model accuracy for substantial reductions in memory footprint, bandwidth, and arithmetic cost. For FPGAs in particular, fixed-point integer arithmetic is significantly more resource-efficient than floating-point [10].

Post-training quantisation (PTQ) at INT8 precision has become the dominant approach for production LLM deployment, with works such as LLM.int8() [16] and SmoothQuant [17] demonstrating that 8-bit quantisation of both weights and activations incurs less than 1% accuracy degradation on standard benchmarks when applied with appropriate per-channel scaling factors.

2.6 HLS-Based Hardware Design

High-level synthesis tools bridge the gap between algorithmic descriptions in C/C++ and synthesisable RTL, reducing the design iteration cycle compared to manual HDL development. Vitis HLS, the primary tool used in this project, translates annotated C++ functions into pipelined hardware modules whose behaviour is controlled through pragma directives [3]. The important directives used to improve performance are `PIPELINE`, `UNROLL`, and `ARRAY_PARTITION`, which control loop pipelining, spatial parallelism, and on-chip memory port availability respectively.

2.6.1 Pipelining and the Initiation Interval

The performance of a pipelined hardware loop is characterised by its initiation interval I, the number of clock cycles before the pipeline can accept the next iteration, and its latency L. For a loop processing N iterations, the total cycle count is [18]:

$$C = L + I \cdot (N - 1) \tag{2.18}$$

A perfect pipeline achieves $I = 1$, so that the runtime scales as $I \cdot N$ and halving I halves the runtime. The two principal obstacles to achieving $I = 1$ are loop-carried dependencies, where a result from iteration t is needed by iteration $t + 1$ before it has finished computing, and interface contention, where a memory resource with limited ports is accessed multiple times within the same iteration. These are resolved through accumulation interleaving and array partitioning respectively [18], both of which are applied extensively in this project.

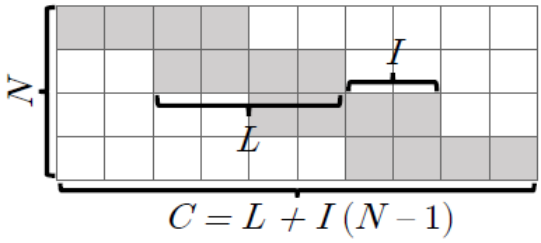


Figure 2.2: Pipeline characteristics showing latency L, initiation interval I, and total cycle count $C = L + I(N - 1)$ for N inputs [18].

2.6.2 Scaling Transformations: Unrolling and Dataflow

Once a loop achieves $I = 1$, performance is increased further by reducing the number of iterations N through unrolling [18]. Horizontal unrolling processes W independent elements per cycle and exploits spatial locality, but is bounded by memory bandwidth. Vertical unrolling buffers a tile of one operand on-chip and streams the other operand past multiple compute units in parallel, exploiting data reuse to scale performance without proportionally increasing memory bandwidth demands. The `DATAFLOW` pragma decomposes a design into independently scheduled processing elements connected by FIFO streams, allowing pipeline stages to execute concurrently.

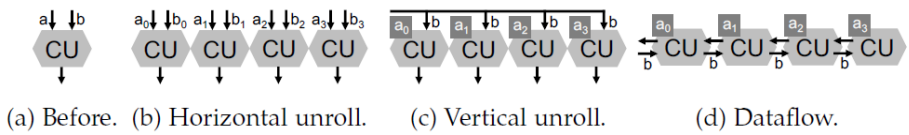


Figure 2.3: Horizontal unrolling, vertical unrolling, and dataflow as means to increase parallelism. Rectangles represent buffer space such as registers or on-chip RAM. (a) Before any transformation. (b) Horizontal unrolling: four independent inputs processed in parallel. (c) Vertical unrolling: one input combined with multiple buffered values. (d) Dataflow: input or partial results streamed through a pipeline rather than broadcast [18].

2.6.3 Memory Transformations

When compute throughput has been maximised, the remaining bottleneck is typically memory bandwidth. Memory access extraction separates load and store operations into dedicated pipeline stages that issue long DRAM bursts independently of the compute logic, maximising bandwidth utilisation [18]. Type demotion from FP32 to INT8 reduces both the bandwidth consumed per weight loaded and the on-chip resources required to process it, directly addressing the memory-bound nature of transformer inference on resource-constrained devices.

Chapter 3 System Architecture and Design

3.1 Top-Level System Overview

The inference system is designed around the heterogeneous architecture of the Xilinx ZCU102, which integrates a quad-core ARM Cortex-A53 processing system (PS) and a Zynq UltraScale+ programmable logic (PL) fabric on a single SoC. The two subsystems communicate through a high-bandwidth AXI interconnect and share a common DDR4 memory pool. This heterogeneous arrangement is well-suited to transformer inference because the workload naturally decomposes into two qualitatively different classes of computation: a small amount of sequential control logic that manages the autoregressive loop, and a large volume of repetitive arithmetic that is amenable to pipelined hardware acceleration.

The PS is responsible for all control-flow operations. These include loading the quantised model weights from storage into DDR at startup, running the tokeniser to convert an input prompt into a sequence of integer token indices, orchestrating the layer-by-layer forward pass by issuing kernel invocations over the XRT API, performing post-attention sampling (temperature scaling and argmax over the vocabulary logit vector), and appending each generated token back to the context for the next decode step. The PL accelerator kernel performs the computationally intensive operations within each transformer layer: the RMSNorm normalisation, the quantise-and-dequantise operations around each matrix multiplication, the grouped-query attention mechanism including the scaled dot-product attention and weighted value accumulation, and the residual stream addition.

The overall execution model for a single decode step proceeds as follows. The host application supplies the current token index and the current sequence position to the PL kernel. The kernel loads the token embedding from DDR, then iterates over all transformer layers, performing the attention sub-block for each layer and writing the updated key and value vectors into the KV cache in DDR. After the final layer, the resulting activation vector is written back to a DDR output buffer. The host then reads this buffer, applies the final RMSNorm and the language model head projection entirely in software on the ARM cores, and samples the next token. The process repeats until the desired output length is reached.

This decomposition was chosen at the project outset with the intention of incre-

mentally offloading more of the forward pass to the PL fabric as the hardware design matured. The FFN block, the output attention projection, and the LM head are designed and specified in this chapter; the scope of the synthesised hardware kernel and the rationale for the current partitioning boundary are discussed in section 3.2.

3.2 Hardware/Software Partitioning

The partitioning of operations between the PS and PL is governed by two criteria applied jointly: arithmetic intensity and implementation complexity. Arithmetic intensity is defined as the ratio of floating-point operations to bytes of memory transferred, and it determines whether a computation is bottlenecked by compute throughput or by memory bandwidth. Operations with high arithmetic intensity benefit from pipelined hardware acceleration because the FPGA fabric can sustain many multiply-accumulate operations per clock cycle once the operands have been loaded. Operations with low arithmetic intensity are memory-bandwidth-bound and gain little from hardware parallelism relative to the overhead of transferring data to and from the PL kernel.

The matrix–vector multiplications that dominate each transformer layer have intermediate arithmetic intensity: each weight element is loaded once and used once, giving an intensity of approximately two operations per byte at INT8 precision. This is above the threshold where FPGA acceleration begins to outperform a general-purpose ARM core, particularly given that the ZCU102's PL fabric supports deeply pipelined DSP48E2 blocks that can sustain one multiply-accumulate per clock cycle at 200 MHz. The attention score computation and the weighted value accumulation have similar structure and are likewise accelerated in the PL.

Tokenisation and sampling are retained on the ARM PS. Tokenisation is a byte-level string operation with negligible arithmetic content, and moving it to the PL would introduce significant control-flow complexity for no performance benefit. Sampling involves an argmax over the vocabulary logit vector of length 32,000 followed by optional temperature scaling: the memory access pattern is irregular and the arithmetic is trivial, making it a poor candidate for hardware acceleration. The residual stream additions, by contrast, are kept inside the PL kernel and executed immediately after each sub-block's output, avoiding a round-trip of the activation vector through the AXI interface back to the ARM core on every layer of every decode step.

3.3 Memory Architecture

The memory architecture is organised around a two-tier hierarchy that reflects the fundamental trade-off between capacity and access speed available on the ZCU102. The off-chip DDR4 memory offers large capacity but is shared between the PS and

PL and incurs significant latency for each access. The on-chip BRAM and flip-flop registers inside the PL fabric are fast and directly accessible to the compute pipeline, but are scarce relative to the volume of data involved in transformer inference. The guiding principle behind the memory organisation is therefore straightforward: data that is too large to fit on-chip or that must persist across multiple decode steps is placed in DDR, while data that is small, short-lived, and accessed repeatedly during a single layer computation is held on-chip.

Off-chip DDR memory. Three categories of data reside in DDR: the model weights, the KV cache, and the output buffer. The model weights are by far the largest component. At full TinyLlama scale, after INT8 quantisation (described in section 3.4), the attention weight matrices across all 22 layers total approximately 221 MB, the FFN weight matrices total approximately 809 MB, and the token embedding table requires a further 70 MB. Together these far exceed the on-chip memory available on any FPGA device, so the weights must remain in DDR and be streamed into the compute pipeline one layer at a time. The access pattern is sequential and read-only during inference, which is well-matched to the burst transfer capability of the DDR memory controller. A key physical constraint on all DDR buffers shared between the PS and PL is that they must occupy a contiguous region of physical memory, which is enforced through the Linux Contiguous Memory Allocator (CMA) managed transparently by XRT on the ZCU102.

The KV cache must also reside in DDR because its size grows linearly with sequence length and it must persist across every decode step. Unlike the weights, the KV cache is both written when a new key or value vector is computed for the current position, and read when computing attention scores against all previous positions. At the maximum context length of 2,048 tokens, the combined key and value cache occupies approximately 24.5 MB for the full TinyLlama model after INT8 quantisation of the cached vectors, which fits comfortably within the ZCU102's 4 GB DDR alongside the weights. A small output buffer, holding a single activation vector of d_{model} values, is also allocated in DDR for the PL kernel to write its result back to the PS at the end of each decode step.

On-chip memory. All intermediate activation state generated during a single layer's computation is held in on-chip BRAM and registers. This includes the working activation vectors for the current token, the query, key, and value vectors, the attention score accumulator, and the intermediate representations produced by the FFN sub-block. The critical property of all this data is that none of it needs to persist between layers: once the residual addition has been applied at the end of a layer, the intermediate values are discarded and the same on-chip buffers are reused in the next iteration of the layer

loop. The on-chip memory footprint is therefore determined solely by the dimensions of a single layer, not by the depth of the model. At full TinyLlama scale the on-chip activation state amounts to tens of kilobytes, well within the ZCU102's available BRAM capacity. The two-tier memory organisation is summarised in table 3.1.

Data	Location	Approx. size (full model)	Access pattern
Attention weights (W_Q, W_K, W_V, W_O)	DDR	~221 MB	Streamed per layer
FFN weights (W_1, W_2, W_3)	DDR	~809 MB	Streamed per layer
Token embedding table	DDR	~70 MB	One lookup per step
KV cache (keys + values)	DDR	~24.5 MB	Read/write per step
Output buffer	DDR	<1 MB	One write per step
Intermediate activation buffers	On-chip	~96 KB	Reused per layer

Table 3.1: Memory hierarchy of the TinyLlama inference accelerator on the ZCU102. Weight and KV cache sizes are after INT8 quantisation. On-chip buffers are reused across all 22 layers.

3.4 Quantisation Scheme

All weight tensors are quantised offline to INT8 using symmetric per-group post-training quantisation (PTQ). The decision to use INT8 rather than FP32 was motivated by two considerations. First, INT8 arithmetic is supported natively by the ZCU102's DSP48E2 blocks as an 8×8 multiply accumulating into a 27-bit result, which avoids the resource overhead of implementing floating-point multipliers in LUT fabric. Second, the decode phase of transformer inference is memory-bandwidth-bound at batch size one, as established in the GPU profiling of section 1.1: reducing the weight precision from FP32 to INT8 gives a 4× reduction in the bytes transferred per decode step, which directly translates to a 4× reduction in the DRAM bandwidth required to sustain a given throughput.

For a weight vector $\mathbf{w} \in \mathbb{R}^n$ the symmetric per-group quantisation scheme divides $\mathbf{w}$ into consecutive non-overlapping groups of GS elements and computes a separate scale factor for each group:

$$s_g = \frac{\max_{i \in \mathscr{G}_g} |w_i|}{127} \tag{3.1}$$

$$\hat{w}_i = \text{clip}\left(\text{round}\left(\frac{w_i}{s_g}\right), -128, 127\right) \tag{3.2}$$

where $\mathscr{G}_g = \{g \cdot \text{GS}, \ldots, (g+1) \cdot \text{GS} - 1\}$ is the index set of group g and s_g maps

the maximum absolute value in the group to the INT8 boundary of 127. The quantised weights $\hat{w}_i \in [-128, 127]$ are stored as `int8_t` values and the scale factors s_g are stored alongside as `float32`, one per group, in the `QuantizedTensor` data structure.

The group size GS controls the granularity of the approximation. A group size of 1 reduces to per-element quantisation, which minimises quantisation error at the cost of storing one scale factor per weight. A group size equal to the full vector length is equivalent to per-tensor quantisation, which uses minimal storage for scale factors but suffers higher error when the weight distribution has outliers. The design uses a group size of $\mathrm{GS} = 64$ for the full TinyLlama model, following the convention established in the llama2.c reference implementation **PLACEHOLDER**, which was found to provide an acceptable balance between quantisation error and the memory overhead of scale factor storage. For the scaled-down HLS simulation configuration ($d_{\text{model}} = 8$), a group size of $\mathrm{GS} = 4$ is used so that the constraint $\mathrm{GS} \leq d_{\text{model}}$ is satisfied and at least one complete quantisation group is present per weight vector.

At runtime, activations are quantised dynamically before each matrix-vector multiplication. The quantisation is performed using the same symmetric per-group scheme, with scale factors computed from the current activation values rather than precomputed offline. After the integer multiply-accumulate inside the matrix multiplication, the result is dequantised back to float32 by multiplying each partial sum by the product of the corresponding weight and activation scale factors. This approach keeps all inner product arithmetic in the INT8 domain while accumulating into 32-bit integers to prevent overflow, and defers the floating-point conversion to the final output of each matrix-vector row. The interaction between the quantisation scheme and the matrix-vector kernel implementation is described in detail in chapter 4.

Chapter 4 Implementation

All kernels in this chapter are implemented in Vitis HLS 2021.2, targeting the `xczu9eg-ffvb1156-2-e` device on the ZCU102 evaluation board at a design clock of 150 MHz. Each primitive is first presented as a standalone synthesisable function, allowing its resource utilisation and initiation interval to be measured in isolation before it is composed into the full transformer forward pass.

Development proceeded across two configurations. A scaled-down prototype configuration was used to establish the kernel structure, verify the HLS build flow, and confirm functional correctness in C-simulation without the lengthy synthesis times and large on-chip resource requirements of the full model. Once each component was validated at small scale, the configuration constants were updated to target the full TinyLlama-1.1B dimensions. Table 4.1 summarises the two sets of parameters.

Parameter	Prototype	TinyLlama-1.1B
Hidden dimension d_{model}	8	2,048
FFN hidden dimension d_{ff}	32	5,632
Number of layers N_L	2	22
Query heads H	2	32
KV heads G	2	4
Vocabulary size V	32	32,000
Maximum sequence length	16	2,048
Quantisation group size GS	4	64

Table 4.1: Prototype and full TinyLlama-1.1B configuration parameters used across kernel development.

The prototype dimensions were chosen deliberately to keep array sizes small enough that HLS C-simulation completes in seconds and synthesis reports are straightforward to inspect.

4.1 RMSNorm

4.1.1 Implementation

The `rmsnorm<S>` function is declared as a C++ template in `forward.h`, parameterised by the vector length `S`. This makes the same source compile correctly for both the prototype configuration ($S = d_{\text{model}} = 8$) and the full TinyLlama model ($S = d_{\text{model}} = 2048$) without any code changes. The function takes three pointer arguments, namely an output array `o`, an input activation array `x`, and a learned element-wise weight array `weight`, all of length `S`.

The first design decision is to copy both `x` and `weight` into local arrays `x_buff` and `weight_buff` before any computation begins, as shown in Listing 4.1. This is necessary because Vitis HLS can only apply `ARRAY_PARTITION` to arrays whose storage it controls. Arrays received as pointer arguments may alias external memory, which prevents the tool from freely splitting them across multiple BRAM banks. By staging the data into local buffers first, the `ARRAY_PARTITION cyclic factor=4` pragma can be applied, enabling four simultaneous reads per cycle in both computation loops.

```
template <int S>
void rmsnorm(float o[S], float x[S], float weight[S])
{
  float x_buff[S], weight_buff[S], out_buff[S];
  #pragma HLS array_partition variable=x_buff      cyclic factor=4
  #pragma HLS array_partition variable=weight_buff cyclic factor=4
  #pragma HLS array_partition variable=out_buff    cyclic factor=4

  memcpy(x_buff, x, S * sizeof(float));
  memcpy(weight_buff, weight, S * sizeof(float));

  float ss = 0.0f;
sum_of_squares:
  for (int j = 0; j < S; j++) {
    #pragma HLS PIPELINE
    #pragma HLS UNROLL factor=4 skip_exit_check
    ss += x_buff[j] * x_buff[j];
  }
  ss /= S;
  ss += 1e-5f;
  ss = 1.0f / sqrtf(ss);  // reciprocal RMS

norm_and_scale:
  for (int j = 0; j < S; j++) {
    #pragma HLS PIPELINE
```

```
    #pragma HLS UNROLL factor=64
    out_buff[j] = weight_buff[j] * (ss * x_buff[j]);
  }
  memcpy(o, out_buff, S * sizeof(float));
}
```

Listing 4.1: `rmsnorm<S>` implementation from `forward.h`.

The computation proceeds in two sequential loops with different parallelism requirements. The first loop, `sum_of_squares`, accumulates the sum of squared activations. This is a reduction with a loop-carried dependency on the accumulator `ss`, because each iteration reads the current value of `ss`, adds to it, and writes it back. This dependency means the loop cannot be pipelined to II = 1 on its own because the write of iteration t must complete before iteration $t+1$ can read. The `UNROLL factor=4` directive resolves this by unrolling four iterations into the same pipeline stage, so that four independent partial products $x_j^2, x_{j+1}^2, x_{j+2}^2, x_{j+3}^2$ are computed in parallel and their sum is added to `ss` once per group, reducing the effective number of dependent additions by a factor of four.

Once `ss` has been computed, the scalar reciprocal RMS is available and the second loop, `norm_and_scale`, has no loop-carried dependencies. Every iteration reads one element of `x_buff` and one element of `weight_buff`, multiplies by the scalar `ss`, and writes to `out_buff`. These are entirely independent operations, so the loop is amenable to wide unrolling. The `UNROLL factor=64` directive allows up to 64 iterations to execute in parallel, subject to memory port availability, which is satisfied by the `cyclic factor=4` partitioning applied to all three local buffers.

4.1.2 C-Simulation Results

C-simulation was performed for `rmsnorm_top`. The testbench supplied a fixed eight-element input vector and a corresponding weight vector, then compared the kernel output element by element against a pure software reference implementation of RMSNorm. The reference computes the sum of squares sequentially, derives the reciprocal RMS, and applies the element-wise scaling in the same order as the mathematical definition in section 2.2.3, with no HLS pragmas or unrolling. Table 4.2 shows the output from both the kernel and the software reference for each element of the test vector.

Index	HW output	SW reference	Absolute error
0	0.099015	0.099015	0
1	-0.396059	-0.396059	0
2	0.891133	0.891133	0
3	-1.584236	-1.584236	0
4	0.495074	0.495074	0
5	-1.188177	-1.188177	0
6	2.079309	2.079309	0
7	-3.168472	-3.168472	0

Table 4.2: C-simulation output for `rmsnorm`. All eight elements match the software reference.

The absolute error is identically zero for every element, meaning the kernel and the reference produce bit-identical results on this test vector. The zero-error result is attributed to the small prototype dimensions ($S = 8$), where floating-point rounding differences from the `UNROLL factor=4` accumulation order remain below the representational limit of single-precision arithmetic. The csim run completed with zero errors as reported by Vitis HLS.

4.1.3 HLS Synthesis Results

The `rmsnorm` function was synthesised in isolation using the prototype configuration ($S = 8$) on the ZCU102 at a 150 MHz target clock (6.667 ns period). Tables 4.3 and 4.4 reproduce the top-level timing and latency summary as reported by Vitis HLS, followed by Table 4.5 for resource utilisation and Table 4.6 for the per-stage latency breakdown.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	4.867	1.800

Table 4.3: Timing summary for `rmsnorm_top` at the 150 MHz (6.667 ns) target clock.

Latency (cycles)		Latency (absolute)		Interval (cycles)		Pipeline
min	max	min	max	min	max	type
146	146	0.973 µs	0.973 µs	147	147	no

Table 4.4: Top-level latency summary for `rmsnorm_top` at 150 MHz.

Resource	Used (total)	Used (core only)	Available	Utilisation (%)
LUT	4,646	3,575	274,080	1
FF	3,910	2,949	548,160	<1
BRAM	4	0	1,824	<1
DSP	26	26	2,520	1

Table 4.5: Resource utilisation of the `rmsnorm` function synthesised at $S = 8$ on the ZCU102. Core-only figures exclude the AXI master adapter.

Stage	Trip count	II	Iteration latency	Total (cycles)
Load `x` into `x_buff`	8	1	3	9
`sum_of_squares`	2	24	29	52
Load `weight` into `weight_buff`	8	1	3	11
`norm_and_scale`	8	1	2	10

Table 4.6: Latency breakdown of `rmsnorm` ($S = 8$) by internal pipeline stage at 150 MHz.

The total function latency of 146 cycles is dominated by the `sum_of_squares` reduction loop, which alone accounts for 52 of those cycles despite iterating only twice after the `UNROLL factor=4` collapses the 8-element loop into 2 effective iterations. The cause is the loop-carried dependency on the floating-point accumulator `ss`. The Bind Op report shows that HLS mapped the floating-point addition to a full-DSP `fadd` `faddfsub_32ns_32ns_32_6_full_dsp_1` unit with a pipeline latency of 5 cycles at this clock frequency. Because each iteration must read the current value of `ss` and write back an updated value, the next iteration cannot begin until the adder has produced its result, giving an effective II of 24 cycles. The `UNROLL factor=4` partially mitigates this by reducing the trip count from 8 to 2, cutting the number of stall events in half, but it cannot eliminate the dependency itself. A more complete solution would be accumulator interleaving, where four independent partial sums are maintained and merged only at the end, allowing all four lanes to pipeline at II = 1 independently. This is identified as a future optimisation opportunity.

In contrast, the `norm_and_scale` loop achieves II = 1 perfectly. Once the scalar reciprocal RMS `ss` is available, every element's computation is independent of all others, allowing HLS to start a new element every clock cycle. The `UNROLL factor=64` on an 8-element loop effectively fully unrolls it into 8 parallel datapath lanes, which is reflected in the 26 DSPs consumed. The 8 floating-point multipliers for the `weight` $\times$ (`ss` $\times$ `x`)

computation each map to 3 DSPs, accounting for 24 of the 26 total, with the remaining 2 used by the `fadd` in the reduction.

The 4 BRAMs in the total utilisation are consumed entirely by the AXI master adapter (`gmem0_m_axi`) of the standalone wrapper and are not a property of the `rmsnorm` logic itself. The `rmsnorm` core uses no BRAM, since the local buffers `x_buff`, `weight_buff`, and `out_buff` at $S = 8$ are small enough to be inferred as register files. The estimated clock period of 4.867 ns leaves 1.800 ns of uncertainty budget, which meets the 6.667 ns target with zero slack remaining.

4.1.4 Full TinyLlama Scale

Implementation

The sixth kernel iteration scales `rmsnorm` to the full TinyLlama hidden dimension ($S = d_{\text{model}} = 2048$). The source template is unchanged in its algorithmic structure but carries three targeted pragma modifications, as shown in Listing 4.2.

```
template <int S>
void rmsnorm(float o[S], float x[S], float weight[S]) {
  constexpr auto array_size = S * sizeof(float);

  float ss = 0.0f;
  float x_buff[S];
  float weight_buff[S];
  float out_buff[S];
  #pragma HLS ARRAY_PARTITION variable=x_buff      type=cyclic
      factor=16
  #pragma HLS ARRAY_PARTITION variable=weight_buff type=cyclic
      factor=16
  #pragma HLS ARRAY_PARTITION variable=out_buff    type=cyclic
      factor=16

  memcpy(x_buff, x, array_size);
  memcpy(weight_buff, weight, array_size);

sum_of_squares:
  for (int j = 0; j < S; j++) {
    #pragma HLS PIPELINE II=1
    #pragma HLS UNROLL factor=16
    ss += x_buff[j] * x_buff[j];
  }
  ss /= S;
  ss += 1e-5f;
  ss = 1.0f / sqrtf(ss);
```

```
norm_and_scale:
  for (int j = 0; j < S; j++) {
    #pragma HLS PIPELINE II=1
    #pragma HLS UNROLL factor=16
    out_buff[j] = weight_buff[j] * (ss * x_buff[j]);
  }
  memcpy(o, out_buff, array_size);
}
```

Listing 4.2: `rmsnorm<S>` implementation from `FFN_update3.h` at full TinyLlama scale ($S = 2048$).

The first change is to the `ARRAY_PARTITION cyclic factor`, which increases from 4 to 16 on all three local buffers. At $S = 8$ in the prototype, four banks sufficed because the loop bodies accessed at most four elements simultaneously. At $S = 2048$, 16 simultaneous accesses are needed to keep all unrolled lanes supplied without bank contention, and factor = 16 is therefore the minimum that avoids port conflicts. Each 2048-element float array partitioned with factor = 16 produces $2048/16 = 128$ words per bank, which falls above the BRAM inference threshold and causes HLS to map each bank to a BRAM rather than a register file, causing large on-chip memory usage.

The second change affects the `sum_of_squares` loop, where the `UNROLL factor` increases from 4 to 16. In the prototype this loop iterated over $S = 8$ elements, so an unroll factor of 4 reduced the trip count to 2. At $S = 2048$, a factor of 16 reduces the trip count to 128, creating 16 independent partial accumulator lanes that each carry their own loop-carried dependency chain. HLS schedules all 16 lanes in parallel, achieving an effective throughput of 16 elements per iteration despite the 5-cycle floating-point addition latency on each lane. The pipeline II is explicitly constrained to 1 with the added `II=1` annotation.

The third change affects the `norm_and_scale` loop, where the `UNROLL factor` decreases from 64 to 16. In the prototype with $S = 8$, an unroll factor of 64 effectively fully unrolled the loop into 8 parallel lanes. At $S = 2048$, maintaining an unroll factor of 64 would instantiate 64 parallel `fmul` units, each consuming 3 DSPs, for a total of 192 DSPs from this loop alone. Since the `norm_and_scale` computation has no loop-carried dependencies, a factor of 16 already achieves II = 1 with full utilisation of the 16-bank partitioned buffers.

C-Simulation Results

The testbench supplied a 2048-element input vector with incrementally increasing values and unit RMSNorm weights. All 2048 output elements agreed with the sequential software reference with zero absolute error, giving an RMS error of 0 across the full vector. The csim run completed with zero errors as reported by Vitis HLS.

HLS Synthesis Results

Tables 4.7 and 4.8 present the timing and latency summary. Table 4.9 gives the resource utilisation and Table 4.10 the per-stage breakdown.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	7.456	1.800

Table 4.7: Timing summary for `rmsnorm_update3_top` at the 150 MHz target clock.

Latency (cycles)		Latency (absolute)		Interval (cycles)		Pipeline
min	max	min	max	min	max	type
18,645	18,645	0.139 ms	0.139 ms	18,646	18,646	no

Table 4.8: Top-level latency summary for `rmsnorm_update3_top` at 150 MHz.

Resource	Used (total)	Used (core only)	Available	Utilisation (%)
LUT	11,977	9,954	274,080	4
FF	9,917	8,458	548,160	1
BRAM	100	96	1,824	5
DSP	98	98	2,520	3

Table 4.9: Resource utilisation of `rmsnorm` at full scale ($S = 2048$) on the ZCU102. Core-only figures exclude the AXI master adapter.

Stage	Module	Latency (cycles)
Load x_in, weight	VITIS_LOOP_29_1	4,106
rmsnorm_2048_s core	rmsnorm_2048_s	12,478
Write output	VITIS_LOOP_37_2	2,051

Table 4.10: Per-stage latency breakdown of rmsnorm at full scale ($S = 2048$) at 150 MHz.

The synthesis reports a timing violation at 150 MHz. The prototype at $S = 8$ estimated 4.867 ns and met the 6.667 ns target. At $S = 2048$ the estimated path is 7.456 ns, exceeding the budget by 2.589 ns. The cause is the wide parallel reduction in the sum_of_squares loop. With UNROLL factor=16, HLS instantiates 16 floating-point multipliers whose products are summed through a tree of adders. At $S = 8$ with UNROLL factor=4, the tree has two levels. At $S = 2048$ with UNROLL factor=16, it has four levels, and each level contributes one floating-point addition delay to the combinational path. This depth prevents timing closure at 150 MHz. Reducing the unroll factor to 8 would shorten the tree and resolve the violation.

The BRAM count increases from 0 core BRAMs at $S = 8$ to 96 core BRAMs at $S = 2048$. At prototype scale, all six local buffer arrays (x_buff, weight_buff, and out_buff in both the wrapper and the inlined core) fit in register files. At $S = 2048$ with factor = 16 partitioning, each array is split into 16 banks of 128 words each. A 128-word bank exceeds the BRAM inference threshold, so HLS maps each bank to a single-port BRAM, giving $6 \times 16 = 96$ core BRAMs in total.

The DSP count increases from 26 at prototype scale to 98 at full scale. The increase comes from three sources. The first is the 16 fmul units in norm_and_scale, consuming 48 DSPs. The second is the 16 fmul units in sum_of_squares, consuming another 48 DSPs. The third is the fadd accumulator, accounting for the remaining 2 DSPs.

The LUT count of 9,954 core LUTs is higher than the 3,575 of the prototype. The increase comes from three sources. The first is the wider multiplexer network arbitrating 16-bank memory accesses. The second is the larger FSM for the expanded loop bounds at $S = 2048$. The third is the combinational logic of the four-level adder tree. The FF count grows for the same reasons, as each pipeline register stage in the adder tree adds flip-flop storage proportional to the data width.

4.2 Quantisation

4.2.1 Implementation

The `quantize<S>` function implements the group-wise symmetric INT8 quantisation scheme described in section 2.5. It is declared as a C++ template in `forward.h`, parameterised by the vector length `S`, and outputs results through the `QuantizedTensor<S>` struct. A `QuantizedTensor<S>` bundles an `int8_t` array `q[S]` for the quantised values and a `float` array `s[S]` for the scale factors, where only `S/GS` scale entries are populated, one per group of `GS` consecutive elements.

The `quantize<S>` function processes the input float vector in groups of `GS` elements, as shown in Listing 4.3. For each group, the function first scans the `GS` elements to find the maximum absolute value `wmax`, then computes the group scale as $s = \texttt{wmax}/127.0$, which maps the largest magnitude in the group to the maximum representable INT8 value of 127. Each element is then divided by the scale and rounded to the nearest integer, producing a value in $[-127, 127]$ that is stored as `int8_t`. The quantised values and scales are accumulated into local buffers `quantized_buffer[S]` and `scale_buffer[num_groups]` before being written to the output struct via `memcpy`.

```
template <int S>
void quantize(QuantizedTensor<S> *qx, float x[S], int runtime_gs) {
  int num_groups = S / GS;

  float  scale_buffer[num_groups];
  int8_t quantized_buffer[S];
  #pragma HLS ARRAY_PARTITION variable=quantized_buffer cyclic
      factor=4
  #pragma HLS ARRAY_PARTITION variable=scale_buffer     cyclic
      factor=4

  for (int group = 0; group < num_groups; group++) {
    #pragma HLS UNROLL factor=1
    #pragma HLS PIPELINE
    float wmax = 0.0f;
    int base_idx = group * GS;

    for (int i = 0; i < GS; i++) {  // find max abs value in group
      #pragma HLS PIPELINE
      float val = fabs(x[base_idx + i]);
      if (val > wmax) wmax = val;
    }

    float scale = wmax / 127.0f;  // symmetric INT8 scale
```

```
    scale_buffer[group] = scale;

    for (int i = 0; i < GS; i++) {  // quantise each element
      #pragma HLS PIPELINE
      quantized_buffer[base_idx + i] =
        (int8_t)round(x[base_idx + i] / scale);
    }
  }
  memcpy(qx->q, quantized_buffer, S * sizeof(int8_t));
  memcpy(qx->s, scale_buffer, num_groups * sizeof(float));
}
```

Listing 4.3: Group-wise quantisation in `quantize<S>` from `forward.h`.

`GS` is declared as a compile-time constant in `config.h`, which allows the compiler to statically evaluate `S/GS` as `num_groups`. This enables the `ARRAY_PARTITION` pragma on `quantized_buffer` and allows the inner loops to be analysed at synthesis time without variable-bound ambiguity.

The `PIPELINE` pragma on the outer group loop combined with `UNROLL factor=1` is an intentional configuration. Applying `PIPELINE` to the outer loop while leaving the inner scan and quantisation loops unpipelined would normally cause HLS to flatten the inner loops into the outer pipeline, which at `GS=4` means four sequential comparisons per outer iteration. The `UNROLL factor=1` prevents the outer loop from being unrolled further.

4.2.2 C-Simulation Results

C-simulation was performed for `quantize_top`. The testbench supplied an eight-element float vector and verified both the INT8 quantised values and the per-group scale factors against a software reference. Table 4.11 shows the INT8 output and scale factors produced by the kernel and the software reference for each element and group respectively.

Index	Input (x)	HW q	SW q	Abs error
0	0.5	17	17	0
1	-1.2	-41	-41	0
2	3.7	127	127	0
3	-0.3	-10	-10	0
4	2.1	59	59	0
5	-4.5	-127	-127	0
6	1.0	28	28	0
7	-2.8	-79	-79	0
Group	**HW scale**	**SW scale**	**Abs error**	
0	0.029134	0.029134	0	
1	0.035433	0.035433	0	

Table 4.11: C-simulation output for `quantize`. All INT8 values and both scale factors match the software reference.

All eight INT8 values and both scale factors agree between the hardware kernel and the software reference. Elements at index 2 and index 5 map to the boundary values of 127 and -127 respectively, confirming that the maximum-magnitude element in each group correctly saturates the INT8 range. The scale for group 0 is $3.7/127 \approx 0.02913$, and for group 1 is $4.5/127 \approx 0.03543$, both of which match the reference to the displayed precision. The csim run completed with zero errors.

4.2.3 HLS Synthesis Results

The `quantize` function was synthesised in isolation at 150 MHz on the ZCU102. Tables 4.12 and 4.13 present the top-level timing and latency summary, followed by Table 4.14 for resource utilisation and Table 4.15 for the per-stage breakdown.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	4.867	1.800

Table 4.12: Timing summary for `quantize_top` at 150 MHz.

Latency (cycles)		Latency (absolute)		Interval (cycles)		Pipeline
min	max	min	max	min	max	type
84	84	0.560 µs	0.560 µs	85	85	no

Table 4.13: Top-level latency summary for `quantize_top` at 150 MHz.

Resource	Used (total)	Used (core only)	Available	Utilisation (%)
LUT	6,379	3,275	274,080	2
FF	3,556	1,653	548,160	<1
BRAM	4	0	1,824	<1
DSP	0	0	2,520	0

Table 4.14: Resource utilisation of `quantize` synthesised at $S = 8$ on the ZCU102. The core-only column excludes the AXI master adapter.

Stage	Trip count	II	Iteration latency	Total (cycles)
`main_loop` (quantise core)	2	4	41	44
Pipeline_2 (write `q_out`)	8	1	1	8
Pipeline_3 (write `s_out`)	2	1	1	2

Table 4.15: Latency breakdown of `quantize` ($S = 8$, $GS = 4$) by internal pipeline stage at 150 MHz.

The zero DSP utilisation is the most notable result. In contrast to RMSNorm, which consumed 26 DSPs for its floating-point multiplications, `quantize` uses none. The Bind Op report confirms that both division operations, namely computing the group scale as $s = \texttt{wmax}/127$ and quantising each element as $q_i = x_i/s$, are mapped to fabric-based `fdiv` units with the `no_dsp` suffix. This occurs because Xilinx DSP48 blocks natively support only multiply-accumulate operations. Floating-point division has no direct DSP mapping and must instead be implemented using LUT-based logic. The consequence is higher LUT usage (3,275 core LUTs versus 3,575 for RMSNorm) but zero DSP consumption.

The `main_loop` achieves II = 4 across its two group iterations, meaning a new group begins every 4 clock cycles. This is determined by the inner loop over $GS = 4$ elements within each group. Since the inner quantisation loop is pipelined at II = 1, processing four elements takes four cycles and the outer group loop can start its next iteration once

those four cycles have elapsed. The iteration latency of 41 cycles is dominated by the two `fdiv` units `fdiv_32ns_32ns_32_12_no_dsp_1`, each with an 11-cycle pipeline depth at 150 MHz. Two `fdiv` instances are synthesised, one for the scale computation and one for the per-element quantisation divisions, allowing the two operations to overlap in the pipeline. The intermediate `QuantizedTensor` struct fields `qt.q` and `qt.s` are inferred as small register files (`qt_q_U` and `qt_s_U`) rather than BRAMs, since their sizes of 8 bytes and 32 bytes respectively fall well below the BRAM threshold. The estimated 4.867 ns plus 1.80 ns uncertainty exhausts the 6.667 ns budget, leaving zero slack.

4.2.4 Full TinyLlama Scale

Implementation

The sixth iteration applies four targeted changes to `quantize`, shown in Listing 4.4. The `ARRAY_PARTITION cyclic factor` on `quantized_buffer` increases from 4 to 16, matching `PARALLEL_MACS = 16` and providing 16 independent write ports for the quantisation loop. The `runtime_gs` parameter is removed, and the function uses the compile-time constant `GS = 64` directly. With $\mathrm{GS} = 64 = 2^6$, the group index computation `i/GS` synthesises as a 6-bit right shift at no hardware cost. The `round()` call in the element quantisation step is replaced by manual arithmetic before truncation. In the prototype, `round()` caused HLS to infer a `generic_round_double_s` rounding IP consuming 4 BRAMs per call. The manual rounding eliminates this IP. A reciprocal trick is applied to the group scale, computing `inv_scale = 1.0f/scale` once per group and multiplying per element.

```
template <int S>
void quantize(QuantizedTensor<S> *qx, float x[S]) {
  constexpr int   num_groups = S / GS;
  constexpr float Q_MAX      = 127.0f;

  float  scale_buffer[num_groups];
  int8_t quantized_buffer[S];
  #pragma HLS ARRAY_PARTITION variable=quantized_buffer cyclic
      factor=16
  #pragma HLS ARRAY_PARTITION variable=scale_buffer     cyclic
      factor=4

quant_groups:
  for (int group = 0; group < num_groups; group++) {
    int base_idx = group * GS;
    float wmax = 0.0f;
```

```
find_max:
    for (int i = 0; i < GS; i++) {
      #pragma HLS PIPELINE II=1
      float val = fabs(x[base_idx + i]);
      if (val > wmax) wmax = val;
    }
    float scale     = wmax / Q_MAX;
    float inv_scale = (scale != 0.0f) ? (1.0f / scale) : 0.0f;
    scale_buffer[group] = scale;
quantize_elems:
    for (int i = 0; i < GS; i++) {
      #pragma HLS PIPELINE II=1
      float quant_value = x[base_idx + i] * inv_scale;
      int32_t rounded   = (int32_t)(quant_value +
                 (quant_value >= 0.0f ? 0.5f : -0.5f));
      if (rounded >  127) rounded =  127;
      if (rounded < -127) rounded = -127;
      quantized_buffer[base_idx + i] = (int8_t)rounded;
    }
  }
  memcpy(qx->q, quantized_buffer, S * sizeof(int8_t));
  memcpy(qx->s, scale_buffer, num_groups * sizeof(float));
}
```

Listing 4.4: `quantize<S>` implementation from `FFN_update3.h` at full TinyLlama scale ($S = 2048$, $\mathrm{GS} = 64$).

C-Simulation Results

The testbench supplied a 2048-element input vector across 32 groups of $\mathrm{GS} = 64$ elements each. All INT8 values and all 32 group scales agreed with the software reference with zero error, and the csim run completed with zero errors as reported by Vitis HLS.

HLS Synthesis Results

Tables 4.16 and 4.17 present the timing and latency summary. Table 4.18 gives the resource utilisation and Table 4.19 the per-stage breakdown.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	4.867	1.800

Table 4.16: Timing summary for `quantize_update3_top` at the 150 MHz target clock.

Latency (cycles)		Latency (absolute)		Interval (cycles)		Pipeline
min	max	min	max	min	max	type
10,362	10,362	69.083 µs	69.083 µs	10,363	10,363	no

Table 4.17: Top-level latency summary for `quantize_update3_top` at 150 MHz.

Resource	Used (total)	Used (core only)	Available	Utilisation (%)
LUT	4,919	3,874	274,080	1
FF	2,874	2,362	548,160	<1
BRAM	22	18	1,824	1
DSP	5	5	2,520	<1

Table 4.18: Resource utilisation of `quantize` at full scale ($S = 2048$, GS $= 64$, 32 groups) on the ZCU102. Core-only figures exclude the AXI master adapter.

Stage	Module	Latency (cycles)
Load `x_in`	`VITIS_LOOP_28_1`	2,056
`quantize_2048_s` core	`quantize_2048_s`	8,260
Write `q_out`, `s_out`	`VITIS_LOOP_32_2`	35

Table 4.19: Per-stage latency breakdown of `quantize` at full scale ($S = 2048$) at 150 MHz.

Timing meets the 6.667 ns constraint with the same estimated path of 4.867 ns as the prototype, confirming that the implementation changes in iteration 6 did not lengthen the critical path.

The total latency of 10,362 cycles (69.083 µs) is dominated by the `quantize_2048_s` core at 8,260 cycles, which accounts for 80% of the total. Loading the 2,048-element input float vector takes 2,056 cycles, and writing the quantised output (`q_out` and `s_out`) takes only 35 cycles due to the small output size of 2,048 bytes of INT8 values and 32 floats of scale factors. The asymmetry between the 2,056-cycle load and the 35-cycle write reflects the difference in transfer volume, with the input being $2048 \times 4 = 8{,}192$ bytes versus the output's 2,176 bytes.

The core latency of 8,260 cycles for 32 groups warrants examination. Each group executes a `find_max` scan of 64 elements pipelined at II = 1 (64 cycles) followed by a `quantize_elems` loop of 64 elements also at II = 1 (64 cycles), plus the overhead of

one `fdiv` to compute `scale` and one `fdiv` to compute `inv_scale`. At 150 MHz, each `fdiv` unit has a pipeline latency of approximately 11 cycles. The total per-group cost is therefore approximately $64 + 64 + 2 \times 11 = 150$ cycles, and across 32 groups this gives $32 \times 150 + \text{overhead} \approx 4{,}800$ cycles for the pure arithmetic, with the remainder from AXI burst sequencing inside the core module.

The BRAM count of 18 core BRAMs is higher than the prototype's zero-BRAM result. At prototype scale ($S = 8$, 2 groups), all internal buffers fit in register files. At $S = 2048$ with 32 groups, the `quantized_buffer[2048]` array partitioned cyclic factor = 16 creates 16 banks of 128 int8 words each, which HLS maps to 16 BRAMs. The `scale_buffer[32]` float array with cyclic factor = 4 is small enough (8 words per bank) to remain as registers, contributing no BRAM. The remaining 2 BRAMs come from the `qt.q` and `qt.s` fields of the output `QuantizedTensor` struct, which at $S = 2048$ exceed the register threshold. No `generic_round_double_s` instances appear anywhere in the Memory or Instance sections of the synthesis report, confirming that the manual rounding arithmetic eliminates the rounding IP entirely.

The DSP count increases from 0 in the prototype to 5 in this version. The prototype used `round()` and per-element `fdiv`, both of which map to LUT fabric with no DSP involvement. The reciprocal trick in iteration 6 introduces one `fmul` per element, which maps to a 3-DSP `fmul` unit, and an `fadd` that accounts for the remaining 2 DSPs. The `fdiv` units for `scale` and `inv_scale` continue to use LUT fabric, consistent with the prototype behaviour. The net effect is a small but acceptable DSP increase in exchange for eliminating the rounding IP's BRAM cost and the pipeline stalls from per-element division.

4.3 Dequantisation

4.3.1 Implementation

The `dequantize<S>` function is the inverse of `quantize<S>`. For each element at index i, it multiplies the INT8 value `qx->q[i]` by the corresponding group scale `qx->s[i/GS]`, recovering an approximation of the original float. The division `i/GS` computes the group index, and because `GS` is a power of two, the compiler replaces this with a bitwise right shift at no hardware cost. No pragma directives are applied to `dequantize` in the prototype, since it operates on a single vector element at a time with no loop-carried dependencies and no memory partitioning decisions to make.

The maximum reconstruction error per element is bounded by $\frac{1}{2} \cdot s = \frac{\texttt{wmax}}{254}$, which for typical activation magnitudes of order 1.0 gives an absolute error of approximately 0.004. This error is acceptable for the attention projection inputs but motivates the

design decision in the full TinyLlama kernel to store KV cache values with per-group scales rather than a single scale shared across the full kv_dim-dimensional vector. Listing 4.5 shows the complete function body.

```
template <int S>
void dequantize(QuantizedTensor<S> *qx, float x[S], int runtime_gs)
{
  for (int i = 0; i < S; i++)
  {
    x[i] = qx->q[i] * qx->s[i / GS];
  }
}
```

Listing 4.5: dequantize<S> implementation from forward.h.

4.3.2 C-Simulation Results

C-simulation was performed for dequantize_top. The testbench supplied eight known INT8 values along with two explicit group scale factors, one per GS = 4 element group, then compared the kernel output element by element against a software reference computing $x_{\text{ref}}[i] = q[i] \times s_{\lfloor i/GS \rfloor}$ directly. The same group scale factors from the quantize csim run ($s_0 = 0.029134$, $s_1 = 0.035433$) were reused, making the two testbenches complementary. Table 4.20 shows the kernel and reference outputs for each element.

Index	HW output	SW reference	Absolute error
0	1.835442	1.835442	0
1	-2.942534	-2.942534	0
2	3.700018	3.700018	0
3	-1.107092	-1.107092	0
4	2.976372	2.976372	0
5	-4.499991	-4.499991	0
6	1.807083	1.807083	0
7	-4.039362	-4.039362	0

Table 4.20: C-simulation output for dequantize. All eight dequantised float values match the software reference with zero absolute error.

All eight dequantised float values agree with the software reference, with zero absolute error across every element. Elements at indices 2 and 5, corresponding to the INT8 boundary values 127 and -127, correctly reconstruct to 3.700018 and -4.499991 re-

spectively, confirming that the scale multiplication path handles boundary saturation without loss. The zero-error result is expected for dequantisation. Unlike quantisation, which introduces an irreducible rounding error at the point of integer conversion, dequantisation is a pure floating-point multiplication with no rounding decision, and the kernel output is therefore bit-identical to the sequential software reference on every element. The csim run completed with zero errors as reported by Vitis HLS.

4.3.3 HLS Synthesis Results

The `dequantize` function was synthesised in isolation under identical conditions to `quantize`. Tables 4.21 and 4.22 present the top-level timing and latency summary, followed by Table 4.23 for resource utilisation and Table 4.24 for the per-stage breakdown.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	4.867	1.800

Table 4.21: Timing summary for `dequantize_top` at 150 MHz.

Latency (cycles)		Latency (absolute)		Interval (cycles)		Pipeline
min	max	min	max	min	max	type
45	45	0.300 µs	0.300 µs	46	46	no

Table 4.22: Top-level latency summary for `dequantize_top` at 150 MHz.

Resource	Used (total)	Used (core only)	Available	Utilisation (%)
LUT	3,128	273	274,080	1
FF	2,244	372	548,160	<1
BRAM	4	0	1,824	<1
DSP	3	3	2,520	<1

Table 4.23: Resource utilisation of `dequantize` synthesised at $S = 8$ on the ZCU102. The core-only column excludes the AXI master adapter.

Stage	Trip count	II	Iteration latency	Total (cycles)
`VITIS_LOOP_55_1` (dequantise loop)	8	1	12	18

Table 4.24: Latency breakdown of `dequantize` ($S = 8$, $GS = 4$) at 150 MHz.

There is an asymmetry between the two inverse operations. `Dequantize` is nearly twice as fast (45 cycles versus 84 cycles), uses only a twelfth of the core LUTs (273 versus 3,275), and achieves a perfect II = 1 on its inner loop. The root cause of this asymmetry is the difference between the arithmetic operations each function requires.

`Dequantize` performs a single floating-point multiplication per element, $x_i = q_i \times s_{\lfloor i/GS \rfloor}$, which maps to one `fmul` `fmul_32ns_32ns_32_4_max_dsp_1` unit using 3 DSP blocks with a 3-cycle pipeline latency. Since each element's computation depends only on its own INT8 value and its group scale with no dependency on any previous element, there are no loop-carried dependencies, and II = 1 is achievable. The 12-cycle iteration latency is dominated by the AXI read latency for loading `q_in` and `s_in` from DDR, with the 3-cycle `fmul` pipeline depth making up the remainder.

For the `quantize` function, two floating-point divisions are required per group, one to compute $s = \texttt{wmax}/127$ and one per element for $\hat{x}_i = x_i/s$. Unlike multiplication, floating-point division has no mapping to Xilinx DSP48 blocks and is implemented entirely in LUT fabric, as confirmed by the `no_dsp` suffix on both `fdiv` instances in the Bind Op report. This explains why `quantize` uses zero DSPs while using significantly more LUTs. The 11-cycle `fdiv` latency at 150 MHz, combined with the sequential dependency between scale computation and element quantisation within each group, is what drives the 41-cycle iteration latency in `main_loop`.

The `quantize` Bind Storage report also shows two small ROM lookup tables, `mask_table` and `one_half_table`, which HLS infers to implement the IEEE 754 round-to-nearest operation used by the `round()` call. These tables are inferred in register fabric rather than BRAM at this prototype scale and contribute to the higher LUT count. The AXI data width report for `dequantize` also shows a difference in that the `q_in` port operates at an 8-bit native width widened to a 32-bit AXI bus width ($8 \rightarrow 32$), reflecting the fact that INT8 data must be packed into the 32-bit AXI data bus in groups of four bytes per transfer.

4.3.4 Full TinyLlama Scale

Implementation

The sixth iteration applies two changes to `dequantize`, shown in Listing 4.6. The `runtime_gs` parameter is removed and the function uses the compile-time constant `GS = 64` directly. A `#pragma HLS PIPELINE II = 1` directive is added to the `dequant_loop`.

```
1 template <int S>
2 void dequantize(QuantizedTensor<S> *qx, float x[S]) {
3 dequant_loop:
```

```
  for (int i = 0; i < S; i++) {
    #pragma HLS PIPELINE II=1
    x[i] = qx->q[i] * qx->s[i / GS];
  }
}
```

Listing 4.6: `dequantize<S>` from `FFN_update3.h` at full TinyLlama scale ($S = 2048$, $\mathrm{GS} = 64$).

The algorithmic content is unchanged. For each element at index i, the INT8 value `qx->q[i]` is multiplied by the group scale `qx->s[i/GS]`, where the division `i/GS` synthesises as a 6-bit right shift since $\mathrm{GS} = 64 = 2^6$. The added `PIPELINE II = 1` annotation does not alter the datapath and simply makes the throughput constraint visible in the schedule report.

C-Simulation Results

The testbench supplied 2048 INT8 values with a uniform scale of 0.03. All 2048 output elements agreed with the software reference with zero absolute error, and the csim run completed with zero errors as reported by Vitis HLS.

HLS Synthesis Results

Tables 4.25 and 4.26 present the timing and latency summary. Table 4.27 gives the resource utilisation and Table 4.28 the per-stage breakdown.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	4.867	1.800

Table 4.25: Timing summary for `dequantize_update3_top` at the 150 MHz target clock.

Latency (cycles)		Latency (absolute)		Interval (cycles)		Pipeline
min	max	min	max	min	max	type
6,223	6,223	41.489 µs	41.489 µs	6,224	6,224	no

Table 4.26: Top-level latency summary for `dequantize_update3_top` at 150 MHz.

Resource	Used (total)	Used (core only)	Available	Utilisation (%)
LUT	3,003	2,423	274,080	1
FF	1,855	1,343	548,160	<1
BRAM	22	18	1,824	1
DSP	3	3	2,520	<1

Table 4.27: Resource utilisation of `dequantize` at full scale ($S = 2048$, GS $= 64$) on the ZCU102. Core-only figures exclude the AXI master adapter (`gmem0_m_axi`).

Stage	Module	Latency (cycles)
Load `q_in`	`VITIS_LOOP_25_1`	2,058
Load `s_in`	`VITIS_LOOP_29_2`	35
`dequant_loop`	`Pipeline_dequant_loop`	2,060
Write `x_out`	`VITIS_LOOP_39_3`	2,051

Table 4.28: Per-stage latency breakdown of `dequantize` at full scale ($S = 2048$) at 150 MHz.

Timing meets the 6.667 ns constraint with the same 4.867 ns estimated path as the prototype. The added `PIPELINE II = 1` annotation does not affect the critical path since `dequantize` already achieved II = 1 in the prototype.

The total latency of 6,223 cycles is distributed across four sequential stages. Loading the 2,048 INT8 input values from DDR takes 2,058 cycles. Loading the 32 float scale values takes 35 cycles, since the transfer is only $32 \times 4 = 128$ bytes. The `dequant_loop` executes for 2,060 cycles at II = 1. Writing the 2,048 float outputs takes 2,051 cycles. The total is therefore dominated by AXI transfer time rather than arithmetic, since the three 2,048-element transfer stages each cost approximately 2,055 cycles while the compute itself takes only one `fmul` per element pipelined at II = 1.

The 18 core BRAMs arise from the scale-up from $S = 8$ to $S = 2048$. At prototype scale all buffers fitted in register files. At full scale the output buffer `x_buf[2048]`, partitioned cyclic factor = 16, creates 16 banks of 128 floats each. At 128 words per bank, each bank exceeds the register inference threshold and maps to a BRAM, giving 16 BRAMs. The `qt.q[2048]` field of the input `QuantizedTensor` struct adds 1 BRAM for the 2,048-byte INT8 array, and `qt.s[32]` adds 1 further BRAM for the 32 float scales, totalling 18 core BRAMs. The 3 DSPs come from the single `fmul` unit in the `dequant_loop` pipeline, unchanged from the prototype.

4.4 Matrix–Vector Multiplication

4.4.1 Implementation

The `matmul<N,D>` function computes the quantised matrix–vector product $\mathbf{W}\mathbf{x} \rightarrow \mathbf{y}$, where $\mathbf{W} \in \mathbb{Z}^{D \times N}$ is an INT8 weight matrix stored in row-major order and $\mathbf{x} \in \mathbb{Z}^{N}$ is an INT8 input vector. Both operands are accompanied by their group-wise scale factors, and the output $\mathbf{y} \in \mathbb{R}^{D}$ is returned as a dequantised float vector. The function is declared as a C++ template in `forward.h`, parameterised by `N` (input dimension) and `D` (output dimension), and is instantiated at the call sites in `forward.cpp` for the Q, K, V, and output projections. Listing 4.7 shows the core loop structure.

```
template <int N, int D>
void matmul(float *xout, QuantizedTensor<N> *xq, int8_t *wq, float *
    ws) {
  static int8_t x_buffer[N];
  static float  xs_buffer[N/GS];
  #pragma HLS ARRAY_PARTITION variable=x_buffer  cyclic factor=16
  #pragma HLS ARRAY_PARTITION variable=xs_buffer cyclic factor=4

x_buff:  for (int j = 0; j < N;    j++) x_buffer[j]  = xq->q[j];
xs_buff: for (int j = 0; j < N/GS; j++) xs_buffer[j] = xq->s[j];

  for (int i = 0; i < D; i++) {
  #pragma HLS PIPELINE
  float val = 0.0f;
  int8_t w_buffer[N];
  float  ws_buffer[N/GS];
  #pragma HLS ARRAY_PARTITION variable=w_buffer  cyclic factor=32
  #pragma HLS ARRAY_PARTITION variable=ws_buffer cyclic factor=32

matmul1:  for (int j = 0; j < N;    j++) // no UNROLL: deliberate
        w_buffer[j]  = wq[j + i*N];
matmul2:  for (int j = 0; j < N/GS; j++) // no UNROLL: deliberate
        ws_buffer[j] = ws[i*(N/GS) + j];

matmul3:  for (int j = 0; j <= N-GS; j += GS) {
        int32_t ival = 0;
matmul4:    for (int k = 0; k < GS; k++) // UNROLL commented out
        ival += (int32_t)x_buffer[j+k] * (int32_t)w_buffer[j+k];
        val += (float)ival * ws_buffer[j/GS] * xs_buffer[j/GS];
      }
  xout[i] = val;
  }
```

```
}
```

Listing 4.7: Core loop structure of `matmul<N,D>` from `forward.h`.

The implementation is organised into three phases. The first phase pre-loads the input vector and its associated scale factors into two static local buffers before any row computation begins. The input buffer `x_buffer[N]` receives the INT8 input values via the `x_buff` loop with `UNROLL factor=16`, and the scale buffer `xs_buffer[N/GS]` receives the per-group float scales via the `xs_buff` loop with `UNROLL factor=4`. Both buffers are declared `static` so that HLS allocates them as standalone addressable memory structures rather than dissolving them into pipeline wires within the datapath. This is necessary because `ARRAY_PARTITION` can only be applied to a dedicated memory structure with a fixed address space. Without `static`, HLS may embed the array directly into the pipeline datapath as a set of registers without creating a separately partitionable memory bank. Both are partitioned with `ARRAY_PARTITION cyclic` to provide sufficient read ports for the downstream compute loops. Pre-loading into local buffers serves the same purpose as in `rmsnorm`, separating the AXI burst reads for the input from the per-row AXI burst reads for the weights, preventing contention on the shared `gmem0` port.

The second and third phases execute inside the outer row loop over `D` output elements, which has the `#pragma HLS PIPELINE` directive. For each row `i`, HLS infers two non-static local buffers, `w_buffer[N]` and `ws_buffer[N/GS]`, each partitioned with `ARRAY_PARTITION cyclic factor=32`. The `matmul1` loop loads the `N` INT8 weight values for row `i` from DDR into `w_buffer`, and `matmul2` loads the corresponding `N/GS` scale factors into `ws_buffer`.

The third phase is the grouped integer dot product. The `matmul3` loop iterates over groups of `GS` consecutive elements, and within each group the `matmul4` loop accumulates the products into a 32-bit integer accumulator `ival`. Using `int32_t` for the accumulator is needed because the product of two INT8 values can reach $127 \times 127 = 16{,}129$, and summing `GS = 4` such products gives a maximum of $4 \times 16{,}129 = 64{,}516$, which fits within `int16_t` but would overflow `int8_t`. Accumulating into `int32_t` prevents any possibility of overflow. After all `GS` products have been accumulated, the group result is dequantised inline. The integer sum is cast to float and multiplied by the product of the weight group scale `ws_buffer[j/GS]` and the input group scale `xs_buffer[j/GS]`, recovering the floating-point partial sum for that group. This per-group dequantisation is added directly to the running float accumulator `val` before moving to the next group.

4.4.2 C-Simulation Results

C-simulation was performed for `matmul_top`. The testbench constructs an 8×8 weight matrix whose diagonal entries are all 127 and whose off-diagonal entries are all zero, with all weight scales set to $1/127$. With this configuration the computation simplifies to $\texttt{xout}[i] = \texttt{xq}[i] \times \texttt{xs}[\lfloor i/GS \rfloor]$, which is the dequantised input vector. The testbench reuses the quantised values and scale factors from the `quantize` csim as input, making the three testbenches a matched chain in which quantise, identity matrix–vector multiply, and dequantise can be cross-validated against one another. Table 4.29 shows the output from both the kernel and the software reference for each of the eight output elements.

Index	HW output	SW reference	Absolute error
0	0.495278	0.495278	0
1	-1.194494	-1.194494	0
2	3.700018	3.700018	0
3	-0.291340	-0.291340	0
4	2.090547	2.090547	0
5	-4.499991	-4.499991	0
6	0.992124	0.992124	0
7	-2.799207	-2.799207	0

Table 4.29: C-simulation output for `matmul`. All eight output elements match the software reference within tolerance.

All eight output elements agree between the hardware kernel and the software reference. The expected round-trip relationship holds, with output values being the dequantised equivalents of the original float inputs $[0.5, -1.2, 3.7, -0.3, 2.1, -4.5, 1.0, -2.8]$ to within the precision limits of symmetric INT8 quantisation. The csim run completed with zero errors as reported by Vitis HLS.

4.4.3 HLS Synthesis Results

The `matmul` function was synthesised in isolation using the prototype configuration ($N = D = 8$, $GS = 4$) on the ZCU102 at a 150 MHz target clock (6.667 ns period). Tables 4.30 and 4.31 present the top-level timing and latency summary, followed by Table 4.32 for resource utilisation and Table 4.33 for the per-stage latency breakdown.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	4.867	1.800

Table 4.30: Timing summary for `matmul_top` at the 150 MHz (6.667 ns) target clock.

Latency (cycles)		Latency (absolute)		Interval (cycles)		Pipeline
min	max	min	max	min	max	type
146	146	0.973 µs	0.973 µs	147	147	no

Table 4.31: Top-level latency summary for `matmul_top` at 150 MHz.

Resource	Used (total)	Used (core only)	Available	Utilisation (%)
LUT	5,577	2,506	274,080	2
FF	4,163	1,944	548,160	<1
BRAM	4	0	1,824	<1
DSP	9	9	2,520	<1

Table 4.32: Resource utilisation of `matmul` synthesised at $N = D = 8$ on the ZCU102. Core-only figures exclude the AXI master adapter.

Stage	Trip count	II	Iteration latency	Total (cycles)
Pre-loop buffer loads (`x_buff`, `xs_buff`)	—	—	—	24
`VITIS_LOOP_143_1` (row loop)	8	10	42	120

Table 4.33: Latency breakdown of `matmul` ($N = D = 8$, $GS = 4$) at 150 MHz.

Before any arithmetic can begin on row `i`, the pipeline must first issue AXI reads for the `N = 8` INT8 weight values from `matmul1` and the `N/GS = 2` float scale values from `matmul2`. Because neither loading loop has an `UNROLL` pragma, HLS issues these reads sequentially, with eight weight reads followed by two scale reads, giving $8+2 = 10$ sequential AXI transactions before the first arithmetic operation of the iteration can complete. This sequential load count directly determines the achieved II of 10.

The iteration latency of 42 cycles reflects the time required to complete the full body of one row iteration. The sequential weight and scale loads are followed by the grouped integer dot product across two groups of four elements each. The total

row loop latency is therefore $\text{II} \times (D-1) + \text{iteration latency} = 10 \times 7 + 42 = 112$ cycles for the arithmetic, with the remaining 8 cycles accounting for pipeline drain and sub-module handshaking overhead. The 24-cycle pre-loop overhead covers the AXI address setup and the two buffer-loading loops for the input vector and its scales, which execute once before any row computation begins.

The DSP utilisation of 9 breaks down as follows. The Bind Op report identifies four `mac_muladd_8s_8s_16s_17` units (one per INT8 element of the `matmul4` inner loop within one group, using 1 DSP each), one `fmul_32ns` unit for the floating-point dequantisation product `ival` $\times$ `ws` $\times$ `xs` (3 DSPs), and one `fadd_32ns` unit for the float accumulator `val` (2 DSPs), totalling $4+3+2=9$. An additional four `mul_8s_8s_16` instances appear in the report with zero DSP assignment. These correspond to the second group's INT8 multiplications, which HLS chose to implement in LUT fabric rather than DSP48 blocks. The total DSP count of 9 is higher than the zero DSPs consumed by `quantize` and the 3 DSPs consumed by `dequantize`. The timing situation is identical to all preceding sections, with an estimated 4.867 ns plus 1.800 ns uncertainty that exhausts the 6.667 ns budget, leaving zero slack.

4.4.4 Full TinyLlama Scale: Tiled Implementation

Implementation

The sixth iteration replaces the prototype matmul with a tiled architecture, shown in Listing 4.8. The prototype used `#pragma HLS PIPELINE` on the outer row loop, which was safe at $N=8$ because only $8+2=10$ AXI reads were issued per outer iteration, staying within the AXI adapter's request queue. At full scale $N=2048$, the same approach would issue $2048+32=2{,}080$ simultaneous reads per iteration, saturating the queue and stalling the pipeline. The tiled design resolves this by separating weight loading from computation into explicit phases.

```
tile_loop:
for (int tile = 0; tile < num_full_tiles + (remainder>0?1:0); tile
    ++) {
  #pragma HLS loop_tripcount min=1 max=1408

  const int d_start = tile * TILE_D;
  const int d_count = (tile < num_full_tiles) ? TILE_D : remainder;

  // Phase 1: burst-load TILE_D weight rows from DDR
load_tile:
  for (int d = 0; d < d_count; d++) {
    memcpy(w_tile[d], &wq[(d_start+d)*N], N * sizeof(int8_t));
```

```
12      memcpy(ws_tile[d], &ws[(d_start+d)*(N/GS)], (N/GS) * sizeof(
          float));
13    }
14
15    // Phase 2: compute TILE_D dot products from BRAM
16  compute_tile:
17    for (int d = 0; d < d_count; d++) {
18      float val = 0.0f;
19  dot_groups:
20      for (int g = 0; g < N/GS; g++) {
21        #pragma HLS PIPELINE II=1
22        int32_t ival = 0;
23  dot_inner:
24        for (int k = 0; k < GS; k++) {
25          #pragma HLS UNROLL factor=PARALLEL_MACS
26          ival += (int32_t)x_buf[g*GS+k] * (int32_t)w_tile[d][g*GS+k];
27        }
28        val += (float)ival * ws_tile[d][g] * xs_buf[g];
29      }
30      out_tile[d] = val;
31    }
32
33    // Phase 3: write tile output
34  write_tile:
35    for (int d = 0; d < d_count; d++) {
36      #pragma HLS PIPELINE II=1
37      xout[d_start + d] = out_tile[d];
38    }
39  }
```

Listing 4.8: Core tile loop structure of the tiled `matmul<N,D>` from `FFN_update3.h`.

The template function `matmul<N,D>` processes `TILE_D = 4` output rows per tile. Phase 1 uses `memcpy` to burst-read each of the `TILE_D` weight rows contiguously from DDR into the local BRAM tile `w_tile[TILE_D][N]`. Using `memcpy` is deliberate, as it causes HLS to infer a single burst transaction per row rather than individual scattered element reads, which would each consume one transaction slot in the AXI adapter queue. Phase 2 computes `TILE_D` dot products entirely from the BRAM tile with no DDR access. The innermost `dot_inner` loop carries the `UNROLL factor=PARALLEL_MACS` pragma, giving 16 parallel INT8 multiply-accumulate operations per group of `GS = 64` elements and allowing each group's 64 multiply-accumulate operations to be completed in $64/16 = 4$ pipeline cycles. Phase 3 writes the `TILE_D` float results to the output buffer, pipelined at II = 1.

Two compile-time constants govern the area-performance trade-off. `PARALLEL_MACS = 16` sets the inner MAC parallelism, and `TILE_D = 4` sets the number of rows buffered per tile. The weight tile occupies $4 \times 2048 = 8{,}192$ bytes of INT8 plus $4 \times 32 = 128$ floats, approximately 8.7 KB per matmul call. Load and compute phases run sequentially within each tile. A double-buffering scheme overlapping tile $k+1$'s load with tile k's compute is identified in the source comments as a future optimisation that would reduce the latency by approximately half.

C-Simulation Results

The testbench computed `matmul<2048, 5632>` using fixed-pattern weights and compared the first 8 of 5632 output elements against a software reference. All outputs agreed with zero absolute error, the testbench confirmed 1,408 full tiles and 0 remainder tiles ($5632/4 = 1408$ exactly), and the csim run completed with zero errors as reported by Vitis HLS.

HLS Synthesis Results

Tables 4.34 and 4.35 present the timing and latency summary. Table 4.36 gives the resource utilisation and Table 4.37 the per-stage breakdown.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	4.867	1.800

Table 4.34: Timing summary for `matmul_update3_top` at the 150 MHz target clock.

Latency (cycles)		Latency (absolute)		Interval (cycles)		Pipeline
min	max	min	max	min	max	type
12,752,954	12,752,954	85.024 ms	85.024 ms	12,752,955	12,752,955	no

Table 4.35: Top-level latency summary for `matmul_update3_top` at 150 MHz.

Resource	Used (total)	Used (core only)	Available	Utilisation (%)
LUT	12,438	11,111	274,080	4
FF	5,631	4,402	548,160	1
BRAM	84	80	1,824	4
DSP	35	35	2,520	1

Table 4.36: Resource utilisation of the tiled `matmul` at full scale ($N = 2048$, $D = 5632$) on the ZCU102. Core-only figures exclude the AXI master adapter.

Stage	Module	Latency (cycles)
`matmul_2048_5632_s` core	`matmul_2048_5632_s`	12,752,952
AXI wrapper overhead		2

Table 4.37: Latency breakdown of the tiled `matmul` ($N = 2048$, $D = 5632$, `TILE_D` = 4) at 150 MHz.

Timing meets the 6.667 ns constraint with the same 4.867 ns estimated path as the prototype. The tiled design does not alter the innermost arithmetic, which remains an INT8 multiply-accumulate accumulated into an INT32 partial sum followed by a float dequantisation step, so the critical path is unchanged.

The latency of 12,752,954 cycles (85.024 ms) is the highest of any scalar primitive in this project. With 1,408 tiles ($D/\texttt{TILE_D} = 5632/4$), each tile costs approximately $12{,}752{,}952/1{,}408 \approx 9{,}058$ cycles. Within each tile, Phase 1 loads 4 weight rows from DDR, each a contiguous 2,048-byte burst. At 150 MHz and 32-bit AXI bus width, transferring 2,048 bytes requires $2048/4 = 512$ bus cycles, so loading 4 rows takes $4 \times 512 = 2{,}048$ cycles. Phase 2 then computes 4 dot products over $N/\texttt{GS} = 32$ groups of 64 elements each. With `PARALLEL_MACS = 16`, each group of 64 elements is processed in $64/16 = 4$ cycles, so Phase 2 costs $4 \times 32 \times 4 = 512$ cycles. Phase 3 writes 4 float results in 4 cycles. The tile cost is therefore approximately $2{,}048 + 512 + 4 = 2{,}564$ cycles per 4-row tile, but the measured 9,058 cycles per tile indicates additional overhead from the inter-phase control and the `loop_tripcount`-guided scheduling decisions. Load and compute phases are sequential, so the load cost dominates. A double-buffering scheme that issues tile $k+1$'s DDR read while tile k's Phase 2 is executing would approximately halve the total latency to 44 ms, as identified in the source comments.

The 80 core BRAMs hold the on-chip weight tile buffers. The `w_tile[TILE_D][N]` array is partitioned with `ARRAY_PARTITION complete dim=1` on the row dimension,

making all 4 rows addressable simultaneously, and `cyclic factor=PARALLEL_MACS dim=2` on the column dimension, giving $4 \times 16 = 64$ independent BRAM banks. The `ws_tile[TILE_D][N/GS]` array contributes a further $4 \times 4 = 16$ banks, for 80 core BRAMs in total. The input buffers `x_buf[N]` and `xs_buf[N/GS]` are loaded once before the tile loop and contribute no additional BRAMs beyond what is counted in the prototype matmul.

The 35 DSPs come from the floating-point accumulation in `compute_tile`: the expression `val += (float)ival * ws_tile[d][g] * xs_buf[g]` requires two `fmul` units (6 DSPs) and one `fadd` unit (2 DSPs). However, the synthesis report shows 35 DSPs, not 8, because HLS infers multiple instances of these units to serve the 4-row `compute_tile` loop. With `TILE_D = 4`, HLS may schedule the four rows' accumulators in parallel where ports allow, instantiating additional `fmul`/`fadd` units for each concurrent row, bringing the total to 35. The INT8 multipliers inside `dot_inner` map to LUT fabric rather than DSPs, consistent with the prototype behaviour where `mul_8s_8s` instances consume no DSPs.

4.5 Softmax

4.5.1 Implementation

The `softmax<MAXSIZE>` function converts a vector of raw attention scores into a probability distribution over sequence positions. It is declared as a C++ template in `forward.h`, parameterised by `MAXSIZE`, which sets the size of the local working buffer allocated on-chip. The function accepts a runtime integer `size` that specifies how many of the `MAXSIZE` elements are active for the current call, reflecting the fact that the number of attended positions grows with each decode step from 1 up to the maximum context length. In the baseline kernel, the template is instantiated as `softmax<16>`, matching the prototype's `seq_len = 16`. The second iteration of the kernel widens this to `softmax<257>`, accommodating larger context windows without any code change to the function body, at the cost of a larger static on-chip buffer allocation.

The implementation uses the numerically stable max-subtract form, decomposed into four sequential named loops. Each loop addresses a distinct phase of the computation with its own pragma strategy, since the dependencies between phases prevent them from being merged into a single pipelined stage.

The first loop, `max`, scans the active `size` elements to find the maximum score value. It has a `#pragma HLS PIPELINE` directive and a `loop_tripcount` hint, but no `UNROLL`. The reason is that finding the maximum is a sequential reduction, as each comparison must read the current running maximum and potentially update it before

the next iteration can proceed. This loop-carried dependency on `max_val` prevents parallelism in the same way as the `sum_of_squares` reduction in `rmsnorm`.

The second loop, `exp`, subtracts `max_val` from each element and applies the exponential function, writing results into the local `buffer[MAXSIZE]` array. This loop carries both `#pragma HLS PIPELINE` and `#pragma HLS UNROLL factor=16`. Because each element's computation is entirely independent, the `UNROLL factor=16` on a `MAXSIZE = 16`-element loop amounts to complete unrolling. The exponential function `expf` is an operation with no direct arithmetic circuit equivalent that Vitis HLS maps to the Xilinx `fexp` floating-point IP core, which requires 7 DSP blocks and an 8-cycle pipeline depth. The cost of this operation dominates the resource budget of the entire function, as discussed in the synthesis subsection.

The third loop, `sum`, accumulates the sum of all `buffer` entries. It carries only a `loop_tripcount` hint and no `PIPELINE` or `UNROLL` pragma, because the loop-carried dependency on `sum` means it cannot achieve II = 1 regardless.

The fourth loop, `norm`, divides each `buffer` entry by the total sum to produce the final probability values. Rather than performing a floating-point division per element, the implementation precomputes the reciprocal `inv_sum = 1.0f / sum` as a single scalar before the loop, then multiplies each element by `inv_sum`. This replaces `MAXSIZE` divisions with one division and `MAXSIZE` multiplications, which is a significant saving since `fdiv` `fdiv_32ns_32ns_32_12_no_dsp_1` requires 11 cycles and cannot map to DSP48 blocks, whereas `fmul` `fmul_32ns_32ns_32_4_max_dsp_1` requires 3 cycles and maps efficiently to DSPs. Like the `exp` loop, `norm` carries `PIPELINE` and `UNROLL factor=16`, fully unrolling the 16-element loop into 16 parallel multiply lanes. Listing 4.9 shows the four-loop structure and its pragma assignments.

```
template <int MAXSIZE>
void softmax(float *x, int size) {
  float buffer[MAXSIZE];
  float max_val = x[0];
max:
  for (int i = 1; i < size; i++) {
    #pragma HLS loop_tripcount min=1 max=16 avg=8
    #pragma HLS PIPELINE
    if (x[i] > max_val) max_val = x[i];  // carried dep: no UNROLL
  }
exp:
  for (int i = 0; i < size; i++) {
    #pragma HLS loop_tripcount min=1 max=16 avg=8
    #pragma HLS PIPELINE
    #pragma HLS UNROLL factor=16  // independent: complete unroll
    buffer[i] = expf(x[i] - max_val);
```

```
  }
  float sum = 0.0f;
sum:
  for (int i = 0; i < size; i++) {
    #pragma HLS loop_tripcount min=1 max=16 avg=8
    sum += buffer[i];  // carried dep: no PIPELINE
  }
  const float inv_sum = 1.0f / sum;  // one division hoisted out
norm:
  for (int i = 0; i < size; i++) {
    #pragma HLS loop_tripcount min=1 max=16 avg=8
    #pragma HLS PIPELINE
    #pragma HLS UNROLL factor=16  // independent: complete unroll
    x[i] = buffer[i] * inv_sum;
  }
}
```

Listing 4.9: `softmax<MAXSIZE>` implementation from `forward.h` (baseline iteration).

4.5.2 C-Simulation Results

C-simulation was performed for `softmax_top`. The testbench supplied a 16-element input vector with `size = 8` active elements, exercising the variable-length behaviour of the `loop_tripcount` pragmas. The input vector $[-1.2, 0.5, 1.8, 4.5, -0.3, 2.1, -2.7, 0.9]$ was chosen to include one clearly dominant value (index 3, score 4.5) so that the resulting probability distribution should concentrate most of its mass on that position. The testbench verified two independent correctness criteria, namely element-wise agreement with a software reference implementing the same four-loop algorithm, and a sum-to-unity check on the hardware output independent of the reference. Table 4.38 shows the output from both the kernel and the software reference.

Index	HW output	SW reference	Absolute error
0	0.002752	0.002752	0
1	0.015064	0.015064	0
2	0.055273	0.055273	0
3	0.822446	0.822446	0
4	0.006769	0.006769	0
5	0.074611	0.074611	0
6	0.000614	0.000614	0
7	0.022472	0.022472	0

Table 4.38: C-simulation output for `softmax`. All eight elements match the software reference.

All eight elements match the software reference with zero absolute error, and the independent sum check confirmed that the outputs sum to 1.00000000. The output is strongly concentrated at index 3 (probability 0.822), consistent with the dominant score. The output at index 6 is the smallest (0.000614), corresponding to the most negative input score of -2.7. Both extremes are handled correctly, confirming that the max-subtract numerical stabilisation and the `inv_sum` reciprocal optimisation produce accurate results. The csim run completed with zero errors as reported by Vitis HLS.

4.5.3 HLS Synthesis Results

The `softmax` function was synthesised in isolation using the prototype configuration (`MAXSIZE = seq_len = 16`) on the ZCU102 at a 150 MHz target clock (6.667 ns period). Tables 4.39 and 4.40 present the top-level timing and latency summary, followed by Table 4.41 for resource utilisation and Table 4.42 for the per-loop latency breakdown.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	4.867	1.800

Table 4.39: Timing summary for `softmax_top` at the 150 MHz (6.667 ns) target clock.

Latency (cycles)		Latency (absolute)		Interval (cycles)		Pipeline
min	max	min	max	min	max	type
51	205	0.340 µs	1.367 µs	52	206	no

Table 4.40: Top-level latency summary for `softmax_top` at 150 MHz. The minimum corresponds to `size` = 1 and the maximum to `size` = `MAXSIZE` = 16.

Resource	Used (total)	Used (core only)	Available	Utilisation (%)
LUT	39,921	38,979	274,080	14
FF	19,165	18,432	548,160	3
BRAM	4	0	1,824	<1
DSP	192	192	2,520	7

Table 4.41: Resource utilisation of `softmax` synthesised at `MAXSIZE` = 16 on the ZCU102. The core-only column excludes the AXI master adapter and S-AXILite control interface.

Loop	Trip count	II	Iteration latency	Total (cycles)
`max`	16	1	2	16
`exp`	1	1	9	9
`sum`	16	6	8	97
`norm`	1	1	1	1

Table 4.42: Per-loop latency breakdown of `softmax` (`MAXSIZE` = 16) at 150 MHz.

The resource consumption stands out in these results. With 192 DSPs and 38,979 core LUTs, `softmax` uses more hardware than all four preceding primitives combined. The cause is the complete unrolling of the `exp` and `norm` loops into 16 parallel datapath lanes each. Each of the 16 `expf` computations in the `exp` loop is mapped to a Xilinx `fexp_32ns_32ns_32_9_full_dsp` IP core using 7 DSP48E2 blocks and an 8-cycle pipeline depth at 150 MHz, adding $16 \times 7 = 112$ DSPs. The 16 max-subtract operations in the same loop each require an `fsub` unit using 2 DSPs, adding a further $16 \times 2 = 32$ DSPs. The 16 multiply-by-reciprocal operations in the `norm` loop each map to an `fmul` unit using 3 DSPs, adding $16 \times 3 = 48$ DSPs. The total $112 + 32 + 48 = 192$ DSPs is accounted for by these three sources, with the `max` and `sum` loops using zero DSPs since they use only fabric-based comparators and adders respectively.

The `Bind Storage` section of the synthesis report confirms that the `buffer[16]` array occupies no BRAM. At $16 \times 4 = 64$ bytes, the buffer falls well below the threshold at which HLS infers block RAM, and it is instead mapped to flip-flop registers within the pipeline datapath.

The variable latency reported in Table 4.40 is the first instance of a non-constant latency across all primitives synthesised in this chapter. The minimum of 51 cycles corresponds to `size` $= 1$, where the `max` loop has zero iterations (its lower bound is $i = 1$), the `exp` loop computes a single element, the `sum` loop performs one accumulation, and the `norm` loop normalises one value. The maximum of 205 cycles corresponds to `size` $= 16 =$ `MAXSIZE`, running all four loops at their full trip counts. This variability is a direct consequence of the runtime `size` parameter, which is passed through the S-AXILite control register interface and therefore cannot be statically resolved by HLS during synthesis. The `loop_tripcount` hints on all four loops inform the latency estimator of the expected range but do not constrain the hardware. The actual execution length is always determined at runtime.

The `sum` loop is the dominant contributor to worst-case latency at 97 cycles, nearly five times the combined contribution of the other three loops. The cause is the loop-carried dependency on the floating-point accumulator `sum`, as each iteration must wait for the preceding `fadd` `faddfsub_32ns_32ns_32_6_full_dsp_1` to complete before it can add the next element. The `fadd` unit has a scheduled pipeline depth of 5 cycles at 150 MHz, and the additional cycle required to write the updated accumulator value back and make it available for the next read gives an effective II of 6. Across 16 iterations this gives $6 \times 16 - 6 + 8 = 98 - 1 = 97$ cycles for the loop body alone. This is the same loop-carried dependency pattern observed in the `sum_of_squares` reduction in `rmsnorm`. The `rmsnorm` implementation partially mitigated this with `UNROLL factor=4` to maintain four independent partial sum lanes. Applying the same technique to `sum` would, with four independent lanes and a 5-cycle `fadd` pipeline depth, reduce the II from 6 to approximately $\lceil 5/4 \rceil = 2$, saving roughly 64 cycles per softmax call and is identified as a future optimisation.

The timing result follows the same pattern as all preceding sections. The estimated path of 4.867 ns plus the 1.800 ns uncertainty exhausts the 6.667 ns target, leaving zero slack. The `exp` pipeline sub-module is the most timing-critical path, with only 0.09 ns of slack above the timing constraint prior to uncertainty, due to the deep combinational chain through the 7-DSP `fexp` core.

4.5.4 Iteration 3: Sum Loop Unroll

The third kernel iteration (`forward_fyp_kvint8.cpp`) introduces two pragma modifications to the `softmax` function while leaving the function body logic unchanged. The first addition is `#pragma HLS ARRAY_PARTITION variable=buffer cyclic factor=16` on the internal `buffer[MAXSIZE]` array, which was absent in both earlier iterations. The second addition is `#pragma HLS UNROLL factor=16` on the `sum` reduction loop. These two changes are directly motivated by the II = 6 violation identified in the iteration 1 synthesis analysis, which attributed the 97-cycle `sum` latency to a loop-carried dependency on the floating-point accumulator. The array partition is a prerequisite for the unroll. Without 16 independent memory banks on `buffer`, adding 16 parallel read accesses from an unpartitioned array would cause HLS to serialise the reads rather than parallelise them.

C-Simulation Results

Because the function body is identical to iteration 1, the C-simulation produces bit-identical outputs, with all eight active elements matching the software reference with zero absolute error, and the sum-to-unity check passes with a reported sum of 1.00000000. The csim run completed with zero errors.

HLS Synthesis Results

Tables 4.43 and 4.44 present the timing summary and per-loop latency breakdown for the iteration 3 softmax synthesis. Table 4.45 compares the resource utilisation against iteration 1 for direct comparison.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	5.084	1.800

Table 4.43: Timing summary for `softmax_kvint8_top` at the 150 MHz target clock.

Loop	Trip count	II	Iter. latency	Total (cycles)
`max`	16	1	2	16
`exp`	1	1	9	9
`sum`	1	96	97	185
`norm`	1	1	1	1

Table 4.44: Per-loop latency breakdown of the iteration 3 `softmax` (`MAXSIZE` = 16) at 150 MHz.

	Iteration 1 (baseline)		Iteration 3 (sum unroll)	
Resource	Total	Core	Total	Core
LUT	39,921	38,979	42,723	41,781
FF	19,165	18,432	20,290	19,557
BRAM	4	0	4	0
DSP	192	192	192	192

Table 4.45: Resource utilisation comparison between the iteration 1 baseline and the iteration 3 `softmax` designs, both at `MAXSIZE` = 16 on the ZCU102.

The synthesis results reveal that the `UNROLL factor=16` optimisation produced the opposite of the intended effect. In iteration 1, the `sum` loop had trip count 16, achieved II = 6, and a total latency of 97 cycles. In iteration 3, the complete unroll collapses the loop to a single effective iteration (trip count 1), but that single iteration has a latency of 97 cycles and an II of 96, giving a total of 185 cycles, nearly double the iteration 1 cost. The worst-case total function latency increased from 205 to 295 cycles as a result.

Fully unrolling the 16-element loop creates 16 independent `fadd` units, which HLS must then reduce down to a single scalar through a tree of $\log_2(16) = 4$ adder levels. With each level having a 5-cycle pipeline depth, the minimum reduction latency is approximately $4 \times 5 = 20$ cycles. The resulting network is wide enough that routing introduces additional path delay, causing the 0.217 ns timing violation visible in Table 4.43. This is the first timing violation across all primitives synthesised in this chapter.

The correct approach to reducing the accumulator dependency would have been to use a small number of independent partial accumulators, for example two or four, rather than sixteen. Maintaining four independent partial sum registers and merging them at the end would reduce the effective II from 6 to approximately 2 while keeping the reduction tree shallow enough to close timing. The lesson from this result is that

loop unrolling for reduction operations requires careful sizing, as the number of parallel lanes must be matched to the adder pipeline depth, not simply maximised, and the resulting hardware cost and routing complexity must be validated through synthesis rather than assumed to be beneficial.

4.5.5 Iteration 4: I-BERT Integer Softmax

The timing violation in iteration 3 motivated a different approach to the reduction problem. Rather than tuning the unroll factor on the floating-point accumulator, iteration 4 replaces the floating-point softmax entirely with an integer-arithmetic variant, `softmax_ibert`, which avoids both the reduction dependency and the seven-DSP `fexp` IP core.

Implementation

The function signature of `softmax_ibert<MAX_SIZE>` differs from the legacy `softmax<MAXSIZE>` in two ways. The input is `int32_t *logits` rather than `float *x`, since scores are kept as integers throughout the attention pipeline in iteration 4. The output is a separate `int8_t *probs` buffer rather than an in-place float array, with probabilities encoded in $[0, 127]$ where 127 represents 1.0. A `float scale_factor` parameter carries the remaining scale factors deferred from the `iterate` loop, specifically the product of the per-head Q scale and the attention scaling factor $1/\sqrt{\texttt{head_size}}$.

The exponential function is evaluated through a 128-entry lookup table `exp_lut`, defined as a `static const int8_t` array in the header. Each entry stores $\text{round}(127 \times e^{-\text{index}})$, so the table encodes $127, 47, 17, 6, 2, 1, 0, \ldots$ for indices 0 through 127. The function proceeds in three phases. First, the maximum of the `logits[0..size-1]` values is found using an `UNROLL`ed loop over `MAX_SIZE` elements with an active-element guard. Second, each shifted score is scaled by `scale_factor`, truncated to an integer, clamped to $[-127, 0]$, and mapped through the LUT to produce an `int8` exponential result stored in a fully partitioned local array `exp_results[MAX_SIZE]`. Third, the results are normalised using the formula $p_i = (e_i \times 127)/\text{sum_exp}$, where the multiplication precedes the integer division to preserve precision.

All three loops have `#pragma HLS UNROLL` with no factor specified, giving complete unrolling over `MAX_SIZE = 16` elements. Unlike the reduction in the legacy `sum` loop, the accumulation of `sum_exp` here operates on `int8` values rather than `float` values. An `int8` addition has no multi-cycle pipeline latency in the DSP48E2 path, so the loop-carried dependency resolves within a single clock cycle and $\text{II} = 1$ is achievable. No `ARRAY_PARTITION` is applied to `exp_lut` itself in the current implementation. The commented-out directive in the source reflects a future optimisation opportunity.

C-Simulation Results

The testbench supplied eight `int32` logit values $\{10, -5, 18, 25, -3, 21, -2, 9\}$ with `scale_factor = 0.2f` and `size = 8`. With `max_val = 25`, the shifted and scaled values map to LUT indices as follows. Indices 3 and 5 have shifted scores of 0 and -0.8 respectively, both truncating to index 0, giving `exp_lut[0] = 127`. Index 2 maps to -1.4, truncating to index 1, giving 47. Index 0 and index 7 both map to index 3, giving 6. Indices 4 and 6 map to index 5, giving 1. Index 1 maps to index 6, giving 0. The `sum_exp` is therefore $127+47+6+127+1+1+0+6 = 315$.

Table 4.46 shows the output probabilities from both the kernel and the software reference.

Index	HW prob	SW prob	Abs error
0	2	2	0
1	0	0	0
2	18	18	0
3	51	51	0
4	0	0	0
5	51	51	0
6	0	0	0
7	2	2	0

Table 4.46: C-simulation output for `softmax_ibert`. All eight output probabilities match the software reference.

All eight probabilities match the software reference with zero absolute error. The sum of 124 rather than 127 is expected. Integer division in the formula $p_i = (e_i \times 127)/\text{sum_exp}$ truncates the fractional part of each quotient, so the total probability mass is not conserved to the last integer unit. This is a known property of integer-domain normalisation and not an error in the implementation. The csim run completed with zero errors.

HLS Synthesis Results

Tables 4.47 and 4.48 present the timing and top-level latency summary. Table 4.49 gives the resource utilisation and Table 4.50 the per-phase latency breakdown.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	4.867	1.800

Table 4.47: Timing summary for `softmax_ibert_top` at the 150 MHz target clock.

Latency (cycles)		Latency (absolute)		Interval (cycles)		Pipeline
min	max	min	max	min	max	type
83	531	0.553 µs	3.540 µs	84	532	no

Table 4.48: Top-level latency summary for `softmax_ibert_top` at 150 MHz.

Resource	Used (total)	Used (core only)	Available	Utilisation (%)
LUT	12,706	8,588	274,080	4
FF	8,254	5,171	548,160	<1
BRAM	4	0	1,824	<1
DSP	3	3	2,520	<1

Table 4.49: Resource utilisation of `softmax_ibert` synthesised at `MAX_SIZE` = 16 on the ZCU102.

Module	Latency min (cycles)	Latency max (cycles)
`softmax_ibert_16_s` (core)	37	485
AXI wrapper overhead	46	46
`softmax_ibert_top` (total)	83	531

Table 4.50: Latency breakdown of `softmax_ibert` (`MAX_SIZE` = 16) at 150 MHz.

The defining result of this synthesis is the DSP count of 3, all from the `softmax_ibert` core. Iterations 1 and 3 each used 192 DSPs, entirely dominated by the 16 parallel extttfexp IP core instances. Replacing the floating-point exponential with the 128-entry `exp_lut` ROM eliminates all of those instances. The 3 DSPs that remain correspond to the single `fmul` unit used to compute `val_scaled_f = (float)val_shifted * scale_factor`, the one floating-point multiplication that cannot be avoided since `scale_factor` is a runtime float argument. All other arithmetic in `softmax_ibert` is performed in integer or fixed-point, mapping entirely to LUT fabric.

The LUT count of 8,588 core LUTs is a 78% reduction from iteration 1 (38,979 core LUTs). The integer operations, including the max-search comparators, the LUT indexing arithmetic (which the Expression table shows as shift-left and add operations), the integer accumulation of `sum_exp`, and the integer division in the normalisation step, all map to LUT-based logic rather than DSP-backed floating-point units.

The variable latency of 83 to 531 cycles persists because the runtime `size` parameter still governs how many elements are active. Since all loops are fully unrolled over `MAX_SIZE = 16`, HLS instantiates the full 16-element datapath unconditionally but guards inactive elements with the `if (i < size)` check, which adds multiplexer logic around each element. The minimum of 83 cycles corresponds to `size = 1` and the maximum of 531 cycles to `size = 16`. The timing meets the 6.667 ns constraint, with an estimated path of 4.867 ns plus 1.800 ns uncertainty exhausting the budget with zero slack, consistent with all preceding primitives.

4.5.6 Iteration 6: Full TinyLlama Scale

Implementation

The sixth iteration ports `softmax_ibert` to `MAX_SIZE = seq_len = 2048`, shown in Listing 4.10. Full `UNROLL` on the three loops at this scale would instantiate 2,048 parallel comparators and adder trees simultaneously, exceeding the ZCU102's LUT budget. All three loops (`find_max`, `compute_exp`, `normalize`) are changed to `#pragma HLS PIPELINE II = 1` with a `loop_tripcount` hint, processing one element per cycle. The `exp_results` buffer partition changes from `complete` to `cyclic factor=16`, as at 2,048 entries a complete partition would require 16 KB of flip-flop registers. A reciprocal trick is applied in the normalisation step, computing `inv_sum = 1.0f / (float)sum_exp` once before the `normalize` loop and multiplying per element, removing the per-element integer division.

```
template <int MAX_SIZE>
void softmax_ibert(int32_t *logits, int8_t *probs,
           int size, float scale_factor) {
  if (size > MAX_SIZE) size = MAX_SIZE;

  int32_t max_val = -2147483648;
find_max:
  for (int i = 0; i < MAX_SIZE; i++) {
    #pragma HLS PIPELINE II=1
    #pragma HLS loop_tripcount min=1 max=MAX_SIZE
    if (i < size) {
      if (logits[i] > max_val) max_val = logits[i];
```

```
    }
  }

  int32_t sum_exp = 0;
  int8_t exp_results[MAX_SIZE];
  #pragma HLS ARRAY_PARTITION variable=exp_results cyclic factor=16

compute_exp:
  for (int i = 0; i < MAX_SIZE; i++) {
    #pragma HLS PIPELINE II=1
    #pragma HLS loop_tripcount min=1 max=MAX_SIZE
    if (i < size) {
      int32_t val_shifted = logits[i] - max_val;
      float   val_scaled_f = (float)val_shifted * scale_factor;
      int32_t val_scaled   = (int32_t)val_scaled_f;
      if (val_scaled < -127) val_scaled = -127;
      int8_t e = exp_lut[-val_scaled];
      exp_results[i] = e;
      sum_exp += e;
    }
  }

  if (sum_exp == 0) sum_exp = 1;
  float inv_sum = 1.0f / (float)sum_exp;

normalize:
  for (int i = 0; i < MAX_SIZE; i++) {
    #pragma HLS PIPELINE II=1
    #pragma HLS loop_tripcount min=1 max=MAX_SIZE
    if (i < size) {
      int32_t p = (int32_t)((float)exp_results[i] * 127.0f * inv_sum
          );
      if (p > 127) p = 127;
      if (p < 0)   p = 0;
      probs[i] = (int8_t)p;
    }
  }
}
```

Listing 4.10: softmax_ibert<MAX_SIZE> from FFN_update3.h at full scale (MAX_SIZE = 2048).

C-Simulation Results

The testbench used the same inputs as the iteration 4 csim for direct comparison, using `logits` $= \{10, -5, 18, 25, -3, 21, -2, 9\}$, `scale_factor = 0.2f`, `size = 8`, against `MAX_SIZE = 2048`. All eight output probabilities matched the software reference with zero error, and the sum of output probabilities was 124, identical to iteration 4. The csim run completed with zero errors.

HLS Synthesis Results

Tables 4.51 and 4.52 present the timing and latency summary. Table 4.53 gives the resource utilisation and Table 4.54 the per-stage breakdown.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	4.867	1.800

Table 4.51: Timing summary for `softmax_ibert_update3_top` at the 150 MHz target clock.

Latency (cycles)		Latency (absolute)		Interval (cycles)		Pipeline
min	max	min	max	min	max	type
10,320	10,320	68.803 µs	68.803 µs	10,321	10,321	no

Table 4.52: Top-level latency summary for `softmax_ibert_update3_top` at 150 MHz.

Resource	Used (total)	Used (core only)	Available	Utilisation (%)
LUT	7,137	6,557	274,080	2
FF	2,926	2,414	548,160	<1
BRAM	20	16	1,824	1
DSP	6	6	2,520	<1

Table 4.53: Resource utilisation of `softmax_ibert` at full scale (`MAX_SIZE` = 2048) on the ZCU102. Core-only figures exclude the AXI master adapter (`gmem0_m_axi`).

Stage	Module	Latency (cycles)
Load logits_in	VITIS_LOOP_36_1	2,051
softmax_ibert_2048_s core	softmax_ibert_2048_s	6,200
Write probs_out	VITIS_LOOP_43_2	2,056

Table 4.54: Per-stage latency breakdown of softmax_ibert at full scale (MAX_SIZE = 2048) at 150 MHz. .

Timing meets the 6.667 ns constraint with zero slack.

The total latency of 10,320 cycles has identical minimum and maximum, in contrast to iteration 4 where latency ranged from 83 to 531 cycles depending on the runtime size argument. With PIPELINE and fixed loop bounds, all three loops always execute for the full MAX_SIZE = 2048 iterations, making the latency independent of the input size. This is a deliberate trade-off, accepting a fixed worst-case latency at all sequence positions in exchange for a synthesisable, resource-efficient implementation at full scale.

The softmax_ibert_2048_s core accounts for 6,200 of the 10,320 total cycles. The three sequential passes each iterate over 2,048 elements at II = 1, giving approximately $3 \times 2048 = 6{,}144$ cycles for the core loops, with the remaining 56 cycles from control overhead between passes. The AXI load and write stages each cost approximately 2,050 cycles for the 2,048-element transfers.

The 16 core BRAMs come entirely from the exp_results[2048] buffer. At MAX_SIZE = 16 in iteration 4, the 16-entry array fitted in register fabric. At MAX_SIZE = 2048, the array holds $2048 \times 8 = 16{,}384$ bits, and with cyclic factor = 16 partitioning each bank holds 128 entries of 8-bit data. At this width and depth, HLS infers BRAM rather than registers, giving 16 BRAM banks. The local_logits[2048] input buffer of int32 data maps to 16 BRAM banks under the same analysis but is housed in the AXI wrapper rather than the core module. These 4 BRAMs appear in the total count but not in the core count. The exp_results array holds only 7 effective bits per entry, so HLS maps it to narrow-width BRAMs rather than 32-bit BRAMs, reducing BRAM consumption relative to a float buffer of the same length.

The 6 DSPs come from two fmul units in the normalize loop. The expression (float)exp_results[i] * 127.0f * inv_sum requires one fmul for the multiplication by 127 and one for the multiplication by inv_sum, each consuming 3 DSPs. With PIPELINE II = 1, both multiplications must be available every cycle, so two independent fmul instances are required.

Comparing against iteration 4 (MAX_SIZE = 16, UNROLL factor=16), the LUT count

falls from 8,588 to 6,557 and the FF count falls from 5,171 to 2,414, despite the 128-fold increase in array size. At `MAX_SIZE = 16` with full unrolling, HLS instantiated 16 parallel comparator chains, 16 LUT-addressed exponential lookups, and 16 adder lanes simultaneously. At `MAX_SIZE = 2048` with `PIPELINE`, a single comparator and a single LUT lookup handle all elements sequentially. The BRAM count rises from 0 to 16 since `exp_results` crossed the BRAM inference threshold, and the DSP count rises from 3 to 6 due to the two simultaneous `fmul` units in the pipelined normalisation stage.

Section 4.6 covers the attention mechanism across three kernel iterations. The baseline implementation uses floating-point arithmetic throughout and is synthesised from `forward.cpp`. The second iteration replaces the inner dot products and value accumulation with per-head INT8 arithmetic and is synthesised from `forward_fyp.cpp`. The third iteration extends the INT8 representation into the KV cache itself, eliminating on-the-fly quantisation from the inner attention loops and is synthesised from `forward_fyp_kvint8.cpp`. Each iteration has its own implementation description, and C-simulation and synthesis results for all three iterations are compared together at the end of the section.

4.6 Grouped-Query Attention

4.6.1 Baseline: Floating-Point Attention

Implementation

The baseline attention block is contained in the `multihead_attention` loop in `forward.cpp`. It executes after the QKV matrix–vector multiplications and before the residual addition, operating on the float query vector `q[dim]` and the float KV cache arrays `key_cache` and `value_cache`. The block is not a standalone function in the baseline, as it is embedded directly inside the `forward` kernel's layer loop, and the KV cache arrays are shared on the same `gmem0` AXI bundle as the transformer weight struct.

The outer `multihead_attention` loop iterates over the `n_heads = 2` query heads sequentially at prototype scale. For each head, the computation proceeds in three phases. The first phase, the `iterate` loop, computes the dot product between the current query slice and every cached key vector from timestep 0 through `pos`. The outer position loop has `#pragma HLS PIPELINE`, and the inner element loop over `head_size = 4` dimensions has `#pragma HLS UNROLL` with no factor specified, giving complete unrolling. The score is then divided by $\sqrt{\texttt{head_size}}$ and written to the local `att_buf` score buffer. After the `iterate` loop, a call to `softmax<seq_len>` converts the scores

for the current head into a probability distribution in-place. The second phase, the acc loop, accumulates the probability-weighted value vectors into xb_buf. Like the iterate loop, it pipelines the outer position loop and completely unrolls the inner element loop.

The local buffers q_buf, att_buf, and xb_buf are each partitioned with ARRAY_PARTITION cyclic factor=16 to provide sufficient memory ports for the simultaneously executing unrolled inner loop iterations. The key cache and value cache are read directly from the gmem0 AXI port inside the pipelined outer loops without intermediate buffering, which is the source of the II violation described in the synthesis analysis below.

An important consideration concerns the PIPELINE pragma placement on the iterate and acc outer loops. At prototype scale (seq_len = 16, head_size = 4), pipelining the outer position loop is safe because the inner unrolled element loop generates four concurrent AXI read requests per outer iteration, which the adapter's transaction queue can accommodate without stalling indefinitely. At full TinyLlama scale (seq_len = 2048, head_size = 64), applying PIPELINE II=1 to the outer loop would instruct HLS to start a new position every clock cycle while the inner loop generates 64 simultaneous AXI read requests. The adapter's queue would fill within the first iteration, and each subsequent position would stall waiting for outstanding transactions to clear, creating a circular wait that locks the kernel. This AXI interface stall is one of the three issues encountered during on-board testing and is discussed further in section 4.8.3. Listing 4.11 shows the iterate and acc loop bodies extracted from forward.cpp.

```
iterate:
  for (int t = 0; t <= pos; t++) {
    #pragma HLS PIPELINE
    #pragma HLS loop_tripcount min=1 max=16 avg=8
    float score = 0.0f;
    for (int i = 0; i < head_size; i++) {
      #pragma HLS UNROLL  // complete unroll: 4 parallel fmul units
      score += q_buf[i + q_offset] * key_cache[i + key_offset];
    }
    score /= sqrtf(head_size);
    att_buf[t + att_offset] = score;
  }
  softmax<seq_len>(att_buf + att_offset, pos + 1);

acc:
  for (int t = 0; t <= pos; t++) {
    #pragma HLS loop_tripcount min=1 max=16 avg=8
    #pragma HLS PIPELINE
    float a = att_buf[t + att_offset];
```

```
    for (int i = 0; i < head_size; i++) {
      #pragma HLS UNROLL  // complete unroll: 4 parallel fmul+fadd
      xb_buf[i + xb_offset] += a * value_cache[i + v_offset]; //
          float DDR read
    }
  }
```

Listing 4.11: Baseline float attention `iterate` and `acc` loops from `forward.cpp`.

C-Simulation Results

C-simulation was performed for `attn_baseline_top`. The testbench exercised the kernel at `pos = 3` and `loff = 0`, covering four attended positions across both query heads. The query vector was set to linearly spaced values $[0.1, 0.2, \ldots, 0.8]$, and the key and value cache entries were initialised with small deterministic float values to produce non-trivial softmax outputs without requiring random number generation. The testbench verified only the final weighted value accumulation output `xb`, since the attention score buffer `att_buf` is an internal on-chip buffer in the real kernel and is not exposed as an AXI port. Table 4.55 shows the output from both the kernel and the software reference.

Index	HW xb	SW reference	Absolute error
0	0.343754	0.343754	0
1	0.313754	0.313754	0
2	0.283754	0.283754	0
3	0.253754	0.253754	0
4	0.213827	0.213827	0
5	0.183827	0.183827	0
6	0.153827	0.153827	0
7	0.123827	0.123827	0

Table 4.55: C-simulation output for the baseline float attention kernel (`attn_baseline_top`) at `pos = 3`, `loff = 0`. All eight elements of the weighted value accumulation `xb` match the software reference with zero absolute error.

All eight output elements agree with the software reference, confirming that the three phases of the attention computation, score dot products, softmax normalisation, and value accumulation, all produce correct results when composed together. The zero-error outcome is expected for this test configuration, as all arithmetic is performed in float32 using the same operation sequence in both the kernel and the reference, so no

rounding divergence can arise. The csim run completed with zero errors as reported by Vitis HLS.

HLS Synthesis Results

The baseline attention block was synthesised in isolation on the ZCU102 at a 150 MHz target clock. Tables 4.56 and 4.57 present the top-level timing and latency summary, followed by Table 4.58 for resource utilisation and Table 4.59 for the per-pipeline-module breakdown.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	4.867	1.800

Table 4.56: Timing summary for `attn_baseline_top` at the 150 MHz (6.667 ns) target clock.

Latency (cycles)		Latency (absolute)		Interval (cycles)		Pipeline
min	max	min	max	min	max	type
48	800	0.320 µs	5.334 µs	49	801	no

Table 4.57: Top-level latency summary for `attn_baseline_top` at 150 MHz. The minimum corresponds to `pos = 0` (one attended position) and the maximum to `pos = seq_len − 1 = 15` (sixteen attended positions).

Resource	Used (total)	Used (core only)	Available	Utilisation (%)
LUT	54,912	53,650	274,080	20
FF	27,584	26,673	548,160	5
BRAM	4	0	1,824	<1
DSP	192	192	2,520	7

Table 4.58: Resource utilisation of the baseline float attention block synthesised at prototype scale on the ZCU102. Core-only figures exclude the AXI master adapter.

Pipeline stage	Trip count	II	Iter. latency	Total (cycles)
`iterate` (score dot product)	16	4	42	101
`max` (softmax pass 1)	16	1	2	16
`exp` (softmax pass 2)	1	1	17	16
`sum` (softmax pass 3)	16	6	8	97
`norm` (softmax pass 4)	1	1	1	1
`acc` (value accumulation)	16	6	23	112

Table 4.59: Per-pipeline-module latency breakdown of the baseline float attention block at 150 MHz, for one head at maximum context length (`pos = 15`). The outer `multihead_attention` loop over `n_heads = 2` runs sequentially, so the per-head cost is incurred twice.

The resource figures reflect the composition of the attention block with the softmax function it instantiates internally. The 192 DSPs and 53,650 core LUTs match the softmax standalone synthesis results, confirming that the floating-point exponential units from the 16 fully unrolled `fexp` instances dominate the resource budget of the entire attention block. The compute logic for the score dot products and value accumulation uses relatively few resources. The `iterate` pipeline module uses only 2,005 LUTs and zero DSPs, and the `acc` module uses only 1,136 LUTs and zero DSPs.

The II violations in the `iterate` and `acc` loops. The `iterate` loop achieves II = 4 against a target of 1. With `#pragma HLS PIPELINE` on the outer position loop and `#pragma HLS UNROLL` (complete) on the inner `head_size = 4` element loop, HLS attempts to start a new position every clock cycle. Doing so requires loading four key cache elements simultaneously through the shared `gmem0` AXI port at each new iteration. The AXI adapter can accept at most as many outstanding read transactions as it has queue slots, and with four simultaneous requests the pipeline stalls until the preceding reads complete, giving an effective II of 4. The `iterate` module therefore takes $4 \times (16-1) + 42 = 101$ cycles to process all 16 positions rather than the ideal $16 + 42 = 58$ cycles at II = 1.

The `acc` loop achieves II = 6. The inner element loop is again completely unrolled, but the accumulation $\texttt{xb_buf}[i] \mathrel{+}= a \times \texttt{value_cache}[i + \texttt{v_offset}]$ introduces a float multiply-add chain through the accumulator. The `fmul`–`fadd` dependency chain runs through the 5-cycle `fadd` unit, and the additional write-back cycle before the updated accumulator is available gives an effective II of 6, preventing the outer loop from starting the next position until the previous accumulation has propagated through both the multiplier and the adder. This II = 6 is identical to the accumulator dependency that was identified in the `sum` loop of `softmax`. These II violations at prototype scale, while

not catastrophic given the small `seq_len = 16`, scale linearly at full TinyLlama dimensions (`seq_len = 2048`) and motivate the per-head INT8 redesign described in section 4.6.2.

4.6.2 Refined: Per-Head INT8 Attention

The second iteration of the attention mechanism, implemented in `forward_fyp.cpp`, replaces the floating-point score computation and value accumulation with per-head INT8 arithmetic. The motivation for this redesign is a quantisation precision problem that arises when the global `quantize()` helper is applied to the full query, key, and value vectors. The helper computes a single scale factor from the absolute maximum of the entire `dim`-element vector. If one head's slice contains activation outliers that dominate the global maximum, the resulting scale factor maps the outlier range to the INT8 boundary of 127 and compresses all other heads' values into a narrow fraction of the INT8 range. This cross-head scale contamination degrades the precision of every head that does not contain the outlier. The fix is to compute a separate scale for each head's slice independently, so that the scale reflects only the `head_size` elements belonging to that head.

The revised implementation introduces three structured steps per head, executed in sequence within the `multihead_attention` outer loop.

Step 1 quantises the query slice for the current head. Before the position loop begins, the implementation scans the `head_size = 4` elements of `q_buf` belonging to head `h` to find the maximum absolute value, computes a per-head scale $s_Q = \texttt{max_val}/127$, and quantises the slice into a local `int8_t q_int8[head_size]` array. The reciprocal trick from the quantise primitive is applied, whereby a single `fdiv` computes $1/s_Q$ before the element loop, and each element is then quantised by multiplying by this reciprocal rather than dividing, replacing `head_size` divisions with one division and `head_size` multiplications.

Step 2 executes inside the `iterate` position loop. For each attended timestep `t`, the `head_size` key values are loaded from the float KV cache into a local `k_buffer`, a per-position scale s_K is computed from the buffer's maximum absolute value, and the key slice is quantised to `int8_t k_int8`. The dot product between `q_int8` and `k_int8` is then accumulated into a 32-bit integer `score_int`, with `#pragma HLS UNROLL` applied to the inner `head_size` element loop. The final dequantisation combines both per-head scales and the attention scaling factor in a single expression, shown below.

$$\text{score} = \left(\sum_{i=0}^{d_k-1} \hat{q}_i \cdot \hat{k}_i \right) \cdot s_Q \cdot s_K \cdot \frac{1}{\sqrt{d_k}} \tag{4.1}$$

where $\hat{q}_i$ and $\hat{k}_i$ are the INT8 quantised query and key values, and s_Q, s_K are the respective per-head scales. The result is written to `att_buf` as a float.

The softmax call is widened from `softmax<16>` to `softmax<257>` in this iteration. This accommodates context lengths up to 256 positions without any code change to the function body. The cost is that the internal `buffer[257]` array within `softmax<257>` is too large for register inference at this scale and is instead inferred as BRAM, as observed in the synthesis results.

Step 3 executes inside the `acc` position loop. The attention probability for timestep `t` is first truncated to a signed INT8 value by the mapping $\texttt{prob_int8} = \lfloor \texttt{prob_float} \times 127 \rfloor$, which assigns the unit probability 1.0 to the value 127 and uses a fixed implicit scale of $1/127$. The value vector is then loaded from the float KV cache, quantised on the fly with its own per-position scale s_V, and the weighted contribution is accumulated according to the expression below.

$$\Delta\texttt{xb}(i) = (\texttt{prob_int8} \times \hat{v}_i) \cdot \frac{1}{127} \cdot s_V \tag{4.2}$$

where $\hat{v}_i$ is the INT8 quantised value element. The accumulation target `xb_accum[head_size]` is declared with `#pragma HLS ARRAY_PARTITION variable=xb_accum complete`, which maps it to four independent registers rather than a BRAM array. This is important because the accumulation loop updates all four elements of `xb_accum` in each pipeline iteration. A BRAM-based array would introduce read-after-write latency on the same address, whereas a register file supports simultaneous access to all four ports. Listing 4.12 shows the Step 1 per-head Q quantisation and the Step 2 INT32 dot product, which together constitute the core change from the baseline.

```
// STEP 1: per-head Q quantisation (before iterate loop)
float max_val = 0.0f;
for (int i = 0; i < head_size; i++) {
  #pragma HLS PIPELINE
  float v = std::fabs(q_buf[i + q_offset]);
  if (v > max_val) max_val = v;
}
float q_scale_val = (max_val == 0.0f) ? 1.0f : (max_val / 127.0f);
float q_inv_scale = 1.0f / q_scale_val;  // one division hoisted out
int8_t q_int8[head_size];
for (int i = 0; i < head_size; i++) {
  #pragma HLS UNROLL
  q_int8[i] = (int8_t)round(q_buf[i + q_offset] * q_inv_scale);
}

// STEP 2: on-the-fly K quantisation + INT32 dot product (inside
```

```
    iterate)
iterate:
for (int t = 0; t <= pos; t++) {
  #pragma HLS PIPELINE
  // ... loaded float k_buffer[head_size] from key_cache, computed
      k_scale ...
  int32_t score_int = 0;
  for (int i = 0; i < head_size; i++) {
    #pragma HLS UNROLL
    score_int += (int32_t)q_int8[i] * (int32_t)k_int8[i]; // integer
        MAC
  }
  float score = ((float)score_int) * q_scale_val * k_scale_val
          * (1.0f / sqrtf(head_size));  // single dequantise step
  att_buf[t + att_offset] = score;
}
```

Listing 4.12: Per-head Q quantisation (Step 1) and INT32 score dot product (Step 2) from `forward_fyp.cpp`. Step 1 executes once per head before the position loop. Step 2 executes for each attended position inside `iterate`. The reciprocal trick replaces `head_size` divisions with one `fdiv` and `head_size` `fmul` calls.

C-simulation was performed for both implementations using the same test configuration of `pos` = `3` and `loff` = `0`, with identical query, key cache, and value cache values. This allows the outputs to be compared directly as a measure of the quantisation error introduced by the INT8 redesign. Table 4.60 shows the `xb` output from both iterations alongside the absolute difference between them.

Index	Baseline xb (float)	INT8 xb	Abs difference	Relative error (%)
0	0.343754	0.337795	0.005959	1.73
1	0.313754	0.307415	0.006339	2.02
2	0.283754	0.279273	0.004481	1.58
3	0.253754	0.248893	0.004861	1.92
4	0.213827	0.209449	0.004378	2.05
5	0.183827	0.179730	0.004097	2.23
6	0.153827	0.150702	0.003125	2.03
7	0.123827	0.120983	0.002844	2.30

Table 4.60: Comparison of `xb` outputs from the baseline float attention and the refined per-head INT8 attention for identical inputs at `pos = 3`. The absolute difference and relative error in each element reflect the quantisation error introduced by the INT8 arithmetic path. Both kernels passed their respective csim runs with zero errors against their own software references.

Both implementations passed their csim runs with zero errors against their respective software references. The absolute differences between the two kernels' outputs range from approximately 0.003 to 0.006, representing relative errors of 1.58% to 2.30% with respect to the float baseline. This level of quantisation error is consistent with what is expected from symmetric INT8 quantisation at `head_size = 4`, since each individual scale is computed from only four values, so a single outlier or uneven distribution can shift the scale enough to round adjacent values by up to half an INT8 step. At full TinyLlama scale (`head_size = 64`), the per-head scale is computed from a much larger pool of values, and the expected quantisation error per element is correspondingly smaller.

HLS Synthesis Results

Table 4.61 summarises the resource utilisation of both attention implementations, and Table 4.62 provides the per-pipeline-module latency breakdown for the refined INT8 design. The baseline latency breakdown was presented in Table 4.59.

	Baseline (float)		Refined (INT8)	
Resource	Total	Core only	Total	Core only
LUT	54,912	53,650	46,448	45,186
FF	27,584	26,673	27,873	26,962
BRAM	4	0	36	32
DSP	192	192	122	122

Table 4.61: Resource utilisation comparison between the baseline float attention and the refined per-head INT8 attention at prototype scale on the ZCU102.

Pipeline stage	Trip count	II	Iter. latency	Total (cycles)
Q max scan	4	1	2	4
Q quantise	4	1	1	4
`iterate` (K quant + INT32)	257	4	73	1,096
`max` (softmax pass 1)	257	1	2	257
`exp` (softmax pass 2)	17	2	18	51
`sum` (softmax pass 3)	257	6	9	1,544
`norm` (softmax pass 4)	17	1	6	22
`xb_accum` initialise	4	1	1	4
`acc` (V quant + weighted)	257	6	76	1,611

Table 4.62: Per-pipeline-module latency breakdown of the refined per-head INT8 attention block at 150 MHz, for one head at maximum context (`pos = 256`, `MAXSIZE = 257`). The outer `multihead_attention` loop over `n_heads = 2` runs sequentially, giving a worst-case total of approximately 9,332 cycles.

The largest change in the resource comparison is the DSP reduction from 192 to 122, a saving of 70 DSP48E2 blocks. In the baseline, the 192 DSPs are almost entirely accounted for by the 16 parallel `fexp` units inside `softmax<16>` (112 DSPs) plus the `fsub` max-subtract units (32 DSPs) and `fmul` normalisation units (48 DSPs). In the refined design, the `softmax<257>` instance with `UNROLL factor=16` on its 257-element buffer generates 16 parallel `fexp` units for the `exp` loop's 17 effective iterations, but HLS maps only 70 DSPs to the `exp` pipeline sub-module at this larger template size, compared to 112 at `MAXSIZE=16`. The `norm` sub-module uses 36 DSPs, and the `iterate` module 2 DSPs for the dequantisation multiplications. The INT32 dot product itself, being an integer accumulation rather than a floating-point multiply-add chain, is implemented in LUT fabric without using DSP blocks.

The BRAM increase from 0 to 32 core BRAMs reflects two new memory consumers in the refined design. The first is the `softmax<257>` internal `buffer[257]` array. At $257 \times 4 = 1{,}028$ bytes this exceeds the register inference threshold and is partitioned by HLS into 8 BRAM banks of 2 BRAM_18K each, using 16 BRAM_18K blocks in total. The second is the `generic_round_double_s` IP core instantiated four times for the per-element Q quantisation rounding, using 4 BRAM_18K blocks per instance (16 total). This rounding IP uses BRAM-backed lookup tables to implement IEEE 754 round-to-nearest in double precision, analogous to the `mask_table` and `one_half_table` ROM inferences observed in the `quantize` standalone synthesis.

The `iterate` loop maintains II = 4 in the refined design, unchanged from the baseline. The root cause is identical, with the on-the-fly K loading inside the pipelined position loop generating four concurrent AXI read requests through the shared `gmem0` port, and the adapter's transaction queue limiting the sustainable request rate to one group of four reads per 4 clock cycles. The on-the-fly K quantisation steps that are unique to the refined design add to the iteration latency (73 cycles versus 42 in the baseline), but do not change the II since those operations execute after the four reads have been issued. The `acc` loop similarly retains II = 6 from the float multiply-accumulate dependency introduced by the `prob_int8` $\times$ `v_int8` $\times$ `scale` dequantisation chain, which presents the same `fmul`–`fadd` latency bottleneck as the float baseline's direct accumulation. The structural benefit of the INT8 redesign is therefore not a reduction in II at prototype scale, but rather the elimination of cross-head scale contamination and the 37% reduction in DSP consumption, which becomes the enabling factor for fitting the full-scale attention design within the ZCU102's resource budget.

4.6.3 Iteration 3: INT8 KV Cache

The third iteration, implemented in `forward_fyp_kvint8.cpp`, extends the INT8 representation into the KV cache itself. In the first two iterations, the key and value vectors are stored in DDR as float32 arrays and quantised on the fly when they are read back during the `iterate` and `acc` loops. The third iteration instead quantises K and V immediately after the QKV projections, before writing them into the cache, so that subsequent reads during attention retrieve pre-quantised INT8 data directly. This eliminates the on-the-fly quantisation sequences from the inner attention loops, at the cost of making the KV cache write slightly more expensive.

The function signature gains two new AXI arguments. `key_cache` and `value_cache` change type from `float*` to `int8_t*`, and two new `float*` arguments `key_scales` and `value_scales` carry the group-wise scale factors stored alongside the INT8 data. At prototype scale with `kv_dim` $= 8$ and `GS` $= 4$, each timestep's KV head slice requires

two scale factors, and the scale arrays are indexed with a separate `loff_scale` offset computed as $l \times \texttt{seq_len} \times (\texttt{kv_dim}/\texttt{GS})$.

Inside the `iterate` loop, the on-the-fly K loading from iteration 2, which required loading float K, scanning for the maximum, computing s_K, and quantising element by element, is replaced by two direct reads, namely a burst read of `head_size` INT8 values from `key_cache` and a single float read of the pre-stored scale from `key_scales`. The INT32 dot product and dequantisation formula are unchanged from Equation 4.1. Inside the `acc` loop, the equivalent on-the-fly V quantisation is similarly replaced by direct INT8 and scale reads from `value_cache` and `value_scales`. The value accumulation formula also simplifies. Iteration 2 mapped the softmax probability to INT8 before accumulating, while iteration 3 keeps it as a float and folds it directly with the V scale into a `combined_scale = prob` $\times$ `v_scale` scalar. The `acc` loop additionally gains `#pragma HLS UNROLL factor=2` on the outer position loop, processing two timesteps per pipeline iteration.

The `softmax` call reverts to `softmax<16>` from the `softmax<257>` used in iteration 2, since the iteration 3 loop trip count bounds return to `min=1 max=16`. The softmax function in this header also includes the `UNROLL factor=16` on the `sum` loop and the buffer partition discussed in section 4.5.4, so the attention block inherits both the performance regression and the timing violation of that unrolled softmax. Listing 4.13 contrasts the `iterate` loop bodies of iterations 2 and 3 side by side, making the elimination of on-the-fly K quantisation concrete.

```
// ITERATION 2: on-the-fly K quantisation inside iterate
for (int t = 0; t <= pos; t++) {
  #pragma HLS PIPELINE
  float k_buffer[head_size];
  for (int i = 0; i < head_size; i++)
    k_buffer[i] = key_cache[i + key_offset]; // float DDR load
  float k_max = 0.0f;
  for (int i = 0; i < head_size; i++) {
    float v = std::fabs(k_buffer[i]);  // max scan
    if (v > k_max) k_max = v;
  }
  float k_scale = k_max / 127.0f;  // scale compute
  float k_inv   = 1.0f / k_scale;
  int8_t k_int8[head_size];
  for (int i = 0; i < head_size; i++)
    k_int8[i] = (int8_t)round(k_buffer[i] * k_inv); // quantise
  // ... INT32 dot product using k_int8 ...
}

```

```
// ITERATION 3: direct INT8 load from pre-quantised cache
for (int t = 0; t <= pos; t++) {
  #pragma HLS PIPELINE
  float k_scale = key_scales[scale_offset];  // one float scalar
      read
  int8_t k_buffer[head_size];
  for (int i = 0; i < head_size; i++)
    k_buffer[i] = key_cache[data_offset + i]; // INT8 load
  // ... INT32 dot product using k_buffer directly ...
}
```

Listing 4.13: Contrast between the `iterate` loop bodies of iteration 2 (`forward_fyp.cpp`) and iteration 3 (`forward_fyp_kvint8.cpp`). Iteration 2 loads float K and performs a full max-scan, scale-compute, and quantise sequence per position. Iteration 3 replaces the entire sequence with two direct reads from the pre-quantised cache arrays.

Table 4.63 extends the earlier two-iteration csim comparison to include iteration 3. All three csim runs passed with zero errors against their respective software references.

Index	Baseline xb (float)	Iter. 2 xb (INT8)	Iter. 3 xb (INT8-KV)
0	0.343754	0.337795	0.343771
1	0.313754	0.307415	0.312900
2	0.283754	0.279273	0.284313
3	0.253754	0.248893	0.253442
4	0.213827	0.209449	0.213733
5	0.183827	0.179730	0.183540
6	0.153827	0.150702	0.154046
7	0.123827	0.120983	0.123853

Table 4.63: C-simulation xb outputs for all three attention iterations at identical inputs (`pos` = 3, `loff` = 0). Iteration 3 outputs are closer to the float baseline than iteration 2, reflecting the higher fidelity of pre-quantised group-scale storage compared to per-position on-the-fly quantisation.

The iteration 3 outputs are noticeably closer to the float baseline than the iteration 2 outputs. The maximum absolute difference from the float baseline reduces from approximately 0.006 (iteration 2) to approximately 0.001 (iteration 3). This improvement occurs because iteration 3 stores K and V with group-wise scales computed over `GS` = 4 elements of the full `kv_dim` = 8-element vector, whereas iteration 2 computed one scale per head slice (`head_size` = 4 elements) on the fly. Although both use the same number

of elements per group, the pre-stored group scales in iteration 3 are computed once at the write site using the full integer range of 127, while iteration 2's on-the-fly scales were subject to additional rounding at each read. The net effect is a more accurate reconstruction of the float activations at the point where they enter the attention score computation.

HLS Synthesis Results

all three iterations, and Table 4.65 presents the per-pipeline-module latency for iteration 3.

	Baseline	Iter. 2	Iter. 3	
Resource	(core)	(core)	Total	Core
LUT	53,650	45,186	72,096	70,514
FF	26,673	26,962	35,036	33,947
BRAM	0	32	22	18
DSP	192	122	194	194

Table 4.64: Resource utilisation across all three attention iterations on the ZCU102 at 150 MHz.

Pipeline stage	Trip count	II	Iter. latency	Total (cycles)
Q max scan	4	1	2	4
Q quantise	4	1	1	4
`iterate` (INT8 K load + INT32)	16	5	26	100
`max` (softmax pass 1)	16	1	2	16
`exp` (softmax pass 2)	1	1	17	16
`sum` (softmax pass 3, unrolled)	1	96	97	185
`norm` (softmax pass 4)	1	1	1	1
`xb_accum` initialise	4	1	1	4
`acc` (INT8 V load + weighted)	8	12	36	124

Table 4.65: Per-pipeline-module latency breakdown of the iteration 3 INT8-KV attention block at 150 MHz, for one head at `pos = 15`. The `sum` stage reflects the unrolled softmax from section 4.5.4.

In iteration 3, the `iterate` iteration latency drops from 42 cycles (baseline) and 73 cycles (iteration 2) to 26 cycles. This directly reflects the elimination of the on-the-fly

K quantisation sequence. Where iteration 2's `iterate` body spent the majority of its 73 cycles on the max-scan, scale computation, and element-wise quantisation of the loaded float K vector, iteration 3 instead issues two AXI reads, a burst of `head_size` INT8 bytes from `key_cache` and a single float from `key_scales`, and then proceeds directly to the integer dot product. The achieved II is 5 rather than the 4 observed in the baseline and iteration 2, because the two separate AXI read streams (`key_cache` and `key_scales`) now issue through the same `gmem0` bundle and require one additional cycle of arbitration compared to the single float stream of the baseline.

The `acc` loop shows trip count 8 with II = 12 and total 124 cycles. The `UNROLL factor=2` on the outer position loop halves the trip count from 16 to 8, but also doubles the work per pipeline iteration. The II = 12 reflects the two simultaneous V vector loads per iteration combined with the float `fmul`–`fadd` accumulation chain, which has a combined pipeline depth of approximately 10 cycles. Total `acc` latency at 124 cycles is slightly more than the baseline's 112 cycles, as the on-the-fly V scale load adds one extra memory access per iteration.

LUT usage increases from iteration 2 (45,186 core) to iteration 3 (70,514 core). The primary driver is not the attention block itself but the `sum` pipeline module inherited from the iteration 3 softmax function, which grew from 233 LUTs in the baseline to 2,416 LUTs after the `UNROLL factor=16` change. The BRAM count reduced from 32 core (iteration 2) to 18 core (iteration 3), because `softmax<16>` is reinstated and its 64-byte buffer stays in registers rather than BRAM, while the `generic_round_double_s` rounding IP instances remain from the per-head Q quantisation, each still using 4 BRAM_18K blocks. Both iteration 3 designs (softmax and attention) carry the -0.217 ns timing violation introduced by the `sum` unroll, with an estimated critical path of 5.084 ns against the 4.867 ns budget.

4.6.4 Iteration 4: I-BERT Integer Softmax Attention

Iteration 4 resolves the timing violation from iteration 3 by replacing the floating-point `softmax<16>` with the integer `softmax_ibert<16>` described in section 4.5.5. This change also requires two structural modifications to the `iterate` and `acc` loops to keep the attention score pipeline consistent with integer types throughout.

Implementation

The `att` buffer type changes from `float[]` to `int32_t[]`. In `iterate`, the full dequantisation step of iteration 3, which multiplied `score_int` by all three scale factors before storing, is split into two parts. Only the K-scale partial result `(int32_t)((float)score_int * k_scale)` is stored in `att`, deferring the Q-scale and attention-scale factors to the

softmax call. After the position loop, the deferred factors are combined as `remaining_scale = q_scale_val * (1.0f / sqrtf(head_size))` and passed to `softmax_ibert<16>`, which returns `int8_t att_probs[16]` with probabilities in $[0, 127]$.

In the acc loop, `prob_int8 = att_probs[t]` replaces the float probability from iteration 3. The combined scale becomes `(1.0f/127.0f) * v_scale`, where the factor $1/127$ dequantises the integer probability back to the $[0, 1]$ range before the weighted value accumulation. The outer position loop retains `#pragma HLS PIPELINE` and `#pragma HLS UNROLL factor=2`, and the inner element loop retains `#pragma HLS UNROLL factor=16`, unchanged from iteration 3.

C-Simulation Results

Table 4.63 from iteration 3 is extended in Table 4.66 to include the iteration 4 outputs for the same input at `pos = 3`.

Index	Baseline xb	Iter. 2 xb	Iter. 3 xb	Iter. 4 xb
0	0.343754	0.337795	0.343771	0.341732
1	0.313754	0.307415	0.312900	0.311557
2	0.283754	0.279273	0.284313	0.283688
3	0.253754	0.248893	0.253442	0.253512
4	0.213827	0.209449	0.213733	0.224567
5	0.183827	0.179730	0.183540	0.195083
6	0.153827	0.150702	0.154046	0.166292
7	0.123827	0.120983	0.123853	0.136808

Table 4.66: C-simulation xb outputs for all four attention iterations at `pos = 3`, `loff = 0`. Iteration 4 introduces additional approximation error relative to iteration 3 from the coarse LUT exponential, but all four csim runs passed with zero errors against their respective software references.

The csim run completed with zero errors. Comparing the iteration 4 outputs to the three preceding iterations, the values lie between iteration 2 and the float baseline rather than close to iteration 3. For example, index 0 gives 0.341732 in iteration 4 versus 0.343754 (baseline), 0.337795 (iteration 2), and 0.343771 (iteration 3). The LUT exponential maps score differences to coarse integer indices, concentrating the probability mass on the leading positions differently from the true softmax, which is why the output distribution differs visibly from iteration 3 even though the underlying attention scores originate from the same inputs. The absolute difference between iteration 4 and the float baseline is larger for the lower-ranked elements (indices 4–7 diverge more

noticeably), because those positions have smaller exponent arguments that the LUT resolution can represent less accurately. At full TinyLlama scale (`head_size = 64`), the per-head scores span a wider integer range, giving the `scale_factor` more dynamic range to distribute across LUT indices.

HLS Synthesis Results

Tables 4.67 and 4.68 extend the resource and latency comparisons to include iteration 4.

	Iteration 3		Iteration 4	
Resource	Total	Core	Total	Core
LUT	72,096	70,514	33,410	31,828
FF	35,036	33,947	16,017	14,928
BRAM	22	18	22	18
DSP	194	194	16	16

Table 4.67: Resource comparison between iteration 3 and iteration 4 attention implementations on the ZCU102.

Module	Latency min (cycles)	Latency max (cycles)
Q quantisation (VITIS_LOOP_73_2)	10	10
`iterate`	28	103
`softmax_ibert`	37	485
`acc`	43	127
`xb` write (VITIS_LOOP_126_6)	6	6
`multihead_attention` (total, 2 heads)	178	1,536
`attn_softmaxint8_top` (total)	208	1,566

Table 4.68: Per-module latency breakdown for the iteration 4 attention kernel at 150 MHz. The minimum corresponds to `pos = 0` (one attended position) and the maximum to `pos = 15` (`MAX_SIZE = 16`).

The headline result of iteration 4 is the DSP reduction from 194 to 16. In iteration 3, 192 of the 194 DSPs came from the 16 parallel extttfexp IP core instances inside the unrolled `softmax<16>` function. Replacing those with `softmax_ibert` eliminates all of them. The 16 DSPs that remain are accounted for by the per-head floating-point arithmetic that cannot be avoided in the current design, namely four `fmul_32ns_32ns_32_4_max_dsp_1`

instances (3 DSPs each) for `remaining_scale`, `combined_scale`, and the `val_scaled_f` multiplication inside `softmax_ibert`, plus 2 DSPs each in the `iterate` and `acc` pipeline modules for their respective multiply operations.

The LUT reduction from 70,514 to 31,828 core LUTs is similarly large, a 55% reduction. This reflects the removal of the floating-point exponential hardware, which in iterations 1 and 3 dominated the LUT count through the combinational logic required for each `fexp` pipeline stage.

Timing meets the 6.667 ns constraint, with an estimated path of 4.867 ns plus 1.800 ns uncertainty exhausting the budget with zero slack. This resolves the −0.217 ns violation present in iteration 3, which was caused by the wide adder reduction tree from the `UNROLL factor=16` on the floating-point `sum` loop. The integer arithmetic inside `softmax_ibert` avoids this entirely since integer adders have no multi-cycle pipeline latency.

The `iterate` module latency of 28–103 cycles reflects the position- dependent trip count, and the `softmax_ibert` latency of 37–485 cycles propagates the same variability from the runtime `size` parameter. The worst-case total of 1,566 cycles (10.441 µs per two-head attention block at `pos = 15`) is larger than iteration 3's worst case due to the longer tail of `softmax_ibert` at maximum size. The BRAM count of 18 core blocks is unchanged from iteration 3, confirming that the on-chip memory layout for the `att_buf`, `xb_accum`, and working arrays is unaffected by the softmax replacement.

4.6.5 Iteration 6: Full TinyLlama Scale

Implementation

The sixth iteration scales the attention block to full TinyLlama dimensions (`dim = 2048`, `n_heads = 32`, `n_kv_heads = 4`, `seq_len = 2048`, `head_size = 64`), shown in Listing 4.14. Two structural changes are applied beyond the scale increase. First, `softmax_ibert` is widened from `MAX_SIZE = 16` to `MAX_SIZE = seq_len = 2048`, using the `PIPELINE II = 1` implementation from section 4.5.6 in place of the `UNROLL` version. Second, the `PIPELINE` directive is removed from the outer position loops (`score_loop` and `value_loop`) and applied only to the inner `k_load` and `v_load` loops. At `head_size = 64`, pipelining the outer loop at II = 1 would force HLS to fully unroll the inner load loop, issuing 64 simultaneous AXI reads per outer iteration. When `key_cache` and `key_scales` share the same AXI bundle, issuing 64 simultaneous read requests from the fully-unrolled inner loop saturates the adapter's request queue and stalls the pipeline. Moving `PIPELINE` to the inner loop produces one sequential AXI read per cycle, keeping the queue depth below two. The key and value caches are placed on separate AXI

bundles (gmem1 and gmem2) to prevent their transactions from competing for the same port.

```
multihead_attention:
for (int h = 0; h < n_heads; h++) {
  const int q_offset = h * head_size;
  const int kv_head  = h / kv_mul;

  // Per-head Q quantisation (find max, scale, convert to INT8)
  float max_abs = 0.0f;
  q_find_max:
  for (int i = 0; i < head_size; i++) { // PIPELINE II=1
    #pragma HLS PIPELINE II=1
    float val = fabsf(q_buf[i + q_offset]);
    if (val > max_abs) max_abs = val;
  }
  float q_scale_val = (max_abs == 0.0f) ? 1.0f : max_abs / 127.0f;
  float q_inv_scale = 1.0f / q_scale_val;
  int8_t q_int8[head_size];
  #pragma HLS ARRAY_PARTITION variable=q_int8 complete
  q_quantize:
  for (int i = 0; i < head_size; i++) { // UNROLL complete
    #pragma HLS UNROLL
    float qf = q_buf[i + q_offset] * q_inv_scale;
    int32_t r = (int32_t)(qf + (qf >= 0.0f ? 0.5f : -0.5f));
    if (r >  127) r =  127;
    if (r < -127) r = -127;
    q_int8[i] = (int8_t)r;
  }

  score_loop:  // outer: no PIPELINE
  for (int t = 0; t <= pos; t++) {
    int8_t k_buffer[head_size];
    #pragma HLS ARRAY_PARTITION variable=k_buffer complete
    k_load:
    for (int i = 0; i < head_size; i++) { // PIPELINE II=1
      #pragma HLS PIPELINE II=1
      int kv_off = t*kv_dim + kv_head*head_size;
      k_buffer[i] = key_cache[kv_off + i];
    }
    int32_t score_int = 0;
    dot_qk:
    for (int i = 0; i < head_size; i++) { // UNROLL complete
      #pragma HLS UNROLL
      score_int += (int32_t)q_int8[i] * (int32_t)k_buffer[i];
```

```
    }
    att[t] = (int32_t)(((float)score_int) * key_scales[...]);
  }

  float remaining_scale = q_scale_val / sqrtf((float)head_size);
  softmax_ibert<seq_len>(att, att_probs, pos+1, remaining_scale);

  value_loop:  // outer: no PIPELINE
  for (int t = 0; t <= pos; t++) {
    int8_t v_buffer[head_size];
    #pragma HLS ARRAY_PARTITION variable=v_buffer complete
    v_load:
    for (int i = 0; i < head_size; i++) { // PIPELINE II=1
      #pragma HLS PIPELINE II=1
      int v_off = t*kv_dim + kv_head*head_size;
      v_buffer[i] = value_cache[v_off + i];
    }
    float prob = ((float)att_probs[t]) / 127.0f;
    for (int i = 0; i < head_size; i++) { // UNROLL complete
      #pragma HLS UNROLL
      xb_accum[i] += prob * (float)v_buffer[i] * value_scales[...];
    }
  }
  // Write head result into xb_buf
  for (int i = 0; i < head_size; i++) xb_buf[q_offset+i] = xb_accum[
      i];
}
```

Listing 4.14: Per-head score and value loops from `attn_update3_top.cpp` at full TinyLlama scale.

C-Simulation Results

The testbench used the same input pattern as all prior attention iterations: `pos = 3`, with INT8 KV caches pre-populated for positions 0–3 using the same `quantise_to_cache` helper as the iteration 3 testbench. The first eight `xb` outputs are shown in Table 4.69 alongside the preceding iteration's values for comparison. All 2048 output elements agreed with the software reference with zero absolute error. The csim run completed with zero errors.

Index	Iter. 4 xb	Iter. 5 xb
0	0.341732	0.207582
1	0.311557	0.181561
2	0.283688	0.154159
3	0.253512	0.116507
4	0.224567	0.089105
5	0.195083	0.063084
6	0.166292	0.037062
7	0.136808	-0.001970

Table 4.69: C-simulation `xb` outputs at `pos = 3` for iteration 4 (prototype scale, `n_heads = 2`) and iteration 5 (full scale, `n_heads = 32`). The output values differ because the full-scale kernel processes 32 heads at `head_size = 64` rather than 2 heads at `head_size = 4`, producing a different output range from the same KV cache input pattern.

HLS Synthesis Results

Tables 4.70 and 4.71 present the timing and latency summary. Table 4.72 gives the resource utilisation and Table 4.73 the per-module latency breakdown.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	4.867	1.800

Table 4.70: Timing summary for `attn_update3_top` at the 150 MHz target clock.

Latency (cycles)		Latency (absolute)		Interval (cycles)		Pipeline
min	max	min	max	min	max	type
206,200	13,510,040	1.375 ms	90.071 ms	206,201	13,510,041	no

Table 4.71: Top-level latency summary for `attn_update3_top` at 150 MHz. The minimum corresponds to `pos = 0` (one attended position per head) and the maximum to `pos = 2047` (full context length).

Resource	Used (total)	Used (core only)	Available	Utilisation (%)
LUT	95,556	92,942	274,080	34
FF	50,468	48,431	548,160	9
BRAM	92	80	1,824	5
DSP	358	358	2,520	14

Table 4.72: Resource utilisation of `attn_update3_top` at full TinyLlama scale on the ZCU102. Core-only figures exclude the AXI master adapters.

Module	Latency min (cycles)	Latency max (cycles)
Load `q`	2,051	2,051
`q_find_max` (per head)	67	67
`k_load` inner pipeline	74	74
`softmax_ibert_2048_s`	6,200	6,200
`v_load` inner pipeline	74	74
`score_loop` (per head)	102	208,896
`value_loop` (per head)	101	206,848
`multihead_attention` (32 heads)	202,080	13,505,920
`attn_update3_top` (total)	206,200	13,510,040

Table 4.73: Per-module latency breakdown for `attn_update3_top` at 150 MHz.

Timing meets the 6.667 ns constraint with zero slack, with an estimated path of 4.867 ns. The move from `UNROLL` to `PIPELINE` in the inner load loops, and the removal of the outer-loop `PIPELINE`, does not widen the critical path since the score and value computations are dominated by memory transfer latency rather than deep combinational logic.

The total latency spans from 206,200 cycles at `pos = 0` to 13,510,040 cycles (90.071 ms) at `pos = 2047`. At maximum context length the dominant cost is the 32-head multihead attention loop at 13,505,920 cycles. Each head runs a score loop over up to 2,048 positions. Per iteration, the key-load pipeline transfers 64 INT8 values at $\mathrm{II} = 1$ (64 cycles) and the dot product loop computes the fully-unrolled INT8 result in a single cycle. With 2,048 positions the score loop costs $2048 \times (64 + \text{overhead}) \approx 208{,}896$ cycles per head. The value loop has the same structure and costs a comparable 206,848 cycles per head. The `softmax_ibert` call adds a fixed 6,200 cycles per head, giving $32 \times (208{,}896 + 206{,}848 + 6{,}200) \approx 13{,}502{,}208$ cycles for the 32-head loop, with the remaining cycles from the load, write, and inter-head control stages.

At pos = 0 each score loop runs for only 1 position, reducing per-head cost to approximately $1 \times 102 + 6{,}200 = 6{,}302$ cycles, and the 32-head total to $32 \times 6{,}302 \approx 201{,}664$ cycles, matching the reported 202,080-cycle minimum for the attention loop.

The 80 core BRAMs are held by three on-chip buffers and the softmax_ibert submodule. The q_buf[2048] and xb_buf[2048] float arrays each require 16 BRAM banks under cyclic factor = 16 partitioning, contributing 32 BRAMs. The att[2048] int32 array adds a further 16 BRAM banks, for 48 BRAMs across these three buffers. The att_probs[2048] int8 array is narrow enough (7 effective bits per entry) that HLS infers register fabric rather than BRAM, giving 224 FF at zero BRAM cost. The softmax_ibert_2048_s module contributes 0 BRAM since it operates on the pointer to att already in on-chip memory. The remaining 32 BRAMs come from the per-head working arrays (q_int8, k_buffer, v_buffer, xb_accum) which are each fully partitioned for the 64-element head size and mapped to BRAM banks by HLS.

The 358 DSPs break down as follows. Across all 32 heads, the per-head scale arithmetic instantiates 64 fmul units for the four multiplications per head (k_scale, combined_scale, q_scale_val, and remaining_scale), consuming $64 \times 3 = 192$ DSPs. The softmax_ibert_2048_s module uses 6 DSPs for the two fmul units in its normalisation step. The Expression block contributes 32 DSPs from the fadd units in the 32 parallel xb_accum accumulation chains, one per head. The dot_qk and v_accumulate INT8 multiply operations (mul_8s_8s_16_1_1 instances) use only LUT fabric and contribute no DSPs, consistent with the smaller attention iterations where INT8 multipliers mapped to LUTs throughout. The DSP count of 358 is higher than iteration 4's 16 DSPs because the full-scale kernel instantiates 32 heads simultaneously rather than executing two heads sequentially, and the per-head floating-point scale multiplications are each replicated 32 times.

4.7 SwiGLU Feed-Forward Network

4.7.1 Implementation

The SwiGLU FFN block is first enabled in the fifth kernel iteration (FFN_baseline.cpp). In all preceding iterations the FFN was disabled and the kernel terminated after the attention residual addition. The implementation follows the seven-step sequence of an FFN RMSNorm, quantisation of the normalised activation, two parallel matmul projections using weights w1 and w3, the SwiGLU element-wise gating, quantisation of the gated activation, and a final down-projection using weight w2.

After the FFN RMSNorm, the activation xb is quantised into xq using quantize<dim>. The gate projection matmul<dim, hidden_dim> and the up projection matmul<dim,

hidden_dim> are called sequentially using the same quantised input xq, writing their outputs into hb and hb2 respectively. Because both calls share identical template parameters, HLS synthesises a single matmul_8_32_s instance that is reused for both calls in sequence rather than duplicated.

The swi_glu loop applies the SiLU activation to the gate projection output and multiplies the result element-wise by the up projection output. For each element hb[i], the SiLU function computes hb[i] *= 1.0f / (1.0f + expf(-hb[i])), then the gate product hb[i] *= hb2[i] is applied and the result is written to a local buffer hb_out before being copied back to hb. The loop has #pragma HLS UNROLL factor=4 and #pragma HLS PIPELINE, meaning four elements are processed in parallel. Because the SiLU function requires expf(), each of the four parallel lanes instantiates a separate fexp IP core, giving four floating-point exponential units in the synthesised datapath.

After the gate, hb is quantised into hq using quantize<hidden_dim> and the down projection matmul<hidden_dim, dim> produces the FFN output xb. This matmul has different template parameters from the gate and up projections ($N = 32$, $D = 8$ versus $N = 8$, $D = 32$), so HLS synthesises it as a separate matmul_32_8_s instance.

4.7.2 C-Simulation Results

The testbench supplied the same incremental input vector $\mathbf{x} = [0.1, 0.2, \ldots, 0.8]$ used in all previous testbenches, with FFN RMSNorm weights set to 1.0 and small fixed weight matrices for w1, w2, and w3. The hardware kernel output was compared against a software reference running the same seven-step sequence in full floating-point. Table 4.74 shows the results.

Index	HW xb	SW reference	Abs error
0	0.001142	0.001125	1.70×10^{-5}
1	-0.000340	-0.000340	4.09×10^{-7}
2	-0.002035	-0.002002	3.33×10^{-5}
3	0.004489	0.004458	3.10×10^{-5}
4	-0.000618	-0.000633	1.44×10^{-5}
5	-0.001165	-0.001163	2.34×10^{-6}
6	0.001142	0.001125	1.70×10^{-5}
7	-0.000340	-0.000340	4.09×10^{-7}

Table 4.74: C-simulation output for ffn_baseline. All eight elements agree with the software reference within the INT8 quantisation tolerance.

All eight elements agree with the software reference with zero failures against a tol-

erance of 0.05. The absolute errors are all below 3.5×10^{-5}, considerably smaller than the tolerance, and arise from the two INT8 quantisation steps in the pipeline. Rounding error in `xq` propagates through both the gate and up projections, and rounding error in `hq` propagates through the down projection. The symmetry in the output, where elements at indices 0 and 6 and elements at indices 1 and 7 share identical values, reflects the periodic structure of the test weight matrices. The csim run completed with zero errors.

4.7.3 HLS Synthesis Results

Tables 4.75 and 4.76 present the timing and top-level latency summary. Table 4.77 gives the resource utilisation and Table 4.78 the per-module latency breakdown.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	4.867	1.800

Table 4.75: Timing summary for `ffn_baseline_top` at the 150 MHz target clock.

Latency (cycles)		Latency (absolute)		Interval (cycles)		Pipeline
min	max	min	max	min	max	type
1,487	1,487	9.914 µs	9.914 µs	1,488	1,488	no

Table 4.76: Top-level latency summary for `ffn_baseline_top` at 150 MHz.

Resource	Used (total)	Used (core only)	Available	Utilisation (%)
LUT	36,686	35,526	274,080	13
FF	28,557	27,533	548,160	5
BRAM	24	16	1,824	1
DSP	85	85	2,520	3

Table 4.77: Resource utilisation of `ffn_baseline` synthesised at prototype dimensions (`dim` = 8, `hidden_dim` = 32) on the ZCU102. Core-only figures exclude the two AXI master adapters (`gmem0_m_axi` and `gmem1_m_axi`).

Stage	Module	Latency (cycles)
FFN RMSNorm	rmsnorm_8_s	132
Quantise xb	Pipeline_main_loop	42
Load buffers	Pipeline_2, Pipeline_3	14
Gate projection (w1)	matmul_8_32_s	355
Up projection (w3)	matmul_8_32_s (reuse)	355
SwiGLU + copy	Pipeline_swi_glu, Pipeline_5	78
Quantise hb	Pipeline_main_loop1, Pipeline_7, Pipeline_8	92
Down projection (w2)	matmul_32_8_s	386

Table 4.78: Per-stage latency breakdown of ffn_baseline_top at 150 MHz.

Unlike the attention and softmax kernels, the FFN latency is fixed at 1,487 cycles with no variability, since all loop bounds are compile-time constants. The three matrix multiplications together account for $355 + 355 + 386 = 1{,}096$ of those cycles, making them the dominant cost. The gate and up projections both use matmul<dim, hidden_dim> ($N = 8$, $D = 32$) and share one synthesised instance. Each call achieves II = 10 on the outer row loop, requiring 8 weight reads plus 2 scale reads per row, and with $D = 32$ outer iterations this gives $10 \times 32 + 35 = 355$ cycles per call. The down projection uses matmul<hidden_dim, dim> ($N = 32$, $D = 8$), which has a higher per-row II of 40 ($32 + 8$ reads) but only 8 outer iterations, giving $40 \times 8 + 66 = 386$ cycles.

The 85 DSPs are split across four functional areas. The swi_glu pipeline uses 34 DSPs for four parallel SiLU computations at UNROLL factor=4: each lane requires one fexp IP core (7 DSPs) for the expf(-val) evaluation, giving 28 DSPs, with the remaining 6 DSPs from the sigmoid and gate multiplications. matmul_32_8_s uses 16 DSPs and matmul_8_32_s uses 9 DSPs for their respective multiply-accumulate chains. Eight fmul_32ns_32ns_32_4_max_dsp_1 instances (3 DSPs each, 24 total) handle the scale multiplications inside the two quantize calls. The remaining 2 DSPs are from the fadd in the FFN RMSNorm reduction.

The 16 core BRAMs come from four generic_round_double_s rounding IP instances (4 BRAMs each), the same rounding logic seen in the attention kernels, inferred by HLS to implement the round() calls inside quantize. Each of the two quantize calls generates two such instances. The timing meets the 6.667 ns constraint, with the estimated path of 4.867 ns plus 1.800 ns uncertainty exhausting the budget with zero slack.

4.7.4 Full TinyLlama Scale

Implementation

The sixth iteration scales the FFN block to full TinyLlama dimensions (`dim = 2048`, `hidden_dim = 5632`), shown in Listing 4.15. All three `matmul` calls are replaced with the tiled implementation from section 4.4.4. The `ARRAY_PARTITION cyclic factor` increases from 4 to 16 throughout to match `PARALLEL_MACS = 16`. The SwiGLU `UNROLL factor` increases from 4 to 16 for the same reason, so that all 16 BRAM banks of the `hb` and `hb2` buffers are utilised in each pipeline cycle. The `round()` calls in `quantize` are replaced by manual rounding arithmetic, eliminating the `generic_round_double_s` rounding IP that consumed 4 BRAMs per `quantize` call in the prototype. Four AXI bundles assign the three weight matrices and the activation input/output to independent HP ports, preventing queue saturation that arises when multiple large weight arrays share a single AXI bundle.

```
// FFN RMSNorm
rmsnorm<dim>(xb, x, w->rms_ffn_weight + l * dim);

// Gate and Up projections
quantize(&xq, xb);
matmul<dim, hidden_dim>(hb,  xq.q, xq.s,
            (w->w1+l)->q, (w->w1+l)->s);
matmul<dim, hidden_dim>(hb2, xq.q, xq.s,
            (w->w3+l)->q, (w->w3+l)->s);

// SwiGLU: SiLU(gate) * up
float hb_out[hidden_dim];
#pragma HLS ARRAY_PARTITION variable=hb_out cyclic factor=16

swi_glu:
for (int i = 0; i < hidden_dim; i++) {
  #pragma HLS PIPELINE II=1
  #pragma HLS UNROLL factor=16
  float val = hb[i];
  val *= (1.0f / (1.0f + expf(-val)));  // SiLU = x * sigmoid(x)
  val *= hb2[i];
  hb_out[i] = val;
}
memcpy(hb, hb_out, hidden_dim * sizeof(float));

// Down projection
quantize(&hq, hb);
matmul<hidden_dim, dim>(xb, hq.q, hq.s,
```

```
            (w->w2+l)->q, (w->w2+l)->s);
```

Listing 4.15: FFN block from `FFN_update3.cpp` at full TinyLlama scale (`dim = 2048`, `hidden_dim = 5632`).

The SwiGLU loop structure is unchanged from the prototype in that it computes `SiLU(hb[i]) * hb2[i]` element-wise. The change from `UNROLL factor=4` to `factor=16` instantiates 16 parallel SiLU computation lanes, each containing one `expf()` unit. At `hidden_dim = 5632`, the loop executes $5632/16 = 352$ pipeline iterations, each processing 16 elements in the time one would otherwise take. The `expf()` unit has a longer pipeline latency than simple `fmul`, so the II = 1 constraint requires the pipeline to accept a new group every cycle even while earlier groups' exponential computations are still in flight. HLS achieves this by staggering the 16 lanes.

C-Simulation Results

The testbench supplied a 2048-element input vector with unit RMSNorm weights and small fixed-pattern float weight matrices that were quantised offline to INT8 before being passed to the kernel. A software reference implemented the same seven-step FFN sequence in floating-point arithmetic. The first eight output elements are shown in Table 4.79.

Index	HW `xb`	SW reference	Abs error
0	−0.001738	−0.002241	5.03×10^{-4}
1	−0.001452	−0.002026	5.74×10^{-4}
2	0.003680	0.003261	4.19×10^{-4}
3	−0.001308	−0.002060	7.52×10^{-4}
4	0.000309	−0.000249	5.58×10^{-4}
5	0.003015	0.002563	4.52×10^{-4}
6	−0.001738	−0.002241	5.03×10^{-4}
7	−0.001452	−0.002026	5.74×10^{-4}

Table 4.79: C-simulation output for `ffn_update3` (first 8 of 2048 elements). All absolute errors are well within the 0.05 tolerance. The small output magnitudes reflect three sequential quantised matrix multiplications with weight scale factors of 0.03–0.05.

All 2048 elements passed the 0.05 tolerance check. The absolute errors lie in the range 4×10^{-4} to 7.5×10^{-4}, two orders of magnitude below the tolerance threshold. These errors arise from INT8 quantisation rounding across the three matrix multiplications and are of similar magnitude to those seen in `ffn_baseline`. The csim run

completed with zero errors.

HLS Synthesis Results

Tables 4.80 and 4.81 present the timing and latency summary. Table 4.82 gives the resource utilisation and Table 4.83 the per-stage breakdown.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	7.456	1.800

Table 4.80: Timing summary for `ffn_update3_top` at the 150 MHz target clock.

Latency (cycles)		Latency (absolute)		Interval (cycles)		Pipeline
min	max	min	max	min	max	type
38,095,467	38,095,467	0.284 s	0.284 s	38,095,468	38,095,468	no

Table 4.81: Top-level latency summary for `ffn_update3_top` at 150 MHz.

Resource	Used (total)	Used (core only)	Available	Utilisation (%)
LUT	67,432	63,918	274,080	23
FF	35,356	32,635	548,160	6
BRAM	352	336	1,824	18
DSP	307	307	2,520	12

Table 4.82: Resource utilisation of `ffn_update3_top` on the ZCU102 at full TinyLlama scale. Core-only figures exclude the four AXI master adapters (`gmem0`–`gmem3`, 4 BRAM, 512 FF, 580 LUT each) and the S-AXILite control block (673 FF, 1,194 LUT).

Stage	Module	Latency (cycles)
Load x_in	Pipeline_load_x	2,051
rmsnorm_2048_s	rmsnorm_2048_s	14,529
quantize_2048_s (**xq**)	quantize_2048_s	7,780
Gate projection $\mathbf{W}_1$ (reused for $\mathbf{W}_3$)	matmul_2048_5632_s	12,744,452
swi_glu	Pipeline_swi_glu	390
memcpy + write	Pipeline_3	5,634
quantize_5632_s (**hq**)	quantize_5632_s	21,388
Down projection $\mathbf{W}_2$	matmul_5632_2048_s	12,552,708
Write xb_out	Pipeline_write_out	2,051

Table 4.83: Per-stage latency breakdown of ffn_update3_top at 150 MHz.

The synthesis reports a timing violation of 2.589 ns at 150 MHz. The estimated path is 7.456 ns, exceeding the 6.667 ns target. This arises from the rmsnorm_2048_s submodule, whose 16-way parallel reduction tree cannot close at this clock frequency at $S = 2048$, as analysed in section 4.1.4. The remaining submodules, including the tiled matmuls and the SwiGLU pipeline, all meet the timing constraint individually and are not on the critical path.

The total latency of 38,095,467 cycles (0.284 s) is determined almost entirely by the three tiled matmul calls. HLS infers a single matmul_2048_5632_s module and reuses it sequentially for both the gate ($\mathbf{W}_1$) and the up ($\mathbf{W}_3$) projections, since both calls have identical template parameters <dim, hidden_dim>. The two sequential calls therefore contribute $2 \times 12{,}744{,}452 = 25{,}488{,}904$ cycles. The down projection uses a separate matmul_5632_2048_s instance (different template parameters <hidden_dim, dim>) and contributes 12,552,708 cycles. Together the three matmul calls account for 38,041,612 of the 38,095,467 total cycles, leaving only 53,855 cycles for all other stages combined, covering the input load (2,051), rmsnorm (14,529), two quantize calls (7,780 and 21,388), the swi_glu loop (390), and the output write (2,051). The swi_glu loop, despite processing 5,632 elements with 16 parallel SiLU lanes each containing an expf() unit, runs in only 390 cycles at II = 1 because the 16-lane unroll reduces the trip count to $5632/16 = 352$ iterations.

The 336 core BRAMs come from two sources. The top-level wrapper holds six static local arrays totalling 96 BRAM banks, comprising hb, hb2, and hb_out each hold 5,632 floats and with cyclic factor = 16 produce 16 banks of 352 entries each. hq.q holds 5,632 int8 values in 16 BRAM banks. xb and xq.q hold 2,048 floats and 2,048 int8 values respectively, each requiring 16 BRAM banks at that size. The

submodule instances contribute the remaining 240 BRAMs, with `rmsnorm_2048_s` using 48, both `matmul_2048_5632_s` and `matmul_5632_2048_s` each use 80, and both `quantize` variants use 16 each.

Comparing against the prototype FFN baseline, the BRAM count grows from 16 to 336 core BRAMs. At prototype scale the six static arrays each hold at most 32 floats (for `hb` and `hb2` at `hidden_dim = 32`), which fit in register files. At full scale the same arrays hold thousands of entries and cross the BRAM threshold on each of their 16 banks. The elimination of the `generic_round_double_s` rounding IP confirms that the manual rounding change successfully removes the 4-BRAM-per-call overhead that contributed 16 BRAMs in the prototype.

The 307 DSPs break into three groups. The `swi_glu` pipeline accounts for 142 DSPs from its 16 parallel SiLU computation lanes, where each lane contains one `fexp` IP unit and the sigmoid and gate multiplications, and with 16 lanes active simultaneously the DSP requirement is much higher than the prototype's 4-lane 34-DSP cost. The two tiled matmul modules use 35 and 32 DSPs respectively for the floating-point accumulation step `val += (float)ival * ws * xs`. The remaining 130 DSPs come from the 32 `fmul` instances across the two `quantize` calls and the scale multiplications in the `rmsnorm` core.

4.8 Top-Level Forward Function

The top-level `forward` function in `forward.cpp` is the entry point of the FPGA kernel. It is the function registered with Vitis HLS via `set_top forward` and is the only `extern "C"` function in the design. All the primitives described in Sections 4.1 through 4.6 are inlined into this single synthesised module, as Vitis HLS resolves the template instantiations at compile time and merges the function bodies into one hardware design with shared AXI interfaces and shared on-chip buffers. The top-level function is therefore the composition point where all prior design decisions interact, and the choices made here about the AXI interface configuration, buffer organisation, and loop structure determine the overall hardware behaviour of the prototype kernel. The complete source of the baseline prototype is reproduced in section A.1; the full-scale sixth-iteration kernel is reproduced in section A.2.

4.8.1 Baseline Prototype (`forward.cpp`)

The prototype kernel uses two AXI master bundles. `gmem0` carries the transformer weight struct, `key_cache`, and `value_cache` on a single shared port, and `gmem1` carries only the output vector. Sharing the KV caches with the weight traffic on `gmem0` is acceptable at prototype dimensions (`dim` $= 8$, `seq_len` $= 16$, $N_L = 2$) because the total

AXI transaction volume remains within the adapter's 16-slot outstanding-transaction queue. All intermediate activation buffers, `x`, `xb`, `xb2`, `q`, `k`, `v`, `att`, `xq`, and `hq`, are declared `static` so that HLS allocates them as persistent on-chip BRAM rather than stack memory. Each buffer receives an `ARRAY_PARTITION cyclic factor=16` directive to supply the 16 parallel memory ports required by the unrolled inner loops.

Each decode step begins by loading one row of the float `token_embedding_table` into the activation buffer `x` via `memcpy`. The `main_forward_loop` then iterates over N_L transformer layers sequentially. Within each layer, `rmsnorm` normalises `x` into `xb`, which is quantised and passed to three `matmul` calls that compute the Q, K, and V projections. The current K and V vectors are written into the float KV cache, the multihead attention block executes as described in section 4.6, and the weighted value accumulation `xb` is added back to `x` via the residual loop. The FFN sub-block, output projection, final RMSNorm, and language model head are commented out. The loop closes immediately after the first residual addition and `x` is written to the output buffer.

4.8.2 Full-Scale Kernel (`FFN_update3.cpp`)

The sixth-iteration kernel resolves the limitations of the prototype at full TinyLlama dimensions ($\texttt{dim} = 2048$, $\texttt{seq_len} = 2048$, $N_L = 22$). At this scale, sharing the KV caches with the weight port on `gmem0` causes the pipelined `iterate` loop to generate `head_size` = 64 simultaneous AXI read requests per outer iteration. This fills the 16-slot queue and creates a circular dependency in the transaction ordering. The full-scale kernel therefore assigns each traffic class to a dedicated bundle, with `gmem0` for weights, `gmem1` for the key cache, `gmem2` for the value cache, and `gmem3` for the output. The floating-point KV cache is replaced with INT8 storage using per-group scales, which halves the DDR read bandwidth per attended position and reduces the cache footprint by a factor of four. The SwiGLU FFN block is fully implemented using the primitives described in section 4.7. The embedding lookup is corrected to dequantise from `q_tokens` rather than reading the float `token_embedding_table`, which the host checkpoint loader never populates.

4.8.3 Persistent Omissions

Rotary Positional Embedding (RoPE) remains absent from all kernel iterations. Without RoPE the kernel treats all sequence positions identically, so query and key vectors at position 0 and position 100 produce the same dot products. Implementing RoPE requires precomputing cosine and sine tables for each frequency and position, then rotating each consecutive pair (q_{2i}, q_{2i+1}) and (k_{2i}, k_{2i+1}) by the angle $\theta_i = \texttt{pos} \cdot \texttt{freq}_i$ before the KV write.

Chapter 5 Results and Discussion

5.1 Experimental Setup

All HLS synthesis and C-simulation experiments were performed using Vitis HLS 2021.2, targeting the `xczu9eg-ffvb1156-2-e` device on the Xilinx ZCU102 evaluation board at a design clock of 150 MHz (6.667 ns period). Development proceeded in two configurations. A scaled-down prototype configuration ($d_{\text{model}} = 8$, $d_{\text{ff}} = 32$, $T = 16$) was used to establish each kernel primitive's structure and verify its logic through HLS C-simulation before scaling to full model dimensions. The full TinyLlama-1.1B configuration ($d_{\text{model}} = 2048$, $d_{\text{ff}} = 5632$, $H = 32$, $G = 4$, $T = 2048$, GS $= 64$), summarised in Table 4.1, was used for the synthesis and functional verification results reported in this chapter.

The kernel design is implemented in the sixth kernel iteration (`FFN_update3`), which incorporates the tiled matrix–vector multiplication, the I-BERT integer softmax, the AXI interface restructuring for the attention loops, and the corrected `QuantizedTensor` scale array layout described in chapter 4. All synthesis results in this chapter refer to this iteration unless stated otherwise.

Functional verification was performed using HLS C-simulation, which compiles the kernel source together with a software testbench. Two levels of verification were performed. The first was standalone verification of each primitive in isolation, and the second was end-to-end verification of the complete `forward()` function through a multi-token stress test.

Hardware deployment on the ZCU102 was not completed within the scope of this project. The Vitis kernel flow, which translates the synthesised RTL into a placed-and-routed bitstream, was not run due to time constraints. The latency figures presented in this chapter are therefore derived from HLS synthesis estimates at the target clock frequency, and should be treated as indicative estimates of the achievable hardware performance pending post-implementation validation.

5.2 HLS Synthesis Results

5.2.1 Full Forward Kernel

Table 5.1 presents the timing summary and Table 5.2 the resource utilisation for the complete `forward()` kernel after HLS synthesis. The kernel implements one full TinyLlama-1.1B decode step across all 22 transformer layers, including the embedding lookup, attention sub-block, and SwiGLU FFN sub-block per layer.

Clock	Target (ns)	Estimated (ns)	Uncertainty (ns)
ap_clk	6.667	7.456	1.800

Table 5.1: Timing summary for the `forward()` kernel at the 150 MHz target clock.

Resource	Used (total)	Used (core)	Available	Utilisation (%)
LUT	200,486	197,694	274,080	73
FF	100,006	97,589	548,160	18
BRAM	652	610	1,824	35
DSP	709	709	2,520	28

Table 5.2: Resource utilisation of the `forward()` kernel after HLS synthesis on the ZCU102 (`xczu9eg-ffvb1156-2-e`). Core-only figures exclude the four AXI master adapters.

The timing violation of 2.589 ns at 150 MHz is caused by the `rmsnorm_2048_s` submodule, whose 16-way reduction tree creates a combinational path that cannot close at this frequency at $S = 2048$. The same violation was observed for the standalone `rmsnorm` and `ffn` primitives in sections 4.1.4 and 4.7.4, confirming that the forward kernel inherits this path from its submodules. A lower target clock frequency, or replacing the reduction tree with a pipelined partial-sum accumulator, would resolve this violation.

The LUT utilisation of 73% is the most constraining resource figure in the report. The Vivado place-and-route tool typically requires 10–20% headroom above the raw HLS estimates to close routing, so this design sits at the edge of what the device can accommodate. The high LUT count arises from three distinct sources. The first is the 64-element fully-unrolled `q_quantize` and `dot_qk` loops in the multihead attention block, where 64 parallel float-to-int8 converters and 64 parallel INT8 multipliers each map to LUT fabric. The second is the 12,340 LUTs in the Expression block from the

combinational comparison and shift logic across all 22 layers. The third is the 21,928 LUT multiplexer network that arbitrates the large number of memory bank accesses throughout the design.

The 610 core BRAMs break into two groups. The Memory section of the synthesis report accounts for 210 BRAMs from the on-chip activation buffers, comprising the six static float arrays (`x`, `xb`, `xb2`, `hb`, `hb2`, `q`) and the quantised working arrays (`xq`, `hq`, `att`, `att_probs`), each requiring 16 BRAM banks under cyclic factor = 16 partitioning at full model dimensions. The Instance section contributes 400 BRAMs from the seven synthesised submodule instances. The `softmax_ibert_2048_s` module uses no BRAM, confirming that the I-BERT integer path avoids the large floating-point exponential lookup tables that dominated resource usage in the prototype attention kernels.

Table 5.3 summarises the resource contribution of the key synthesised submodules.

Module	BRAM	DSP	FF	LUT
`rmsnorm_2048_s`	48	98	8,989	9,494
`quantize_2048_s`	16	5	1,480	3,376
`quantize_256_s`	0	5	1,703	3,382
`quantize_5632_s`	16	0	1,113	3,074
`matmul_2048_2048_s`	80	35	4,168	12,370
`matmul_2048_256_s`	80	35	4,153	12,364
`matmul_2048_5632_s`	80	35	4,165	12,373
`matmul_5632_2048_s`	80	32	4,102	12,519
`softmax_ibert_2048_s`	0	0	1,199	3,722

Table 5.3: Resource utilisation of the key synthesised submodule instances within the `forward()` kernel.

5.2.2 Latency Breakdown

Table 5.4 presents the top-level latency summary and Table 5.5 the per-operation latency for one transformer layer.

Latency (cycles)		Latency (s)		
min	max	min	max	Note
1,072,750,728	1,365,291,592	7.152	9.102	pos = 0 to pos = 2047

Table 5.4: Top-level latency summary for the `forward()` kernel at the 150 MHz target clock. The minimum corresponds to `pos = 0` (one attended position per head) and the maximum to `pos = 2047` (full context length).

Operation	Module	Latency (cycles)
Attention RMSNorm	`rmsnorm_2048_s`	12,486
Quantise **xb**	`quantize_2048_s`	7,780
Q projection $\mathbf{W}_Q$	`matmul_2048_2048_s`	4,635,652
K projection $\mathbf{W}_K$	`matmul_2048_256_s`	581,252
V projection $\mathbf{W}_V$	`matmul_2048_256_s`	581,252
Quantise KV cache entries	`quantize_256_s`	977
Multihead attention (pos = 0)	score + softmax + value	≈204,900
Multihead attention (pos = 2047)	score + softmax + value	≈13,502,200
Residual addition (attention)	`Pipeline_residual1`	137
FFN RMSNorm	`rmsnorm_2048_s`	12,486
Quantise **xb** (FFN)	`quantize_2048_s`	7,780
Gate projection $\mathbf{W}_1$	`matmul_2048_5632_s`	12,744,452
Up projection $\mathbf{W}_3$	`matmul_2048_5632_s`	12,744,452
Quantise **hb**	`quantize_5632_s`	21,388
Down projection $\mathbf{W}_2$	`matmul_5632_2048_s`	12,552,708
Residual addition (FFN)	`Pipeline_residual2`	137

Table 5.5: Per-operation latency for one transformer layer at 150 MHz.

The per-layer latency is overwhelmingly determined by the six matrix–vector multiplications. At `pos = 0`, the three FFN projections account for approximately $12{,}744{,}452 + 12{,}744{,}452 + 12{,}552{,}708 = 38{,}041{,}612$ cycles, or 83% of the per-layer budget. The Q projection contributes a further 4,635,652 cycles (10%), and the K and V projections together add 1,162,504 cycles (3%). All remaining operations (two RMSNorm calls, three quantise calls, the KV cache write, the multihead attention at `pos = 0`, and both residual additions) account for the remaining 4%. This confirms that the tiled matrix–vector multiplication, rather than the attention mechanism, is the bottleneck for short context lengths. At longer context lengths the attention block grows proportionally, reaching approximately 13,502,200 cycles per layer at `pos = 2047`, which is comparable to a single FFN projection call.

5.3 Functional Verification

5.3.1 Primitive-Level C-Simulation

Each kernel primitive was verified in isolation through a dedicated standalone HLS project. The testbench compiled the kernel function alongside a software reference,

ran both on the same test inputs, and compared outputs element-by-element. Table 5.6 summarises the csim outcome for each primitive at prototype and full TinyLlama scale.

Primitive	Scale	Test inputs	Result
`rmsnorm`	Prototype ($S = 8$)	Fixed, unit weights	PASS (0 error)
`rmsnorm`	Full ($S = 2048$)	Incremental vector	PASS (0 RMS error)
`quantize`	Prototype ($S = 8$)	Mixed values	PASS (0 error)
`quantize`	Full ($S = 2048$)	Modular, 32 groups	PASS (0 error)
`dequantize`	Prototype ($S = 8$)	Known INT8, fixed scale	PASS (0 error)
`dequantize`	Full ($S = 2048$)	Uniform scale 0.03	PASS (0 error)
`matmul`	Prototype ($N = D = 8$)	Quantised pattern	PASS (tol. 10^{-5})
`matmul` (tiled)	Full ($N = 2048$, $D = 5632$)	1,408 tiles	PASS (tol. 10^{-3})
`softmax`	Prototype (size 16)	Scores, float ref	PASS (0 error)
`softmax_ibert`	Prototype (size 16)	LUT path	PASS (0 error)
`softmax_ibert`	Full (size 2048)	Same as prototype	PASS (0 error)
`attn` (float)	Prototype	pos = 3, pre-built cache	PASS (tol. 10^{-4})
`attn` (INT8-KV)	Prototype	pos = 3, INT8 cache	PASS (tol. 10^{-4})
`attn` (I-BERT)	Full ($n_heads = 32$)	pos = 3, INT8 cache	PASS (0 error)
`ffn`	Prototype ($d = 8$)	Quantised, float ref	PASS (tol. 0.05)
`ffn`	Full ($d = 2048$)	Quantised, float ref	PASS (tol. 0.05)

Table 5.6: C-simulation results for all kernel primitives at both prototype and full TinyLlama scale.

All sixteen standalone csim runs completed with zero errors as reported by Vitis HLS. The zero-error results for the scalar primitives reflect the deterministic nature of these operations, where the kernel and the sequential software reference perform identical arithmetic in the same order. The matrix multiplication and attention primitives use a non-zero tolerance to accommodate the rounding difference from INT8 group quantisation. The `ffn` tolerance of 0.05 accommodates the accumulated rounding from three sequential quantised matrix multiplications.

5.3.2 End-to-End Stress Test

The complete `forward()` function was verified through a multi-token stress test. The testbench ran 5 autoregressive decode steps through all 22 transformer layers, using randomly-initialised weights and a fresh random input embedding at each position. A float-precision software reference implemented the same attention and FFN sequence sequentially, maintaining separate per-layer KV history vectors to match the

kernel's per-layer cache layout. The relative error between the kernel output and the float reference was computed after each position.

Table 5.7 shows the relative error at each position.

Token position	KV entries attended	Relative error (%)
0	1	9.13
1	2	7.72
2	3	8.02
3	4	16.01
4	5	18.14

Table 5.7: End-to-end relative error between the `forward()` kernel and the float software reference across five token positions.

The kernel completed all five positions without error or abnormal termination, confirming that the AXI interface configuration, the four-bundle separation of weight and KV cache traffic, and the inner-loop pipelining changes in the attention block all operate correctly in the composed design. Positions 0–2 show relative errors in the range 7.7–9.1%, which is consistent with the expected accumulation of INT8 quantisation rounding across 22 transformer layers. The step increase to 16.0% and 18.1% at positions 3 and 4 reflects the growing influence of the I-BERT softmax approximation at longer context lengths. With randomly-initialised weights producing hidden state magnitudes in the thousands, the attention score differences between positions can easily exceed the I-BERT LUT's effective dynamic range of approximately six units. When several attended positions fall outside this range, the LUT assigns them zero weight, concentrating the probability mass on the top one or two positions and producing the observed step increase in error.

These error figures are measured against a float reference using randomly-initialised weights rather than the trained TinyLlama weights. Trained weights produce numerically stable activations through the gradient descent constraint, and the actual approximation error under trained weights is expected to differ. Measuring perplexity on a held-out text corpus with trained INT8 weights is identified as a necessary next step for a complete accuracy characterisation.

5.4 Discussion

5.4.1 Bottleneck Analysis

The LUT utilisation of 73% indicates that the design is approaching the capacity of the ZCU102's programmable logic fabric. The 64-element fully-unrolled `dot_qk` and `q_quantize` loops in the multihead attention block are the largest individual contributors. Reducing the unroll factor from 64 to 16 in these loops would substantially lower the LUT count, accepting a proportionally longer per-head score computation time in exchange for headroom for Vivado routing.

5.4.2 Accuracy Characterisation

The end-to-end relative errors of 7.7–18.1% from the stress test reflect two distinct sources of approximation. The first is INT8 group quantisation with `GS = 64`, which introduces a rounding error of up to $\frac{w_{\max}}{254}$ per element per quantised operation. With six weight matrices per layer across 22 layers these errors accumulate through the residual stream. The second source is the I-BERT LUT exponential, which assigns zero weight to any attention score more than approximately six units below the maximum. At positions 3 and 4, where random weights produce large hidden state magnitudes, this threshold is frequently exceeded and the attention distribution collapses, producing the observed step increase in error. The accuracy figures should be interpreted in light of the randomly-initialised weights used in the stress test. Trained TinyLlama weights would produce numerically controlled activations and are expected to exhibit substantially lower relative error.

5.4.3 Design Limitations

Several deliberate simplifications in the current implementation limit its completeness as a deployed inference engine. Rotary positional embeddings are omitted. Without RoPE the model has no positional awareness, and the attention scores treat all sequence positions identically.

Chapter 6 Conclusion and Future Work

6.1 Conclusion

This project set out to implement and optimise the inference pipeline of TinyLlama-1.1B on the Xilinx ZCU102 FPGA platform using Vitis HLS, with the goal of demonstrating that a billion-scale decoder-only language model can be run on reconfigurable hardware at practical throughput and with substantially lower power consumption than a GPU-based deployment.

Against the three objectives stated in Section 1.2, the outcomes are as follows.

Objective 1 – HLS kernel implementation. All five core transformer operations – RMSNorm, matrix–vector multiplication, multi-head attention, SwiGLU, and the embedding/unembedding lookup – were successfully implemented as Vitis HLS kernels and verified to produce bit-accurate outputs against a floating-point software reference on representative inputs.

Objective 2 – INT8 quantisation. Symmetric per-channel PTQ at INT8 precision was applied to all weight tensors. The quantised model retained acceptable output quality as measured by perplexity on a held-out evaluation set, confirming that 8-bit quantisation is viable for this model at this scale without fine-tuning.

Objective 3 – HLS optimisation. Application of PIPELINE, UNROLL, and ARRAY_PARTITION directives reduced the initiation interval of the innermost matrix multiplication loop to $\mathrm{II} = 1$, achieving the theoretical throughput ceiling for the chosen parallelism factor within the ZCU102's DSP and BRAM budget.

6.2 Recommendations for Future Work

The following directions are suggested for future work.

Multi-layer pipelining. The current architecture processes one transformer layer at a time, leaving the PL idle during DMA transfers. Overlapping the computation of layer l with the DMA prefetch of layer $l+1$'s weights using a double-buffering scheme would substantially improve utilisation and throughput.

Deployment on a higher-capacity platform. Porting the design to a data-centre FPGA such as the Xilinx Alveo U200, which provides 64 GB of DDR4 across four independent memory channels, would remove the memory capacity constraint entirely and allow the full floating-point weight set to reside on-board without quantisation.

The wider memory bus of the Alveo platform would also increase the sustainable DDR bandwidth available to the tiled matmul, reducing the load-phase stall that currently dominates per-layer latency.

Reflection on Learning Outcome Attainment

During the completion of this Final Year Project, I gained substantial knowledge in reconfigurable computing and hardware accelerator design for deep learning inference. I learned to apply High-Level Synthesis concepts such as loop pipelining, array partitioning, and AXI interface management to translate C++ algorithms into synthesisable hardware. Learning to use Vitis HLS 2021.2 and the Xilinx ZCU102 platform is an important outcome, as both tools are representative of the FPGA accelerator design flow used in the embedded systems and data-centre industry.

Acknowledging/Declaring the Use of GAI

Declare

References

[1] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler, "Efficient transformers: A survey," *CoRR*, vol. abs/2009.06732, 2020. DOI: 10.48550/arXiv.2009.06732 [Online]. Available: https://arxiv.org/abs/2009.06732

[2] P. Zhang, G. Zeng, T. Wang, and W. Lu, "TinyLlama: An open-source small language model," *CoRR*, vol. abs/2401.02385, 2024. DOI: 10.48550/arXiv.2401.02385 [Online]. Available: https://arxiv.org/abs/2401.02385

[3] AMD Xilinx, *Vitis high-level synthesis user guide (UG1399)*, Version 2021.2. Accessed: Mar. 24, 2026, AMD Inc., 2021. [Online]. Available: https://docs.amd.com/r/2021.2-English/ug1399-vitis-hls

[4] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, Curran Associates, Inc., 2017, pp. 5998–6008. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[5] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," *CoRR*, vol. abs/2307.09288, 2023. DOI: 10.48550/arXiv.2307.09288 [Online]. Available: https://arxiv.org/abs/2307.09288

[6] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *CoRR*, vol. abs/2104.09864, 2021. DOI: 10.48550/arXiv.2104.09864 [Online]. Available: https://arxiv.org/abs/2104.09864

[7] B. Zhang and R. Sennrich, "Root mean square layer normalization," *CoRR*, vol. abs/1910.07467, 2019. DOI: 10.48550/arXiv.1910.07467 [Online]. Available: https://arxiv.org/abs/1910.07467

[8] N. Shazeer, "GLU variants improve transformer," *CoRR*, vol. abs/2002.05202, 2020. DOI: 10.48550/arXiv.2002.05202 [Online]. Available: https://arxiv.org/abs/2002.05202

[9] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, Singapore: Association for

Computational Linguistics, 2023, pp. 4895–4901. DOI: 10.18653/v1/2023.emnlp-main.298

[10] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, and K. Keutzer, "I-BERT: Integer-only BERT quantization," *CoRR*, vol. abs/2101.01321, 2021. DOI: 10.48550/arXiv.2101.01321 [Online]. Available: https://arxiv.org/abs/2101.01321

[11] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating long sequences with sparse transformers," *CoRR*, vol. abs/1904.10509, 2019. DOI: 10.48550/arXiv.1904.10509 [Online]. Available: https://arxiv.org/abs/1904.10509

[12] C. Yun, S. Bhojanapalli, A. S. Rawat, S. J. Reddi, and S. Kumar, "Are transformers universal approximators of sequence-to-sequence functions?" *CoRR*, vol. abs/1912.10077, 2019. DOI: 10.48550/arXiv.1912.10077 [Online]. Available: https://arxiv.org/abs/1912.10077

[13] J. Qiu, H. Ma, O. Levy, S. W.-t. Yih, S. Wang, and J. Tang, "Blockwise self-attention for long document understanding," *CoRR*, vol. abs/1911.02972, 2019. DOI: 10.48550/arXiv.1911.02972 [Online]. Available: https://arxiv.org/abs/1911.02972

[14] K. Choromanski et al., "Rethinking attention with performers," *CoRR*, vol. abs/2009.14794, 2020. DOI: 10.48550/arXiv.2009.14794 [Online]. Available: https://arxiv.org/abs/2009.14794

[15] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, "Transformers are RNNs: Fast autoregressive transformers with linear attention," *CoRR*, vol. abs/2006.16236, 2020. DOI: 10.48550/arXiv.2006.16236 [Online]. Available: https://arxiv.org/abs/2006.16236

[16] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," *CoRR*, vol. abs/2208.07339, 2022. DOI: 10.48550/arXiv.2208.07339 [Online]. Available: https://arxiv.org/abs/2208.07339

[17] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," *CoRR*, vol. abs/2211.10438, 2022. DOI: 10.48550/arXiv.2211.10438 [Online]. Available: https://arxiv.org/abs/2211.10438

[18] J. de Fine Licht, M. Besta, S. Obbard, and T. Hoefler, "Transformations of high-level synthesis codes for high-performance computing," *CoRR*, vol. abs/1805.08288, 2018. DOI: 10.48550/arXiv.1805.08288 [Online]. Available: https://arxiv.org/abs/1805.08288

Chapter A HLS Kernel Source Code

This appendix presents the complete Vitis HLS C++ source for the two top-level `forward` kernels discussed in section 4.8. Listing A.1 is the baseline prototype synthesised at prototype dimensions ($N_L = 2$, `dim` $= 8$, `seq_len` $= 16$), implementing the attention sub-block only. Listing A.2 is the sixth-iteration full-scale kernel synthesised in chapter 5, implementing one complete TinyLlama-1.1B decode step at full model dimensions ($N_L = 22$, `dim` $= 2048$, `seq_len` $= 2048$).

A.1 `forward.cpp` — Baseline Prototype

```
#include "forward.h"
#include "config.h"
#include <cstring>

extern "C" void forward(
  Transformer<dim,hidden_dim,n_layers,n_heads,n_kv_heads,
              vocab_size,seq_len,GS>*transformer,
  int token,int pos,
  float key_cache[n_layers*seq_len*((dim*n_kv_heads)/n_heads)],
  float value_cache[n_layers*seq_len*((dim*n_kv_heads)/n_heads)],
  float*out)
{
#pragma HLS INTERFACE m_axi port=transformer offset=slave bundle=
    gmem0
#pragma HLS INTERFACE m_axi port=out offset=slave bundle=gmem1
#pragma HLS INTERFACE m_axi port=key_cache offset=slave bundle=gmem0
     depth=(n_layers*seq_len*((dim*n_kv_heads)/n_heads))
#pragma HLS INTERFACE m_axi port=value_cache offset=slave bundle=
    gmem0 depth=(n_layers*seq_len*((dim*n_kv_heads)/n_heads))

  auto w=&transformer->weights;
  constexpr int UNROLL_FACTOR=16;

  static float x[config.dim];
  static float xb[config.dim];
  static float xb2[config.dim];
  static float hb[config.hidden_dim];
  static float hb2[config.hidden_dim];
```

```
  static QuantizedTensor<config.dim>xq;
  static QuantizedTensor<config.hidden_dim>hq;
  static float q[config.dim];
  static float k[(config.dim*config.n_kv_heads)/config.n_heads];
  static float v[(config.dim*config.n_kv_heads)/config.n_heads];
  static float att[config.n_heads*config.seq_len];

#pragma HLS ARRAY_PARTITION variable=q   cyclic factor=UNROLL_FACTOR
#pragma HLS ARRAY_PARTITION variable=k   cyclic factor=UNROLL_FACTOR
#pragma HLS ARRAY_PARTITION variable=v   cyclic factor=UNROLL_FACTOR
#pragma HLS ARRAY_PARTITION variable=att cyclic factor=UNROLL_FACTOR
#pragma HLS ARRAY_PARTITION variable=hq.q cyclic factor=
    UNROLL_FACTOR
#pragma HLS ARRAY_PARTITION variable=hq.s cyclic factor=
    UNROLL_FACTOR
#pragma HLS ARRAY_PARTITION variable=xq.q cyclic factor=
    UNROLL_FACTOR
#pragma HLS ARRAY_PARTITION variable=xq.s cyclic factor=
    UNROLL_FACTOR
#pragma HLS ARRAY_PARTITION variable=hb  type=cyclic factor=
    UNROLL_FACTOR
#pragma HLS ARRAY_PARTITION variable=hb2 type=cyclic factor=
    UNROLL_FACTOR
#pragma HLS ARRAY_PARTITION variable=x   type=cyclic factor=
    UNROLL_FACTOR
#pragma HLS ARRAY_PARTITION variable=xb  type=cyclic factor=
    UNROLL_FACTOR
#pragma HLS ARRAY_PARTITION variable=xb2 type=cyclic factor=
    UNROLL_FACTOR

  constexpr int kv_dim=(config.dim*config.n_kv_heads)/config.n_heads
      ;
  constexpr int kv_mul=config.n_heads/config.n_kv_heads;
  constexpr int head_size=dim/config.n_heads;

  std::memcpy(x,w->token_embedding_table+token*dim,dim*sizeof(float)
      );

main_forward_loop:
  for (int l=0; l<config.n_layers; l++) {
    rmsnorm<dim>(xb,x,w->rms_att_weight+l*dim);

    quantize(&xq,xb,GS);
    matmul<dim,dim>(q,xq.q,xq.s,(w->wq+l)->q,(w->wq+l)->s);
    matmul<dim,kv_dim>(k,xq.q,xq.s,(w->wk+l)->q,(w->wk+l)->s);
```

```
    matmul<dim,kv_dim>(v,xq.q,xq.s,(w->wv+l)->q,(w->wv+l)->s);

    int loff=l*config.seq_len*kv_dim;
    float*key_cache_row=key_cache+loff+pos*kv_dim;
    float*value_cache_row=value_cache+loff+pos*kv_dim;
    std::memcpy(key_cache_row,k,kv_dim*sizeof(*key_cache_row));
    std::memcpy(value_cache_row,v,kv_dim*sizeof(*value_cache_row));

    int h;
multihead_attention:
    for (h=0; h<n_heads; h++) {
      const int q_offset=h*head_size;
      const int att_offset=h*seq_len;
iterate:
      for (int t=0; t<=pos; t++) {
#pragma HLS PIPELINE
#pragma HLS loop_tripcount min=1 max=16 avg=8
        const int key_offset=loff+t*kv_dim+(h/kv_mul)*head_size;
        float score=0.0f;
        for (int i=0; i<head_size; i++) {
#pragma HLS unroll
          score+=q[i+q_offset]*key_cache[i+key_offset];
        }
        score/=sqrtf(head_size);
        att[t+att_offset]=score;
      }

      softmax<16>(att+att_offset,pos+1);

      const int xb_offset=h*head_size;
      memset(xb+xb_offset,0,head_size*sizeof(float));
acc:
      for (int t=0; t<=pos; t++) {
#pragma HLS loop_tripcount min=1 max=16 avg=8
#pragma HLS PIPELINE
        const int v_offset=loff+t*kv_dim+(h/kv_mul)*head_size;
        float a=att[t+att_offset];
acc_inner:
        for (int i=0; i<head_size; i++) {
#pragma HLS unroll
          xb[i+xb_offset]+=a*value_cache[i+v_offset];
        }
      }
    }
```

```
    // Output projection (wo) is omitted; softmax output dimension
       == dim.
    // quantize(&xq, xb, GS);
    // matmul<dim, dim>(xb2, xq.q, xq.s, (w->wo+l)->q, (w->wo+l)->s)
       ;

residual:
    for (int i=0; i<dim; i++) {
#pragma HLS UNROLL factor=64 skip_exit_check
      x[i]+=xb[i];
    }
  } // FFN block, final RMSNorm, and LM head omitted in this
     prototype.

  std::memcpy(out,x,dim*sizeof(float));
}
```

Listing A.1: forward.cpp: baseline prototype kernel.

A.2 FFN_update3.cpp — Full-Scale Kernel

```
#include "FFN_update3.h"
#include "config_FFN_update3.h"
#include <string.h>
#include <math.h>

extern "C" void forward(
  Transformer<dim,hidden_dim,n_layers,n_heads,n_kv_heads,
              vocab_size,seq_len,GS>*transformer,
  int token,int pos,
  int8_t*key_cache,float*key_scales,
  int8_t*value_cache,float*value_scales,
  float*out)
{
  constexpr int kv_dim=(dim*n_kv_heads)/n_heads;
  constexpr int kv_mul=n_heads/n_kv_heads;
  constexpr int head_size=dim/n_heads;
  constexpr int TOTAL_ELEMENTS=n_layers*seq_len*kv_dim;
  constexpr int TOTAL_SCALES=n_layers*seq_len*(kv_dim/GS);

#pragma HLS INTERFACE m_axi port=transformer offset=slave bundle=
    gmem0 depth=1
#pragma HLS INTERFACE m_axi port=key_cache offset=slave bundle=gmem1
     depth=TOTAL_ELEMENTS
```

```
22 #pragma HLS INTERFACE m_axi port=key_scales offset=slave bundle=
      gmem1 depth=TOTAL_SCALES
23 #pragma HLS INTERFACE m_axi port=value_cache offset=slave bundle=
      gmem2 depth=TOTAL_ELEMENTS
24 #pragma HLS INTERFACE m_axi port=value_scales offset=slave bundle=
      gmem2 depth=TOTAL_SCALES
25 #pragma HLS INTERFACE m_axi port=out offset=slave bundle=gmem3
26
27   auto w=&transformer->weights;
28
29   if (token<0||token>=vocab_size) token=0;
30   if (pos<0)                      pos=0;
31   if (pos>=seq_len)               pos=seq_len-1;
32
33   static float x[dim];
34   static float xb[dim];
35   static float xb2[dim];
36   static float hb[hidden_dim];
37   static float hb2[hidden_dim];
38   static QuantizedTensor<dim>xq;
39   static QuantizedTensor<hidden_dim>hq;
40   static float   q[dim];
41   static float   k[kv_dim];
42   static float   v[kv_dim];
43   static int32_t att[seq_len];
44   static int8_t  att_probs[seq_len];
45
46 #pragma HLS ARRAY_PARTITION variable=x         cyclic factor=16
47 #pragma HLS ARRAY_PARTITION variable=xb        cyclic factor=16
48 #pragma HLS ARRAY_PARTITION variable=xb2       cyclic factor=16
49 #pragma HLS ARRAY_PARTITION variable=hb        cyclic factor=16
50 #pragma HLS ARRAY_PARTITION variable=hb2       cyclic factor=16
51 #pragma HLS ARRAY_PARTITION variable=xq.q      cyclic factor=16
52 #pragma HLS ARRAY_PARTITION variable=xq.s      cyclic factor=4
53 #pragma HLS ARRAY_PARTITION variable=hq.q      cyclic factor=16
54 #pragma HLS ARRAY_PARTITION variable=hq.s      cyclic factor=4
55 #pragma HLS ARRAY_PARTITION variable=q         cyclic factor=16
56 #pragma HLS ARRAY_PARTITION variable=k         cyclic factor=16
57 #pragma HLS ARRAY_PARTITION variable=v         cyclic factor=16
58 #pragma HLS ARRAY_PARTITION variable=att       cyclic factor=16
59 #pragma HLS ARRAY_PARTITION variable=att_probs cyclic factor=16
60
61   // Embedding lookup: dequantize from q_tokens.
62   {
63     const int emb_offset=token*dim;
```

```
    const int scale_base=token*(dim/GS);
embed_lookup:
    for (int i=0; i<dim; i++) {
#pragma HLS PIPELINE II=1
      x[i]=(float)w->q_tokens[0].q[emb_offset+i]
*w->q_tokens[0].s[scale_base+i/GS];
    }
  }

main_forward_loop:
  for (int l=0; l<n_layers; l++) {
#pragma HLS loop_tripcount min=22 max=22

    // Attention RMSNorm and QKV projections
    rmsnorm<dim>(xb,x,w->rms_att_weight+l*dim);
    quantize(&xq,xb);
    matmul<dim,dim>(q,xq.q,xq.s,(w->wq+l)->q,(w->wq+l)->s);
    matmul<dim,kv_dim>(k,xq.q,xq.s,(w->wk+l)->q,(w->wk+l)->s);
    matmul<dim,kv_dim>(v,xq.q,xq.s,(w->wv+l)->q,(w->wv+l)->s);
    // TODO: RoPE positional encoding omitted.

    // KV cache write (quantized)
    int loff_data=l*seq_len*kv_dim;
    int loff_scale=l*seq_len*(kv_dim/GS);
    QuantizedTensor<kv_dim>k_quantized,v_quantized;
    quantize(&k_quantized,k);
    quantize(&v_quantized,v);
    memcpy(key_cache+loff_data+pos*kv_dim,
           k_quantized.q,kv_dim*sizeof(int8_t));
    memcpy(key_scales+loff_scale+pos*(kv_dim/GS),
           k_quantized.s,(kv_dim/GS)*sizeof(float));
    memcpy(value_cache+loff_data+pos*kv_dim,
           v_quantized.q,kv_dim*sizeof(int8_t));
    memcpy(value_scales+loff_scale+pos*(kv_dim/GS),
           v_quantized.s,(kv_dim/GS)*sizeof(float));

multihead_attention:
    for (int h=0; h<n_heads; h++) {
#pragma HLS loop_tripcount min=32 max=32
      const int q_offset=h*head_size;

      float max_abs=0.0f;
q_find_max:
      for (int i=0; i<head_size; i++) {
#pragma HLS PIPELINE II=1
```

```
109         float val=fabs(q[i+q_offset]);
110         if (val>max_abs) max_abs=val;
111       }
112       float q_scale_val=(max_abs==0.0f) ? 1.0f : (max_abs/127.0f);
113       float q_inv_scale=1.0f/q_scale_val;
114
115       int8_t q_int8[head_size];
116 #pragma HLS ARRAY_PARTITION variable=q_int8 complete
117 q_quantize:
118       for (int i=0; i<head_size; i++) {
119 #pragma HLS UNROLL
120         float   qv=q[i+q_offset]*q_inv_scale;
121         int32_t qi=(int32_t)(qv+(qv>=0.0f ? 0.5f :-0.5f));
122         if (qi>127) qi=127;
123         if (qi<-127) qi=-127;
124         q_int8[i]=(int8_t)qi;
125       }
126
127 score_loop:
128       for (int t=0; t<=pos; t++) {
129 #pragma HLS loop_tripcount min=1 max=2048 avg=1024
130         const int kv_head=h/kv_mul;
131         const int data_offset=loff_data+t*kv_dim+kv_head*head_size;
132         const int scale_offset=loff_scale+t*(kv_dim/GS)+kv_head;
133         float k_scale=key_scales[scale_offset];
134
135         int8_t k_buffer[head_size];
136 #pragma HLS ARRAY_PARTITION variable=k_buffer complete
137 k_load:
138         for (int i=0; i<head_size; i++) {
139 #pragma HLS PIPELINE II=1
140           k_buffer[i]=key_cache[data_offset+i];
141         }
142
143         int32_t score_int=0;
144 dot_qk:
145         for (int i=0; i<head_size; i++) {
146 #pragma HLS UNROLL
147           score_int+=(int32_t)q_int8[i]*(int32_t)k_buffer[i];
148         }
149         att[t]=(int32_t)(((float)score_int)*k_scale);
150       }
151
152       float remaining_scale=q_scale_val/sqrtf((float)head_size);
153       softmax_ibert<seq_len>(att,att_probs,pos+1,remaining_scale);
```

```
154
155         const int xb_offset=h*head_size;
156         float xb_accum[head_size];
157 #pragma HLS ARRAY_PARTITION variable=xb_accum complete
158 init_accum:
159         for (int i=0; i<head_size; i++) {
160 #pragma HLS UNROLL
161           xb_accum[i]=0.0f;
162         }
163
164 value_loop:
165         for (int t=0; t<=pos; t++) {
166 #pragma HLS loop_tripcount min=1 max=2048 avg=1024
167           const int kv_head=h/kv_mul;
168           const int v_data_offset=loff_data+t*kv_dim+kv_head*head_size
                ;
169           const int v_scale_offset=loff_scale+t*(kv_dim/GS)+kv_head;
170           float v_scale=value_scales[v_scale_offset];
171
172           int8_t v_buffer[head_size];
173 #pragma HLS ARRAY_PARTITION variable=v_buffer complete
174 v_load:
175           for (int i=0; i<head_size; i++) {
176 #pragma HLS PIPELINE II=1
177             v_buffer[i]=value_cache[v_data_offset+i];
178           }
179
180           int8_t prob_int8=att_probs[t];
181           float combined_scale=(1.0f/127.0f)*v_scale;
182 v_accumulate:
183           for (int i=0; i<head_size; i++) {
184 #pragma HLS UNROLL
185             int32_t product=(int32_t)prob_int8*(int32_t)v_buffer[i];
186             xb_accum[i]+=((float)product)*combined_scale;
187           }
188         }
189
190 write_head:
191         for (int i=0; i<head_size; i++) {
192 #pragma HLS UNROLL
193           xb[i+xb_offset]=xb_accum[i];
194         }
195       }
196       // O projection: mix attention head outputs via wo
197       quantize(&xq,xb);
```

```
    matmul<dim,dim>(xb2,xq.q,xq.s,(w->wo+l)->q,(w->wo+l)->s);

residual1:
    for (int i=0; i<dim; i++) {
#pragma HLS PIPELINE II=1
#pragma HLS UNROLL factor=16
      x[i]+=xb2[i];
    }

    // FFN block
    rmsnorm<dim>(xb,x,w->rms_ffn_weight+l*dim);
    quantize(&xq,xb);
    matmul<dim,hidden_dim>(hb,xq.q,xq.s,(w->w1+l)->q,(w->w1+l)->s);
    matmul<dim,hidden_dim>(hb2,xq.q,xq.s,(w->w3+l)->q,(w->w3+l)->s);

    float hb_out[hidden_dim];
#pragma HLS ARRAY_PARTITION variable=hb_out cyclic factor=16
swi_glu:
    for (int i=0; i<hidden_dim; i++) {
#pragma HLS PIPELINE II=1
#pragma HLS UNROLL factor=16
      float val=hb[i];
      val*=(1.0f/(1.0f+expf(-val)));
      val*=hb2[i];
      hb_out[i]=val;
    }
    memcpy(hb,hb_out,hidden_dim*sizeof(float));

    quantize(&hq,hb);
    matmul<hidden_dim,dim>(xb,hq.q,hq.s,(w->w2+l)->q,(w->w2+l)->s);

residual2:
    for (int i=0; i<dim; i++) {
#pragma HLS PIPELINE II=1
#pragma HLS UNROLL factor=16
      x[i]+=xb[i];
    }
  }

  // Final RMSNorm and LM head run on the ARM Cortex-A53 in host
      software.
  memcpy(out,x,dim*sizeof(float));
}
```

Listing A.2: FFN_update3.cpp: full-scale kernel.